

OCR A-Level Computer Science Project H446

Name: Suprit Bhatti

Candidate Number: 2389

School: Coundon Court

Title of Project: Treasure Chase

Contents

Analysis.....	5
The Problem and stakeholders	5
How the problem can be solved using Computational methods	6
Thinking Abstractly-	6
Thinking Ahead-	6
Thinking Procedurally and Decomposition-	7
Thinking Logically-	7
Thinking Concurrently-	7
Researching Alternatives.....	8
Interview	11
Transcript.....	11
Features of Proposed Solution.....	14
Success Criteria	16
Limitations	19
Hardware and Software Requirements	20
Design	21
Explaining Each Module	22
Features.....	25
Game Start	25
Play Menu.....	25
Levels	26
Scoreboard.....	26
Shop	27
Settings Menu	27
Gameplay.....	28
Sprites	28
Controls	30
Algorithms.....	31

Class Diagrams	47
Test Data – Iterative Development.....	50
Test Data – Post Development Testing.....	53
Development	58
19/11/2024 – The Start	58
21/11/2024 – Giving the buttons functionality	61
22/11/2024 – Creating gameplay.....	64
28/11/2024 – Adding Gravity and Collisions	67
29/11/2024 – Adding coins to the gameplay	69
3/12/2024 – Adding lives to the game	71
6/12/2024 – Adding a respawn feature.....	72
7/12/2024 – Creating the weapon for the user	73
10/12/2024 – Creating the Enemy.....	74
13/12/2024 – Fixing the enemy class (Part 1).....	75
22/12/2024 - Fixing the enemy class (Part 2)	76
28/12/2024 – Creating final few features (Part 1)	77
30/12/2024 – Creating final few features (Part 2)	79
10/01/2025 – Adding the remaining ending screens	81
10/01/2025 – Adding and fixing some gameplay features	83
16/01/2025 – Linking the Main Menu and level together	85
17/01/2025 – Creating a database to store scores and coins that the player gets.....	87
18/01/2025 – Creating the personal leaderboard.....	89
23/01/2025 – Creating a counter to show the coins	90
24/01/2025 – Creating the Shop (Part 1).....	91
26/01/2025 – Creating the shop (Part 2)	94
28/01/2025 – Fixing the new bugs.....	96
29/01/2025 – Another Big Problem	98
30/01/2025 – Fixing the remaining shop bugs	100
05/02/2025 – Creating weapon sounds.....	101

08/02/2025 – Creating the settings menu to toggle the sounds on or off	103
14/02/2025 – Fixing the toggle	105
16/02/2025 – Bug with the lives	107
Evaluation	109
Post Development Testing	109
Going through the usability features with the stakeholder	116
Menus	116
Main Game	120
Controls	121
Comparing the Solution to the Success Criteria	122
How the criteria was met in some areas	124
How the criteria was partially met in some areas and how can this be addressed ...	125
Why the criteria wasn't met in some areas and how can this be addressed	125
Maintenance	128
Bugs that currently exist	129
How limitations could be managed	129
Final Code	132
Main_Menu.py	132
Play.py	137
CL.py	142
Settings.py	146
Database.py	147

Analysis

The Problem and stakeholders

I will be making a 2D platformer game that consists of multiple levels as well as AI. The player in the game will have one life, so if they die, they must start over again from the very beginning. The AI within the game will consist of various enemies, ranging from weaker enemies that can be dealt with quickly, all the way to bosses that will be much more difficult to take care of. The game will also utilise a leaderboard, which will require a scoring system based on how well the player does. The leaderboard will consist of all previous attempts, with the 5 highest scoring runs being shown. This game will also have different difficulties, with the hardest difficulty bringing the biggest scores. There will also be a currency that can be used to buy better weapons, armour, and power ups, with harder difficulties rewarding more currency. The game will also have a health and damage system, with the user being able to view their health as the game goes on.

The game will have a target audience of people who are 7-16 years old. It will use a keyboard and mouse, with the keyboard used to control the player's movement and the mouse allows the user to attack enemies. This will help to make the game fairly accessible, although some of my users may struggle as they may have never played video games before. As a result, I might choose to give the users the ability to adjust the controls, so they can change them to what they feel will suit their needs. I might also choose to give the users the ability to adjust the difficulty, so they can play at whatever difficulty they wish. My stakeholder for the game will be Ben, who is a big fan of video games in general and someone who would like a game like this. He will be the person who I question during the interview and the person who's needs I must factor in during the development of the program.

How the problem can be solved using Computational methods

Thinking Abstractly-

My game will feature abstraction within the game, so it becomes more user-friendly. Abstraction is the idea of removing all unnecessary details by deciding what is/isn't important. Ways that I'll apply abstraction include:

- In the game menus – What the user sees is all that the user really needs to see. This will include the buttons to play, access the shop and access the settings. There is zero need to add anything else to the main menu otherwise it might start to become a bit messy and hard to navigate.
- Making the game 2D – It will simply take too long for me to produce a 3D version of this game, so I am making a 2D version that will still be playable for anyone.
- Showing health bar of player – This ensures that the player will be aware of how much health they will have as they play the game.
- Having a pause button – Allowing users to press P to pause the game if they have to do something else such as Homework, chores etc.

Thinking Ahead-

Thinking ahead is the process of identifying potential problems and solving them before they arise. This is important to ensure that the designing and programming is as smooth as possible by preventing as many problems as possible. Some ways that I'll apply this includes:

- The colour scheme – I cannot just have it in black and white otherwise it'll be a bit boring to look at as it's only 2 colours. However, it also cannot have too many colours otherwise it may be a bit too distracting.
- Another thing could be how I use the camera – I'm not sure if I should use a camera that follows the floor or a camera that follows the player. I will choose to have the camera following the player as it seems to be a better option compared to other options such as having the camera following the player.

Thinking Procedurally and Decomposition-

Thinking Procedurally is a good way to break down a big problem into smaller, more manageable problems, making it easier to solve big problems.

- One effective way to decompose my game is to use Object Oriented Programming (OOP). This involves making classes that can be used for inheritance, which allows newer classes to be created with some of the methods of pre-existing classes. It also inhibits decomposition with the use of methods, which are the procedures that make up a class in OOP.
- Character movement regarding both the player and the AI enemies will both be similar, with the speed of the AI changing based on difficulty chosen.

Thinking Logically-

Thinking Logically is the process of making decisions based on what the user does.

- One way I will implement this is with the difficulty selection. This is because the AI will behave differently based on difficulty. The easier levels will have weaker and less aggressive enemies, whereas harder difficulties will have tougher, stronger, more aggressive enemies, increasing the challenge that comes with it. Harder difficulties will also result in improved rewards for the player.

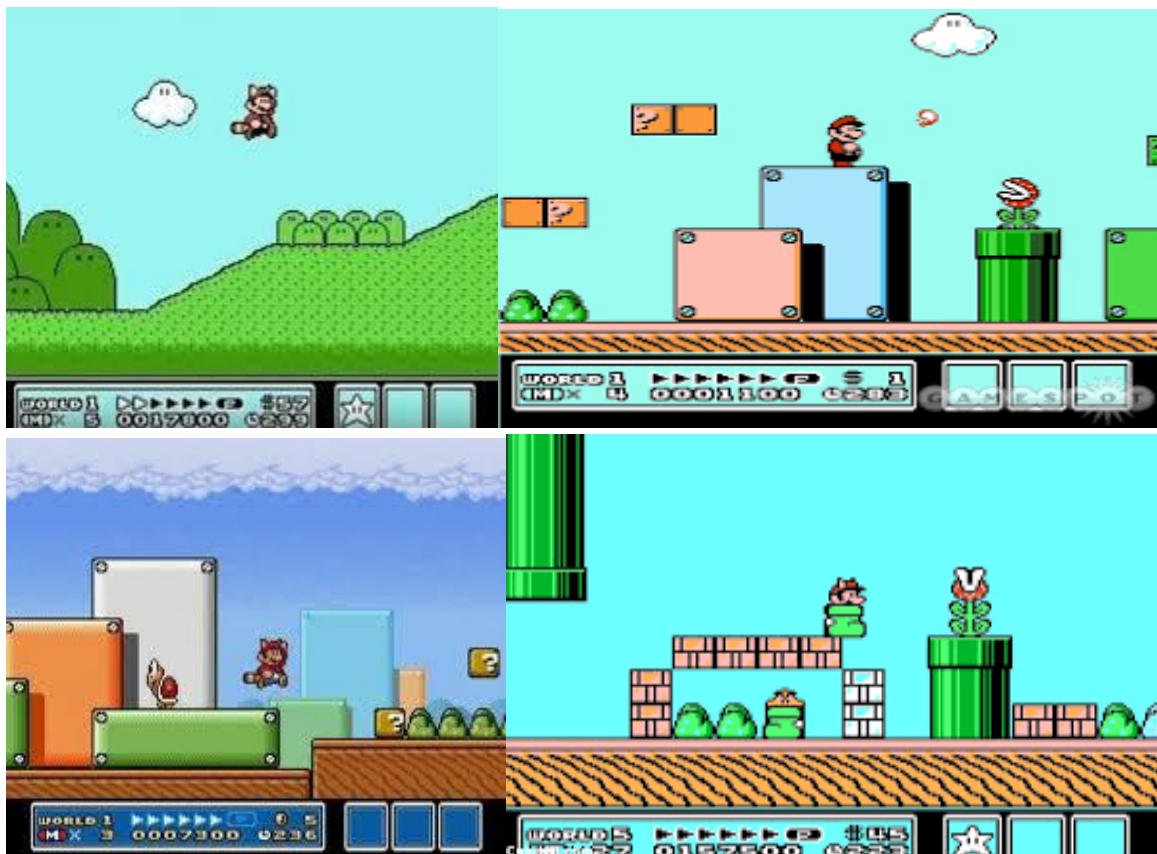
Thinking Concurrently-

Thinking Concurrently is the process of completing more than one task at a given time. This can help me improve the efficiency of my code.

- The screen will update player movements and enemy attacks simultaneously should they happen at the same time.

Researching Alternatives

For my research, I will be looking at Super Mario Bros 3, which is an extremely popular 2D side-scrolling platformer released in 1988. It has been remade several times, once in 1993 for the Super NES as a part of Super Mario All-Stars, then for the Game Boy Advance as Super Mario Advance 4 – Super Mario Bros 3 in 2003, and then was rereleased on the Virtual Console service on the Wii U and Nintendo 3DS and was included on the NES Classic Mini. The game can also be emulated to play on other devices such as Smartphones and Laptops. It is regarded as one of the best video games of all time due to its challenging gameplay. The objective of the game is to control either Mario or Luigi, and they must save Princess Toadstool and the rulers of 7 different kingdoms from Bowser whilst having new abilities such as Flight and Sliding down slopes. The player can also use items that have magical powers, and they can stomp on enemies. The player also has power ups they can use, such as the Super Leaf and Tanooki Suit, which allows the player to fly and float. This game is based on a 2D platform which I will use in my game.



Power-Ups – In this game, there are many power-ups within the game that can be used to help the player go further within the game. Some of these power-ups grants the player with

the ability to fly, float, shoot fireballs, become invincible, and much more. Examples of these power-ups includes (sprites shown):

- Fire Flower – Shoot fireballs 
- Hammer Suit – Throw hammers 
- Starman – Become invincible 
- P-Wing – Flight 
- Super Leaf – Tail whip enemies and slowly descend in the sky, can also fly temporarily 

These power-ups can be found in specific levels of the game and are found in the lucky blocks on the map.

Levels – In Super Mario Bros 3, there are 8 distinct kingdoms that Mario explores, which results in there being around 90 levels, excluding secret levels. All of these kingdoms have distinctive features and themes, making each world unique to one another. In each world, there are different paths that the player can take, meaning that they could perhaps finish worlds with faster routes, or if they wanted to, they could experience all the levels in a world. These different paths can be unlocked using coins.



Lives – In Super Mario Bros 3, you start with 3 lives, and this can be increased all the way to 100 lives as you further progress through the game. If you lose all of your lives, you have to restart all over again. You can progress through the story to gain the extra lives. If an enemy

hits you, you instantly die and lose a life. If you activate the Starman power-up, you will not lose any lives for as long as the power-up is active.

Controls – In Super Mario Bros 3, the controls would've been optimised to play on the devices it was made for. It's worth noting that my game will be running on PC, so the controls that I end up using would ideally be optimised for use on a PC. Because it was released on several different consoles, there will be many varying experiences for players as they use different methods to play the game.

What I Liked from the Game -

Game Feature	Justification
Ability to earn Lives	This is great as it helps the user to build up on the number of lives they have, which can help them increase the number of chances they have to complete a level.
Power-ups	I like this because it helps the user progress through the level faster by using special abilities that are unique to one another.
Ability to unlock different routes	I like this as it allows the user to take different paths as they play the game, and if the user is limited on time, they could unlock the quicker paths using coins.

What I disliked from the Game -

Game Feature	Justification
Lack of Health	Not having a health bar at all means that the user is always one shot away from losing a life, which I personally feel is very harsh. I would try to add a health system to make the game much more accessible
Number of Levels	I don't mind there being many levels. However, I feel that there are way too many levels within the game, and I feel that this can be off-putting for many people.
Difficulty	Whilst it is good to have a challenge, one problem I found was that there didn't seem to be any way to adjust difficulty, which I personally dislike as it there is no way to make the game, which is already quite difficult, easier for those who need it.

Interview

For my interview, I will be interviewing Ben, who is a huge fan of video games. I will ask him questions so I can get a better idea of what I need to implement in my game. This is because he fits into the target audience perfectly, so it will give me an idea of what that target audience would like to see in this type of game. By doing this, I will have an idea of what I should add to my initial idea and what I should remove from it. In this interview, I will try to find what he likes and dislikes about other 2D platformer games.

Transcript

Suprit

Ben

How are you feeling today?

Quite good, thanks.

What do you like about 2D platformers?

I like how unique and addictive they are compared to other video games. A lot of the 2D platformers that I've played previously were also difficult whilst still being enjoyable and unique compared to other video games.

What do you dislike about 2D platformers?

I dislike the graphics of a lot of 2D platformers – a lot of them seem to be lacking in the quality of the environment, although some of these games are older so it isn't necessarily a concern for me.

What features do you think can help make a game unique?

I think that the game should have distinctive features compared to other 2D platformers. Perhaps, it could take ideas from other genres and try to implement them into the game.

Would you like to see a tutorial at the start that can help unpractised players?

Yes, I think that would be a particularly clever idea.

How could I make my game enjoyable?

I think that making a unique game that isn't too difficult and that also has a lot to do is an effective way to make any game enjoyable to play.

Would you like to see special mechanics that are level-specific?

Yes, especially in later levels, I think having some level-specific mechanics will be a particularly effective way to increase difficulty.

Would you want the ability to set different difficulties?

Yes, because it would allow people of different skill levels to access the game, making it playable for everyone. However, I do feel that the rewards should definitely be adjusted based on difficulty.

Is sound important to you?

Yes, as I feel that it is especially important for gaming. I feel that background music isn't necessary, but it can be a decent addition.

Would you like to have a shop where you can buy items to improve your character's power?

Yes, as it would help to incentivise players to carry on playing the game.

Do you like the idea that if you die, the game ends completely regardless of how much progress you have made?

It depends – it would be very harsh to implement this in the earlier levels, but it's also a very good way to add further difficulty, especially in the later levels.

Would you like to see an option to buy power-ups to use?

Yes – I feel that power-ups can be extremely useful for the player. The player should be able to buy these items from the shop and activate them whenever they feel that they need them. They should last for 20 seconds.

Would you like to see a timer on each level?

Yes – I believe that this can also be an effective way to implement difficulty into any game. The timer should last for around 5 minutes on an average level, and the player will fail the level if they do not complete it in time. If you want, you can put longer times on levels that you feel need longer times to complete.

Would you want a way to pause the game?

Yes, this can be beneficial for many users who may need to do chores, homework, or anything else that can disrupt their gaming experience.

Would you like a scoring system?

Yes, and there should also be a leaderboard that shows the 5 best scores from the user.

Should I add coins that can be earned through the levels?

Yes, the player should be able to earn coins as they play the levels and also earn them as a reward at the end of each level.

What should the scoring system take into consideration when scoring a player's run of any level?

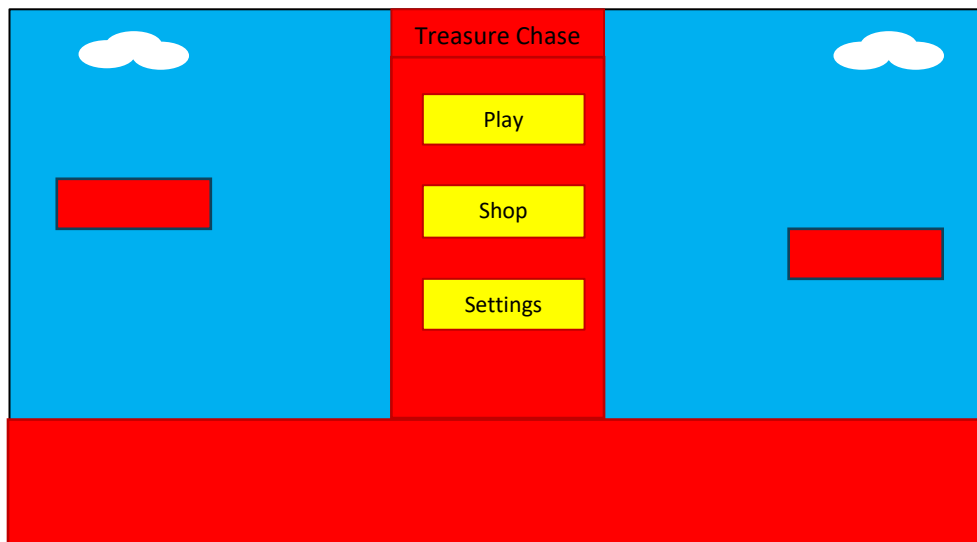
It should consider how difficult the level is, how many coins the player picked up, how long they took to finish the level, and if they used power-ups or not. There should also be a leaderboard that shows the score of the 5 highest earning saves that the player has achieved.

Would you like to see anything else that we haven't discussed?

I would like it if there were different themes for some of the levels. I would also like to see a final boss on the last level.

Features of Proposed Solution

- My game will have 3 power-ups that can be brought from the in-game shop, one which increases the user's damage output, another which gives the user the ability to reduce the amount of damage they take, and a third which makes the user invincible. I'm not too sure what to name these for now.
- After buying these power-ups, they can be used whenever the player feels that they need to use them.
- Some enemies will move randomly forwards and backwards. These enemies will have the ability to shoot at the player. Other enemies will have a melee attack, which means that they will run towards the user in order to attack the player.
- My lives system will be similar to Super Mario Bros 3, where there will be a lives counter that will affect every level in the game. However, I will also use a health system, which will allow the player to survive more than 1 hit before they lose a life, compared to Super Mario Bros 3, where the player loses a life the moment they get hit.
- I intend to only make 5 levels for the whole game. This is because I know that I simply will not have the time to make a similar number of levels compared to Super Mario Bros 3. I also feel that I could condense my story into 5 levels.
- I might have distinct levels for different themes, although I only have 5 levels to work with, so it may be a bit difficult to come up with different themes.
- There will be a main menu that could look something like this:



- This is only a rough drawing of my idea of a menu, so this will likely look different to what I could end up doing.
- The player can use coins earned from levels to buy items such as better weapons, armour, and power-ups.

- In terms of controls, I will use A to move to the left, D to move to the right, the spacebar to jump, left click to shoot, P to pause the game, I to interact with certain objects, G to activate powerups, and R to reload.
- This is because it would help to make the experience enjoyable for the player. Having poorly placed controls can dampen the experience of any game.
- As part of the whole combat system, which does include the health system, the user will be able to use weapons to shoot the enemies from a distance. I plan on making 3-5 different weapons for the player to use.
- I will implement sound effects in my game as I feel that it can enhance the user experience.
- I also plan on putting different armour that helps improve the player's defence against enemies.

Success Criteria

Requirement	Justification
At least 5 levels	Having 5 levels is a reasonable number of levels to start with and I feel that this will be sufficient enough.
Start Menu has 3 different options	As shown in the draft in the Proposed Solution – there should be 3 different sections of the Start Screen – Play, which allows the user to either start a new game or continue their progress, Shop, where they can buy items, and Settings, where the player can adjust the settings based on their preferences.
Ability to change difficulty in settings	Can give players an easier/more difficult experience based on what they choose.
Having at least 3 different weapons to use	Allows the player to improve their attacks by buying weapons using coins that they earn throughout the levels.
Scoring System	There will be a scoring system that will determine the score of the player's run of the level. This will consider how many coins they picked up, how long they took, and if they used power-ups. There will be a points multiplier that is influenced depending on the difficulty, with harder difficulties having bigger multipliers.
Timer	Ben wanted a timer to be added that should last around 5 minutes, although I may increase the time for harder levels and difficulties.
Controls	To make the user experience as comfortable as possible, I will use A to move left, D to move right, spacebar to jump, the mouse's left click to shoot, G to activate powerups, P to pause the game, I to interact with objects, and R to reload.
Sound effects	Sounds that play as actions such as jumping and shooting. Sound output is dependent on the type of input.
Pause Game	Allows anybody to pause the game should they need to do so. When the game is paused, a menu should show up that gives them the options to return to the game,

	change some settings, and return to the start menu.
Show the score	This would allow the player to see how well they're doing according to the scoring system.
Player starts with 5 lives on each level	Players will start each level with 5 lives, and that will decrease as the player dies.
Each level has around 50-100 coins that can be found	This is a feature that Ben wanted to see in the game, and I believe that 50-100 coins is a good amount to reward during each level.
Reward players with coins and more lives at end of levels	The number of coins and lives will depend on the player scores, with higher scores rewarding more coins and lives, and lower scores rewarding fewer coins and lives.
Game ends when all lives are lost	Helps to increase difficulty for the user in the game
Final Boss Level	On the final level, the user should face a final boss that should also be quite challenging to complete.
Levels 2-5 will be locked from the start	The game is meant to be based on a story, so players should be prevented from accessing the later levels early on, and they should progress through the story as I intend them to.
All levels can be replayed	If a player wants to play through a level they already completed, then they should have the option to do.
All levels are displayed in their own menu	This can display a separate area where the levels can be viewed and interacted with.
Health Bar is shown	Because the player has health, they need to be aware of how much health they have left before they die.
Number of lives also shown	This will help the player become aware of how many lives they have remaining.
Some enemies have ranged attacks whereas others have melee attacks	This helps to diversify the enemies that the player will come up against and will also increase difficulty as the player has to adopt different strategies against different enemies.
Player loses life if they fall off a platform	If a player misses a platform and ends up dying, they will respawn, but they'll also lose a life.

Shop with different weapons and armour	Allows the player to purchase better weapons and armour using the coins they earned to become much more powerful and to aid them in challenging content.
Harder Difficulties should give better rewards	High Risk – High Reward: It's important that players who play on higher difficulties are rewarded more for playing the harder difficulties.

Limitations

There are some features that I would like to add in the game, but ultimately these features won't be realistic due to certain reasons. These features and the reasons for why they're limitations are listed below:

Requirement	Justification	Why is this a limitation?
Spawn Protection	This will allow the user to be protected for a little while, so they can have a period of time where the enemies cannot kill them.	I'm not sure how I could implement this feature in the game, so I cannot add this and it's ultimately a limitation
Two-Player Mode	This can allow 2 friends to take on the game simultaneously so they could complete the game much easier.	I've not got enough time to learn how to implement a Two-Player Mode and I'm also not familiar with how to go about this.
Scrolling Background	This can allow levels to be much bigger, so the user can take more time on a level whilst being careful as to what is outside of the screen	I don't know much about how a scrolling background could work in Pygame and I don't have much time as earlier mentioned, so it's difficult to add this.
Option to choose sides in Two-Player Mode	This can allow 2 friends to play different roles within the game by controlling certain characters within the game, one could control the main character whilst another could control an enemy to try to disrupt their friend's progress.	Like mentioned earlier, I don't know how to add a Two-Player mode, so I definitely don't know how to do this nor do I have the time to find out about this.
Large-scale levels	This will allow me to add more enemies and coins into certain levels and could potentially add extra difficulty to the game depending on how big it gets.	Time limitations mean that I could potentially do it, but it would take too long to complete this requirement.
More than 5 levels	This can allow the story to go on further – I can experiment with it and add more storyline to the game, and the user could get immersed into a bigger story with more lore.	Like mentioned earlier, I am time-bound so it won't be possible to complete this requirement.

Hardware and Software Requirements

Python and Pygame has a set of requirements that must be followed to run it. I have adjusted this to suit the program that I'm making and the requirements that may be required for my game specifically.

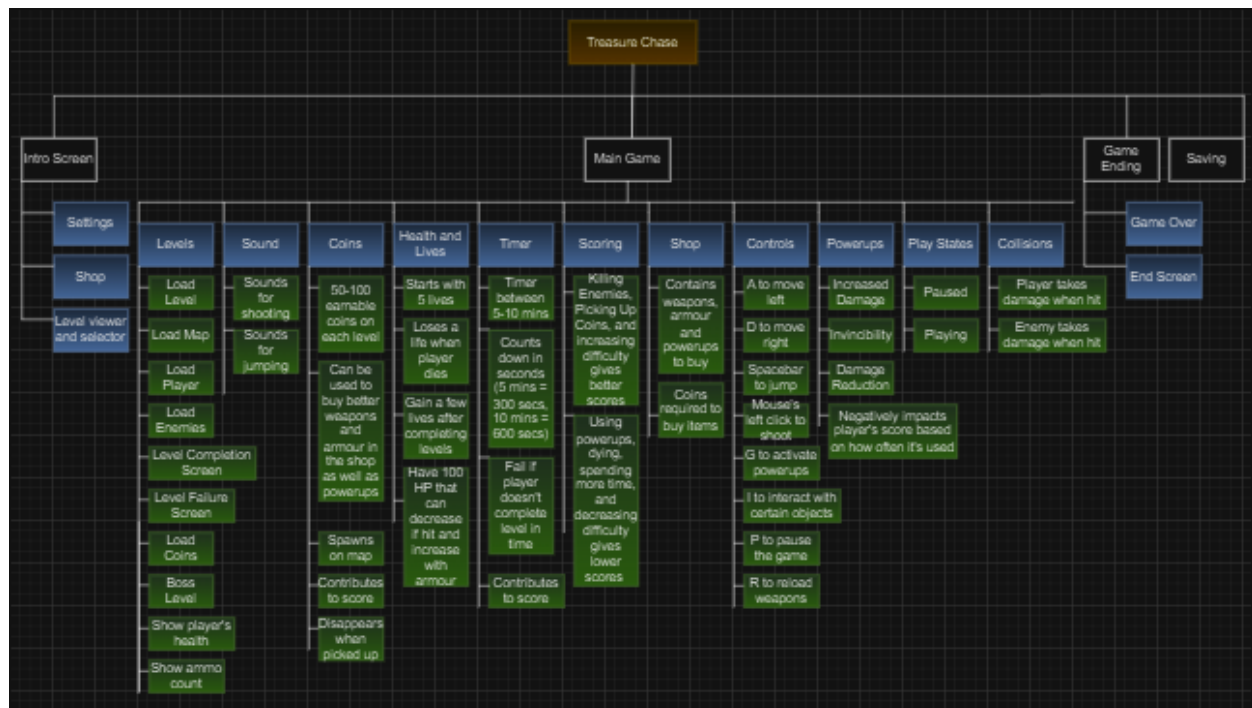
Software Requirements:	Justification:
Operating System: Windows 7/8/10/11, MacOS 10.6 or later, Linux	These are the operating systems that Python can run on. Python may not run on other operating systems.
Python Application	Pygame uses Python, so Python is needed otherwise there is no way to open the game.
Pygame Module	The game will be made using the Pygame module, so the version of Python that the player uses needs to have this installed, otherwise the game cannot run.

Hardware Requirements:	Justification:
CPU: Dual Core, 1GHz clock speed	Will ideally need 2 cores and at least 1GHz clock speed to run the game
RAM: 2GB minimum	Pygame doesn't take up much memory, so 2GB should be fine.
Storage: Around 1GB	Need to store python, Pygame, and the additional libraries needed for Pygame to run
Graphics: Minimum Graphics	Not much graphical power is needed, so basic integrated graphics should be perfectly fine.
Keyboard	Allows user to control their movement
Monitor/Laptop Screen	The user needs to see what is happening
Mouse/Laptop Trackpad	Allows the user to fire their weapon
Speakers	There will be sound effects within the game.

Design

After conducting my interview with Ben as well as doing research in the form of looking at Super Mario Bros 3, I am aware of the requirements for Treasure Chase, and I now know what I need to implement into this game to meet as many requirements as possible. These requirements have adjusted the way I have decided to design my game. As a result, I need to consider absolutely everything before starting the development of the game. As a result, I have used a systems diagram to break down my project as much as possible to have a clear idea of all the smaller problems that I need to solve before development.

Systems Diagram



I have chosen to do a top-down module design as it can help me clearly identify all of the sections of my project that I must consider and plan for before coding. This allows me to focus on all the smaller parts of the project more specifically and it ensures that I do not miss any parts of the project. This also allows me to utilise the computational method “Thinking Ahead” to determine the inputs and outputs needed; this will also allow me to use the computational method “Thinking Logically” to see how one part of the project can impact another. I have also used decomposition to break down the entire project into smaller, more manageable tasks.

Explaining Each Module

Intro Screen-

- Settings – Allows the user to adjust certain settings such as sound and difficulty.
- Shop – Allows the user to purchase items such as weapons, armour, and powerups.
- Level Viewer and Selector – Allows the user to view and select the levels that they want to play.

Main Game-

- Levels –
 - Load Level – Loads the level that the user has selected.
 - Load Map – Loads the map of the level
 - Load Enemies – Loads the enemies within the level
 - Level Completion Screen – Loads a screen that will show if the user successfully completes a level.
 - Level Failure Screen – Loads a screen that will show if the user fails a level
 - Load Coins – Loads the collectable coins within the level
 - Boss Level – A final level that will consist of a boss that the user has to defeat.
 - Show Player's Health – Show how much health the user has remaining.
 - Show Ammo count – Shows how much ammo the user has remaining before they have to reload.
- Sound –
 - Sounds for Shooting – Takes place when the user shoots
 - Sounds for jumping – Takes place when a user jumps
- Coins –
 - 50-100 on each level – Allows the user to earn a decent number of coins every level.
 - Used to buy weapons, armour and powerups – Creates a use for the coins as well as a straightforward way to improve the user's power.
 - Spawns on map – So user can earn them
 - Positively contributes to score – Increases the user's total score.
 - Disappears when picked up – When the user picks up the coin, it will not show on the map anymore.
- Health and Lives –
 - Starts with 5 lives – Creates a baseline for the user to work with.
 - Loses a live upon death – Lives give the user more chances to finish levels, so death will consume a life.

- Earns lives after completing levels – Allows the user to earn back lives, or potentially gain even more lives.
- Base of 100 HP – Gives the user a base HP stat that they can work with that can also be increased with armour from the shop.
- **Timer –**
 - Timer lasts from 5-10 mins – Allows the user to have a good amount of time to complete the level
 - Counts down in seconds – I feel that it would be easier if I counted it in seconds instead of in minutes
 - Fail level if timer runs out – If the user doesn't complete the level in time, they will fail the level.
 - Contributes to score – The longer the user takes, the lower their score will be
- **Scoring –**
 - Good performance and increasing difficulty = Better score
 - Worse Performance, using powerups and decreasing difficulty = Worse Score
 - These ensure that better performances are better rewarded.
- **Shop –**
 - Weapons, Armour, and Powerups for purchase – Allows the user to use the coins they earn to improve their gear.
 - Requires coins earned from levels.
- **Controls –**
 - A to move left
 - D to move right
 - Spacebar to jump
 - Mouse left-click to shoot
 - G to activate powerups
 - I to interact with certain objects
 - P to pause/resume the game
- **Powerups –**
 - Increased Damage – Increases the player's damage
 - Invincibility – Makes the player immune from any damage
 - Damage Reduction – Reduces how much damage enemy attacks deal to the player
 - Negativity impacts score based on how many have been used
- **Play States –**
 - Paused – The game is paused
 - Playing – The game is playing

- Collisions –
 - Player takes damage when hit – Gives the player a hitbox, which makes it realistic and fair.
 - Enemy takes damage when hit – Makes the game fair by giving the enemies a hitbox.

Game Ending-

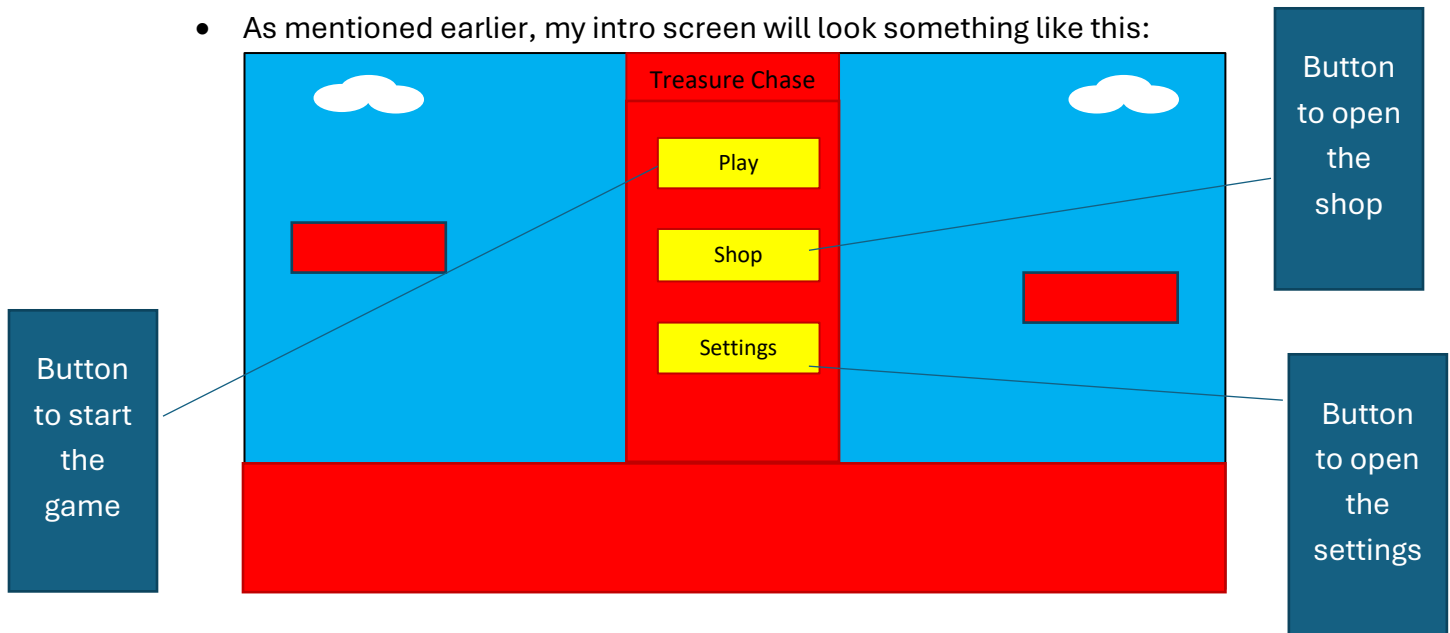
- Game Over – The game ends
- End Screen – Shows when the game ends

Saving – Saves the game

Features

Game Start

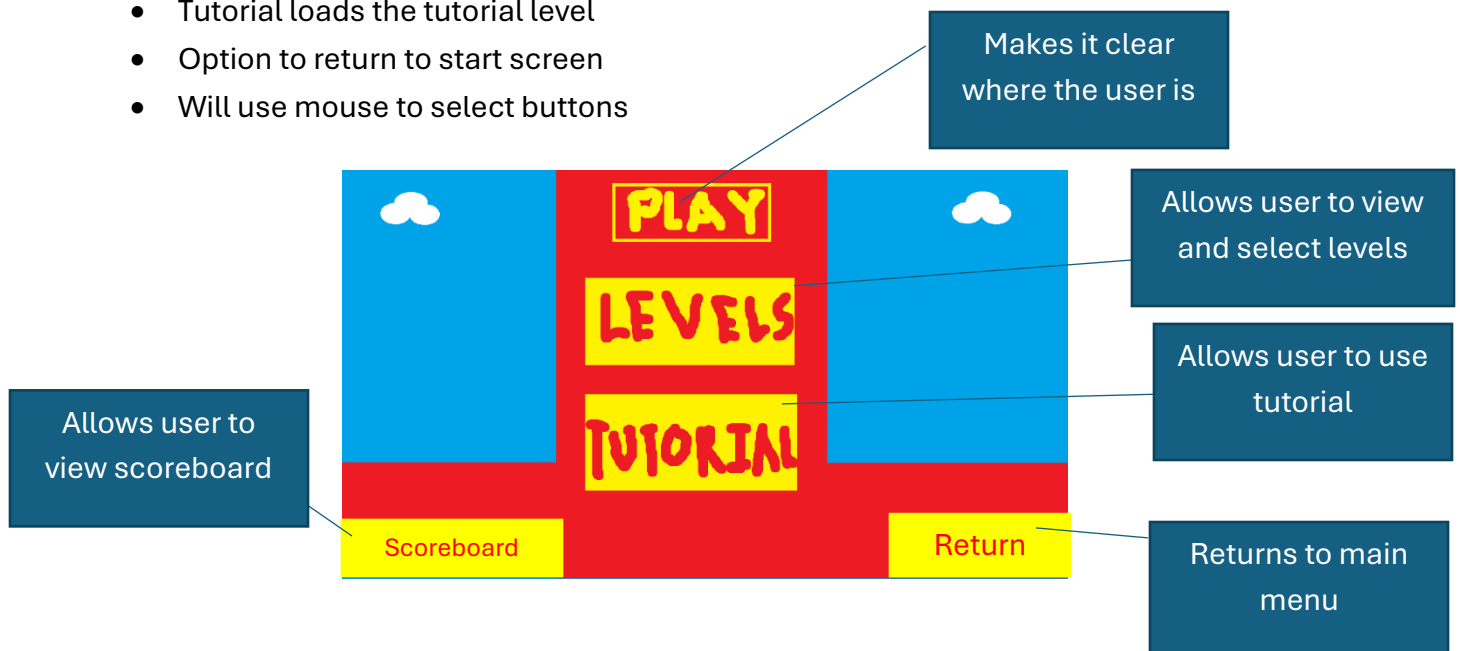
- As mentioned earlier, my intro screen will look something like this:



- This is my first draft of what the start screen will look like, so it's likely that it will look different by the time the project has finished, although the buttons shown here will remain by that time.
- It will use the mouse to select the different options

Play Menu

- Unloads start screen
- Has 2 Options – Tutorial and Levels
- Levels will show all level options that can be played
- Tutorial loads the tutorial level
- Option to return to start screen
- Will use mouse to select buttons



- There will be an option to return to the start screen
- There will also be an option for the player to view their best scores

Levels

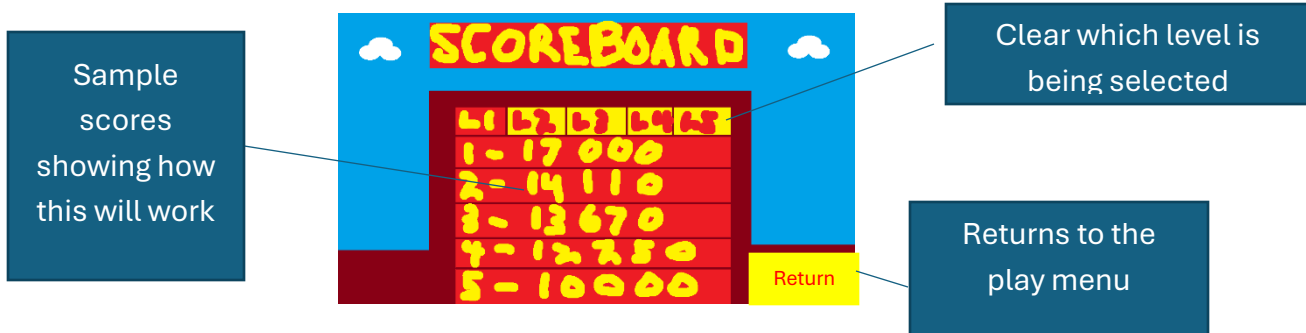
- Can select any level to play
- Final level looks different so it's clear that it's the final level.
- Start on level 1
- Later levels can only be unlocked if previous level is completed.
- Each button loads its associated level



- I think that I may not have enough time to do 10 levels, so I will only do 5 levels instead, which I believe is more realistic with the remaining time that I have.

Scoreboard

- This will show the scores of the player's 5 highest scoring attempts at each level. It will be blank if there are no scores



- I've also switched up how I'm making the scoreboard – Instead of having to add up the scores of one run of the 5 levels and generate a score from that, the scoreboard will now instead show the 5 best runs of each level, which helps to encourage the user to replay the levels to earn better scores.

Shop

- The player can use the coins that they earn to buy items from the shop, such as weapons, armour and powerups.
- The player can also see how many coins they have when they enter the shop.



- The weapons here are not the final weapons and may change in the end, same with the prices of the weapons.

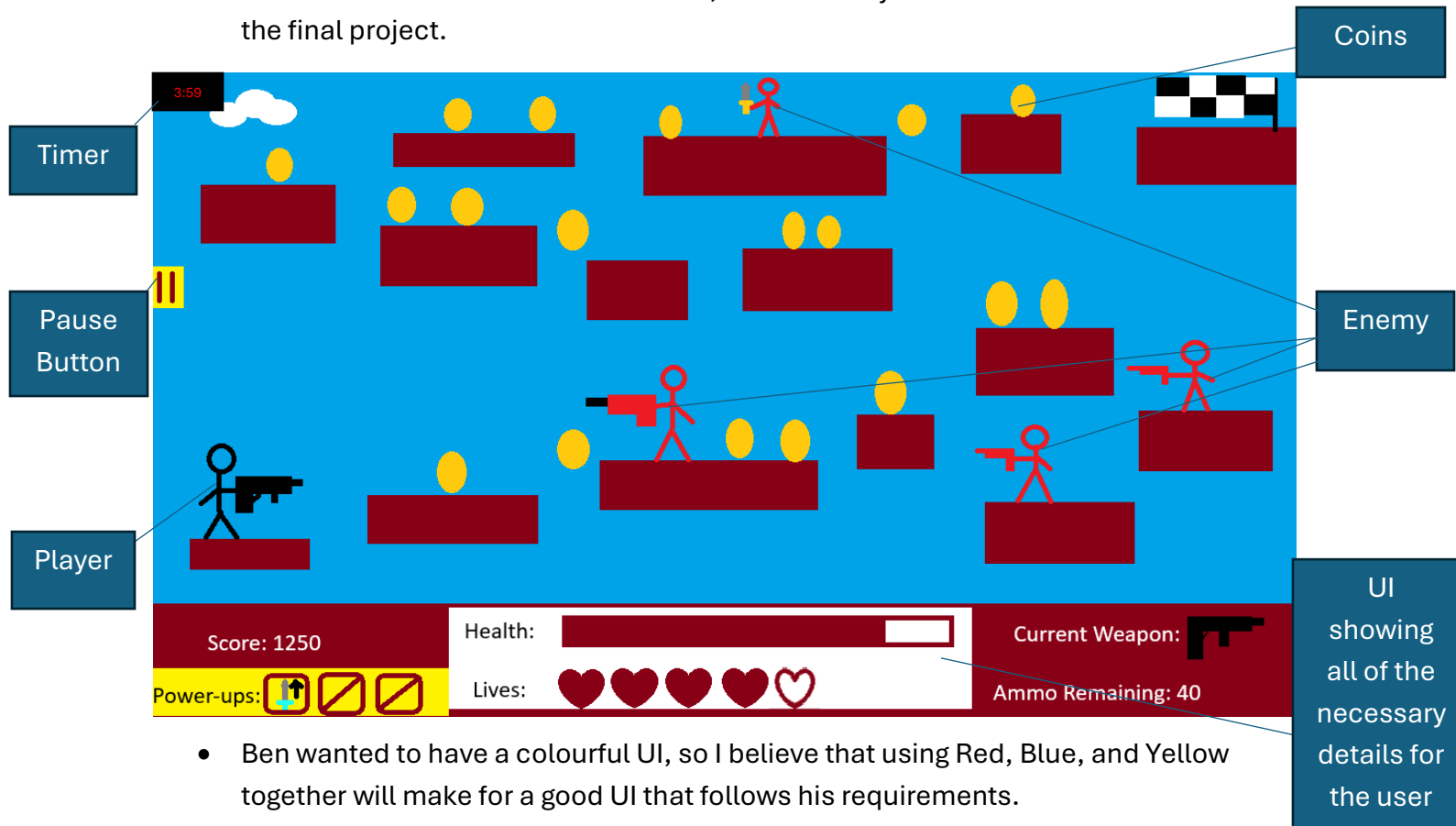
Settings Menu

- The player can change the settings shown in the settings menu
- I may choose to adjust what options can be adjusted in the setting. The options shown are what I think should be in these main settings.



Gameplay

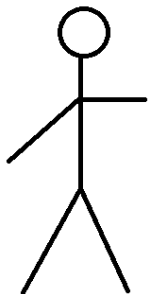
- Like with all the other features shown, this will likely be different when it comes to the final project.



- Ben wanted to have a colourful UI, so I believe that using Red, Blue, and Yellow together will make for a good UI that follows his requirements.
- I feel that the UI must be clear and usable, so there is a good user experience.

Sprites

Player-



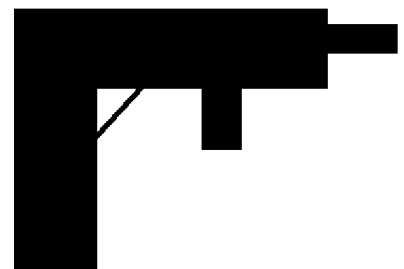
Enemy-



Coin-



Base Weapon-



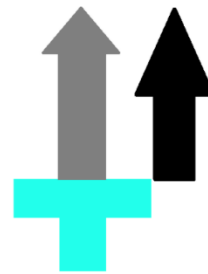
Heart lost-



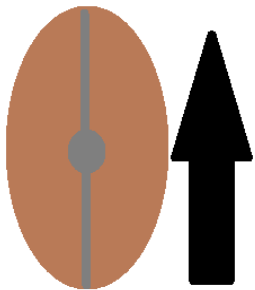
Heart remaining-



Increased Damage Powerup-



Increased Health Powerup-



Ultimate Powerup-

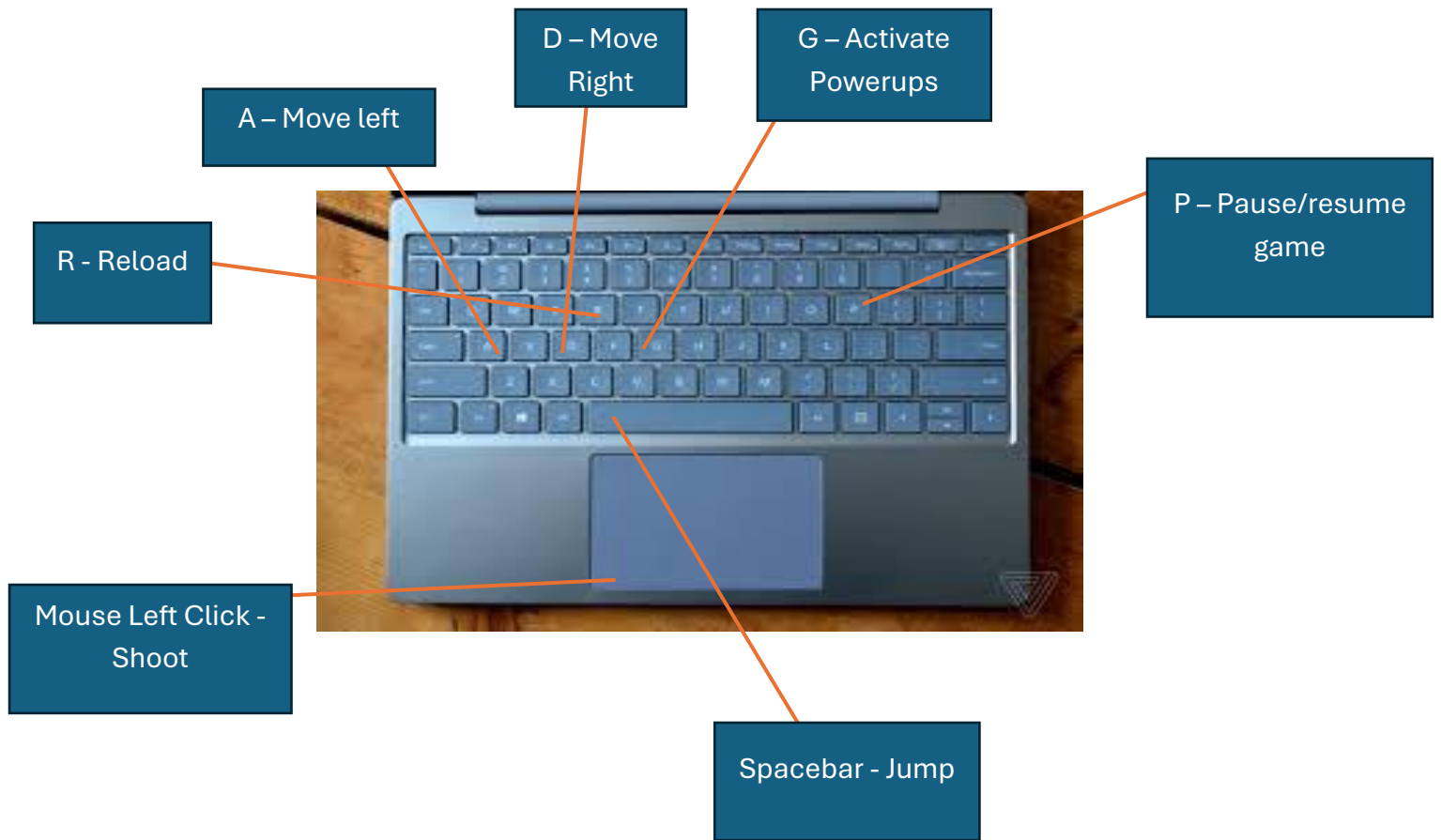


Enemy Weapons-



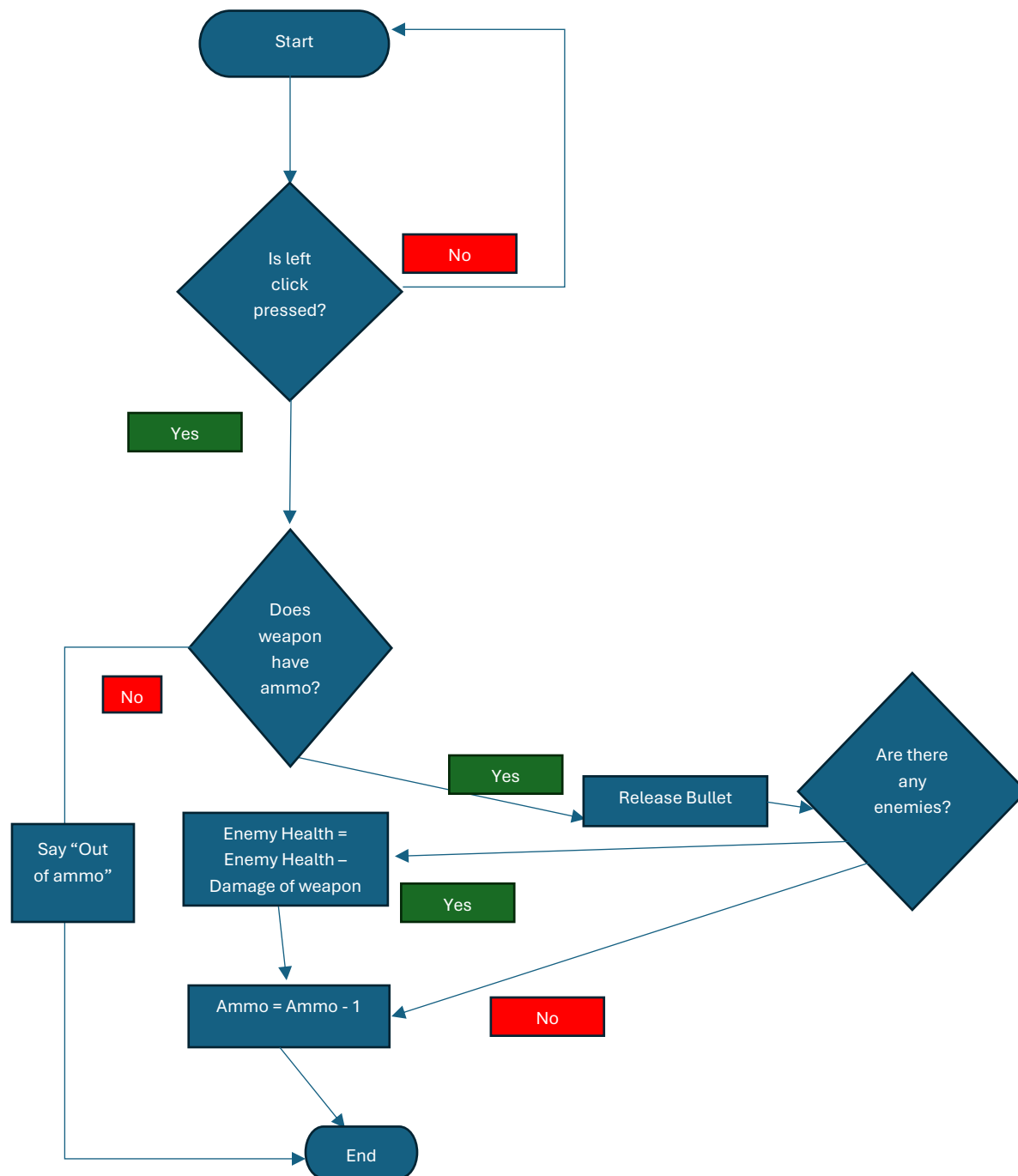
- Because of the nature of the game, I believe that I can make abstract sprites that would allow me to save time whilst still keeping the theme consistent in the sprites.

Controls

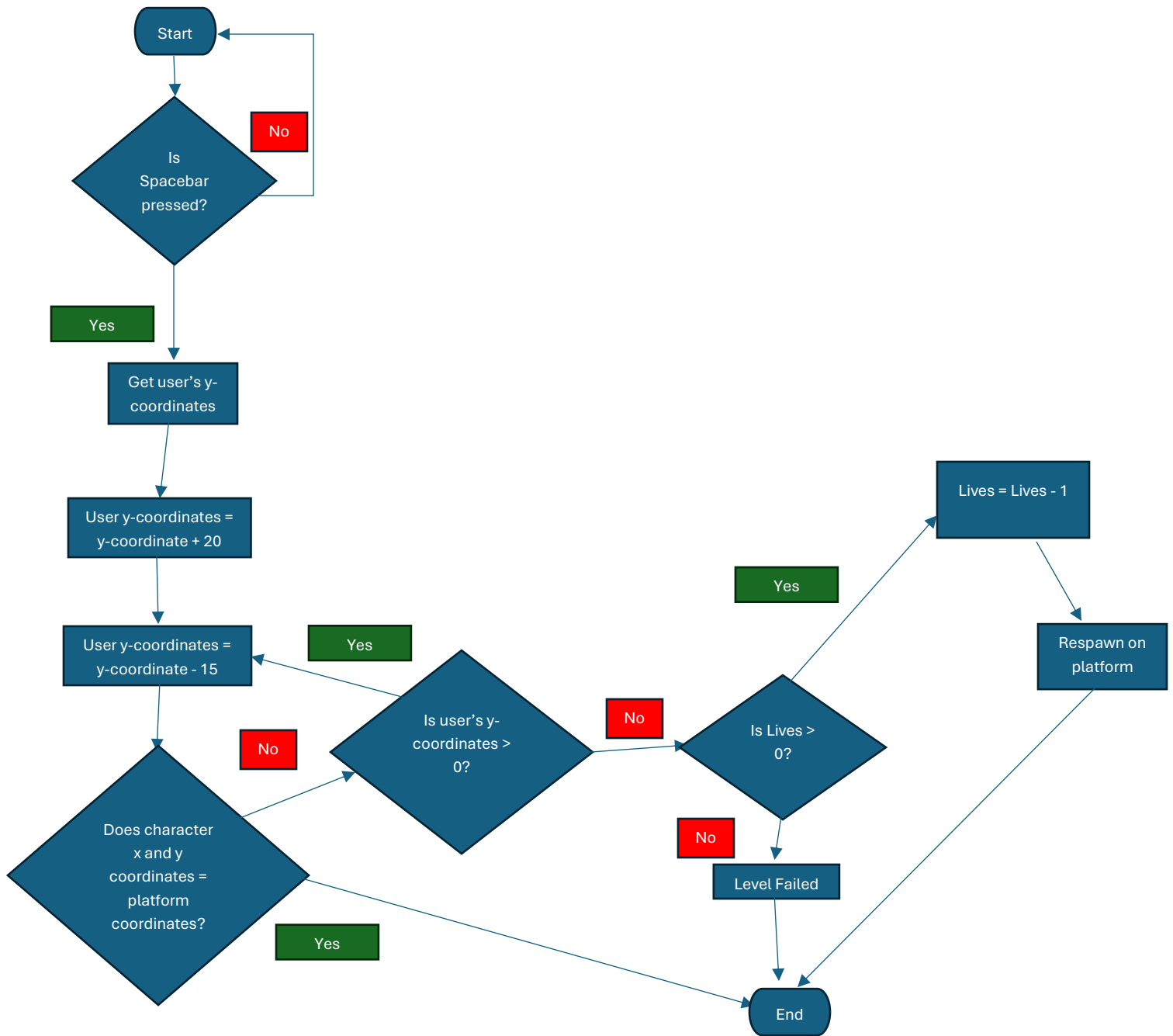


Algorithms

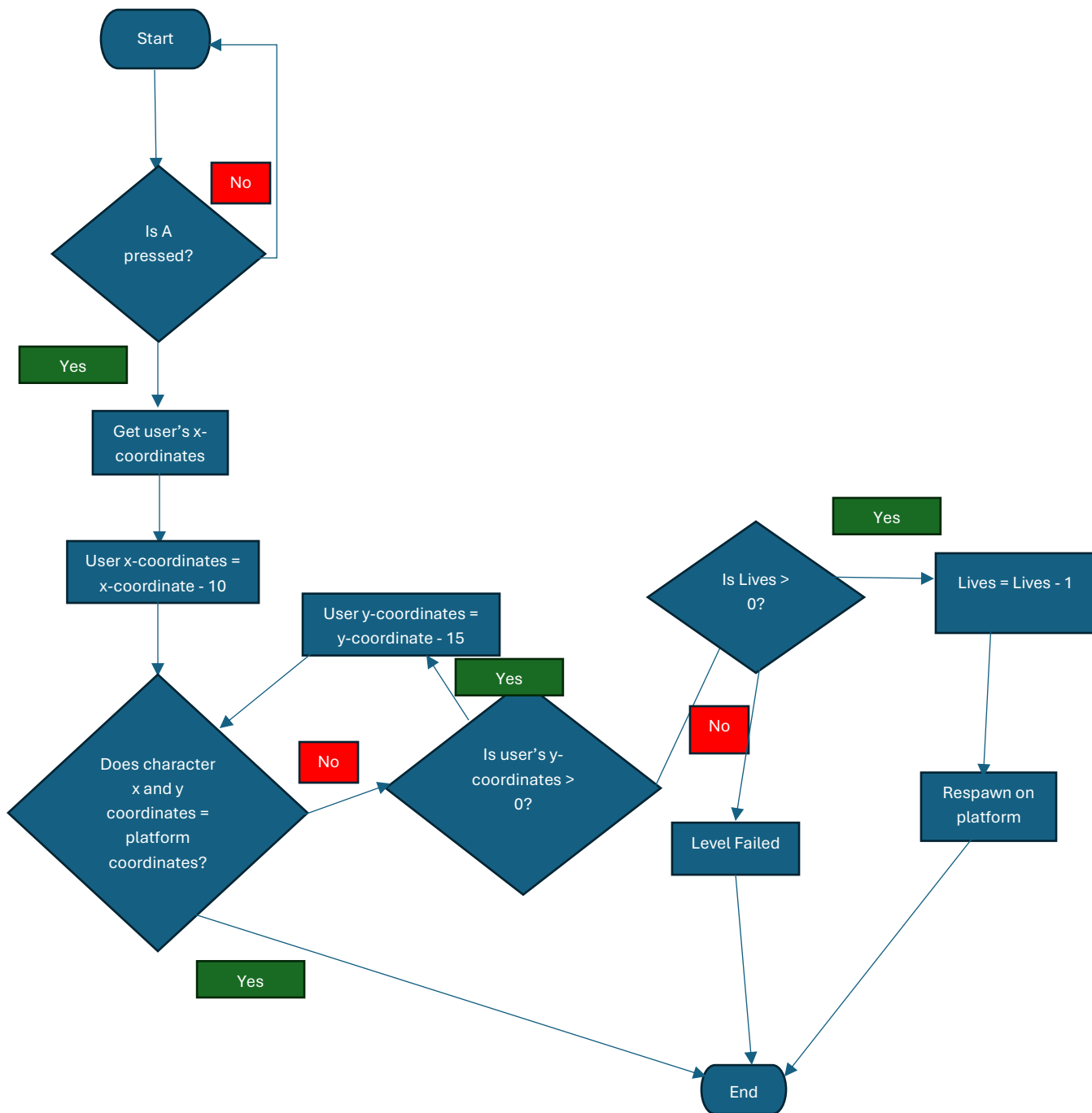
Shooting



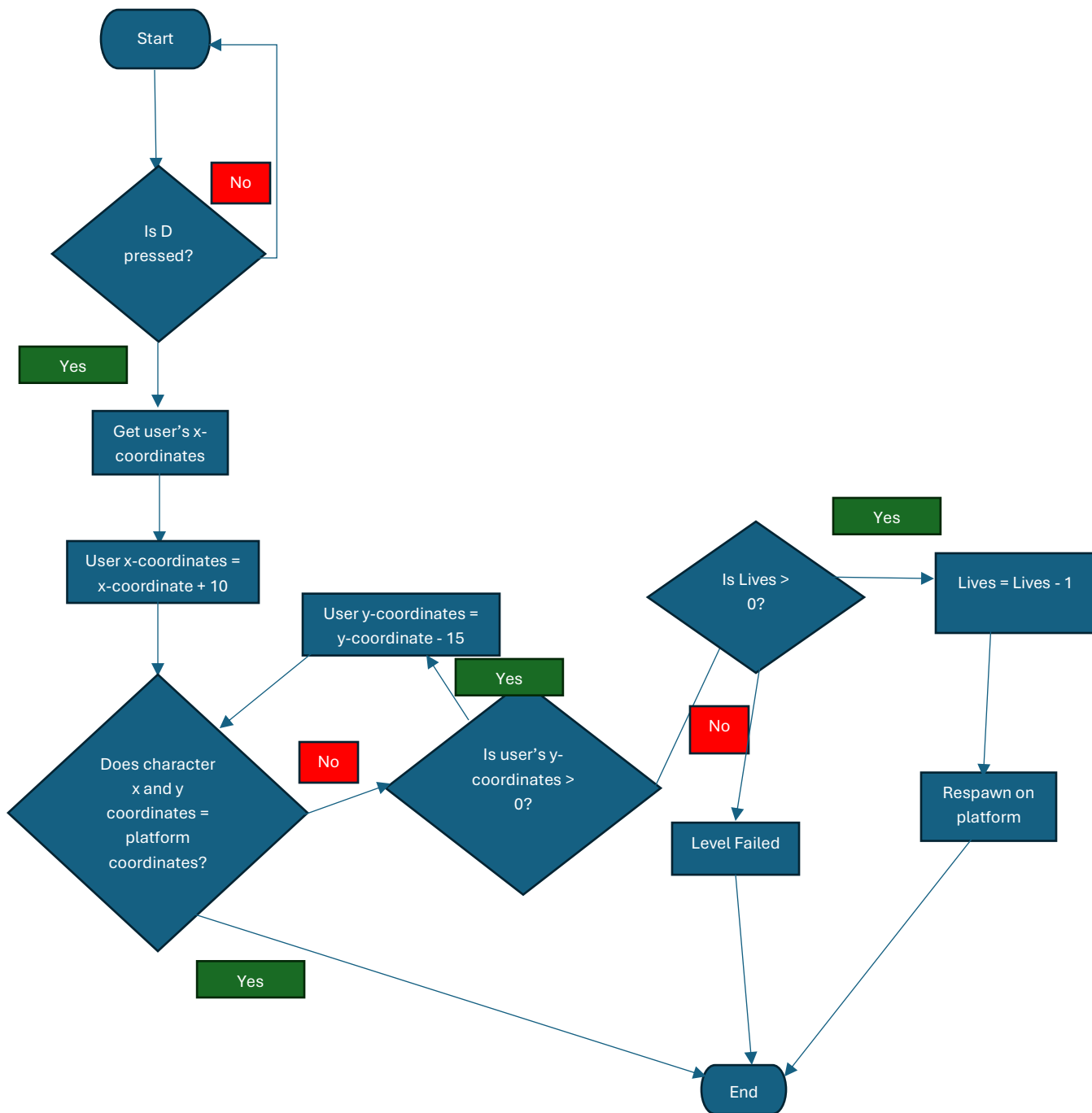
Jump



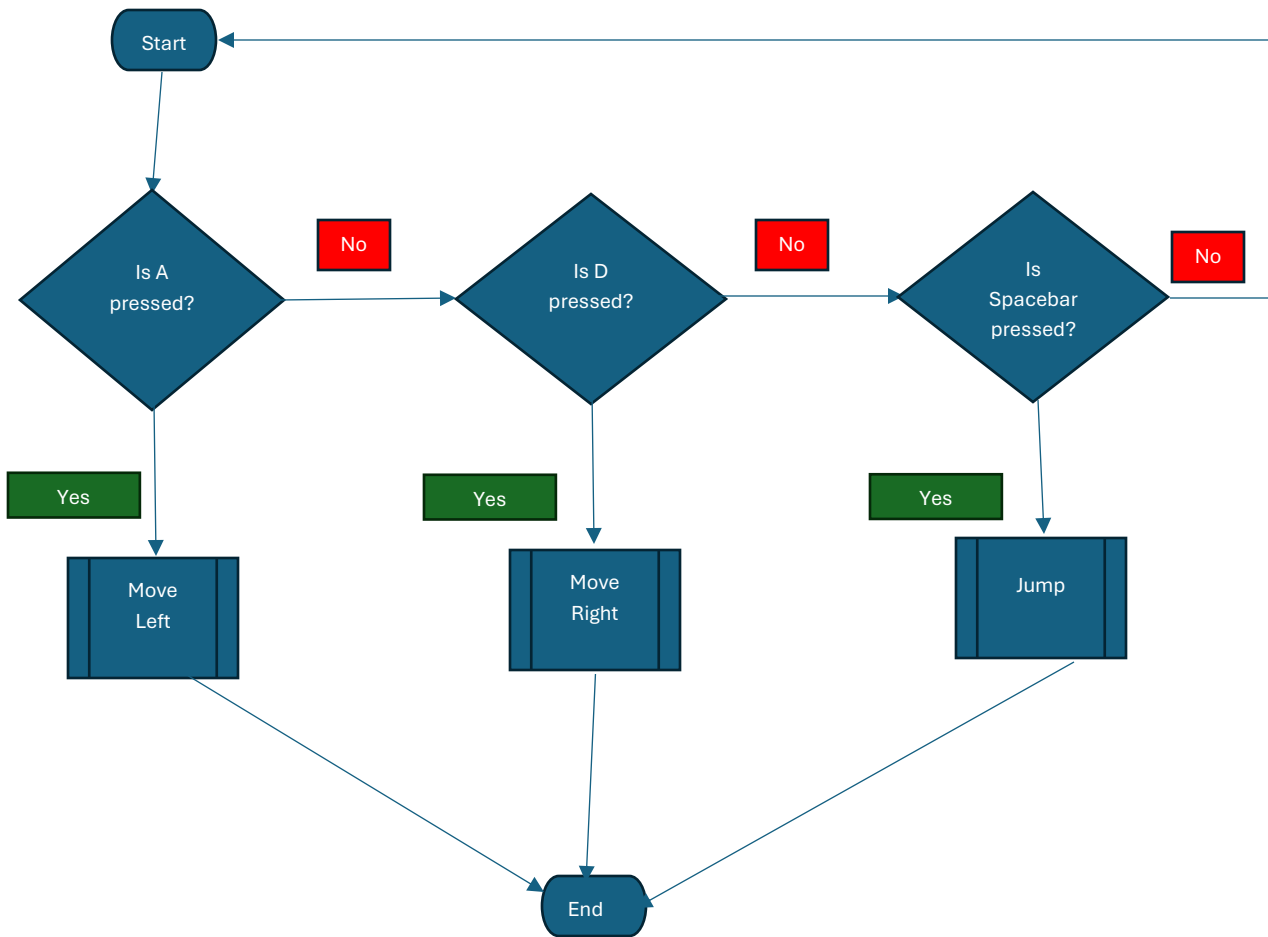
Move Left



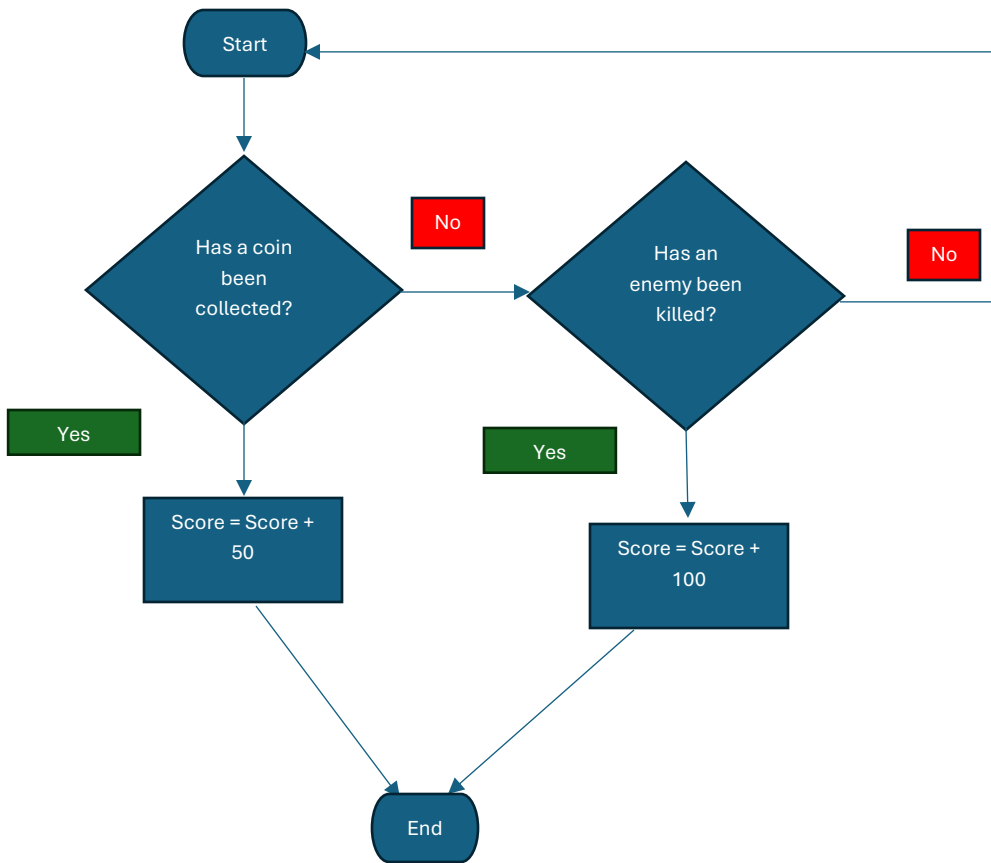
Move Right



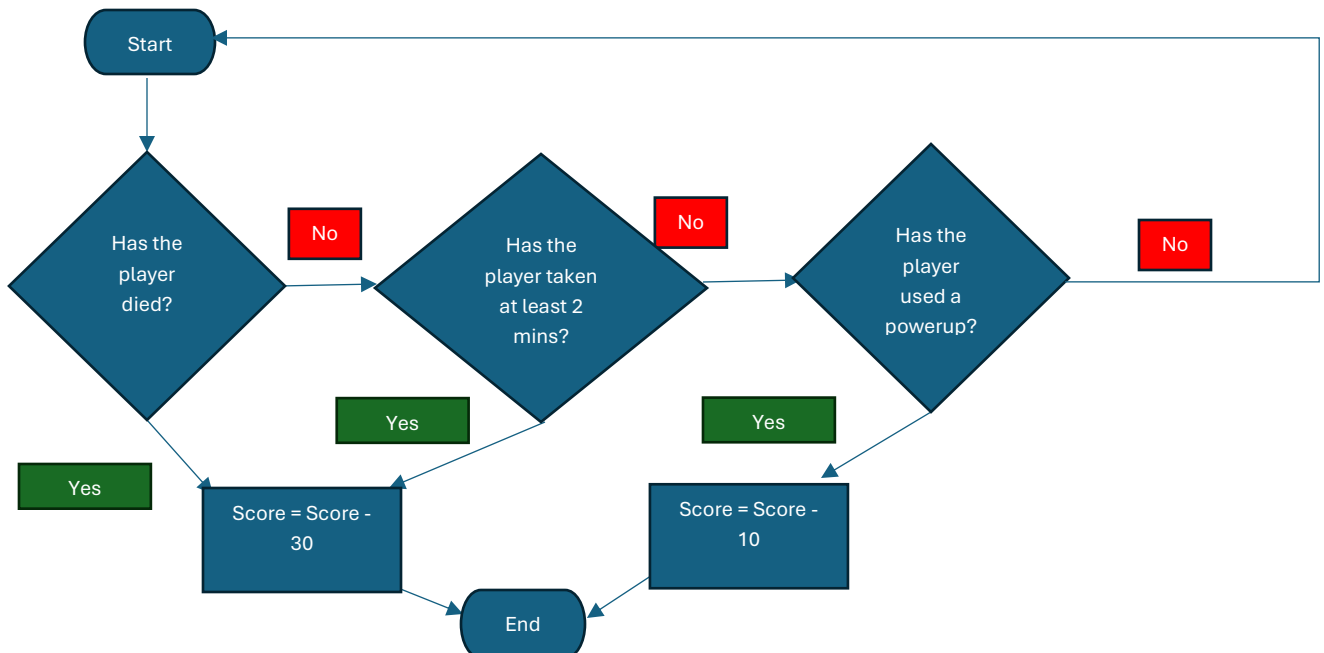
General Movement



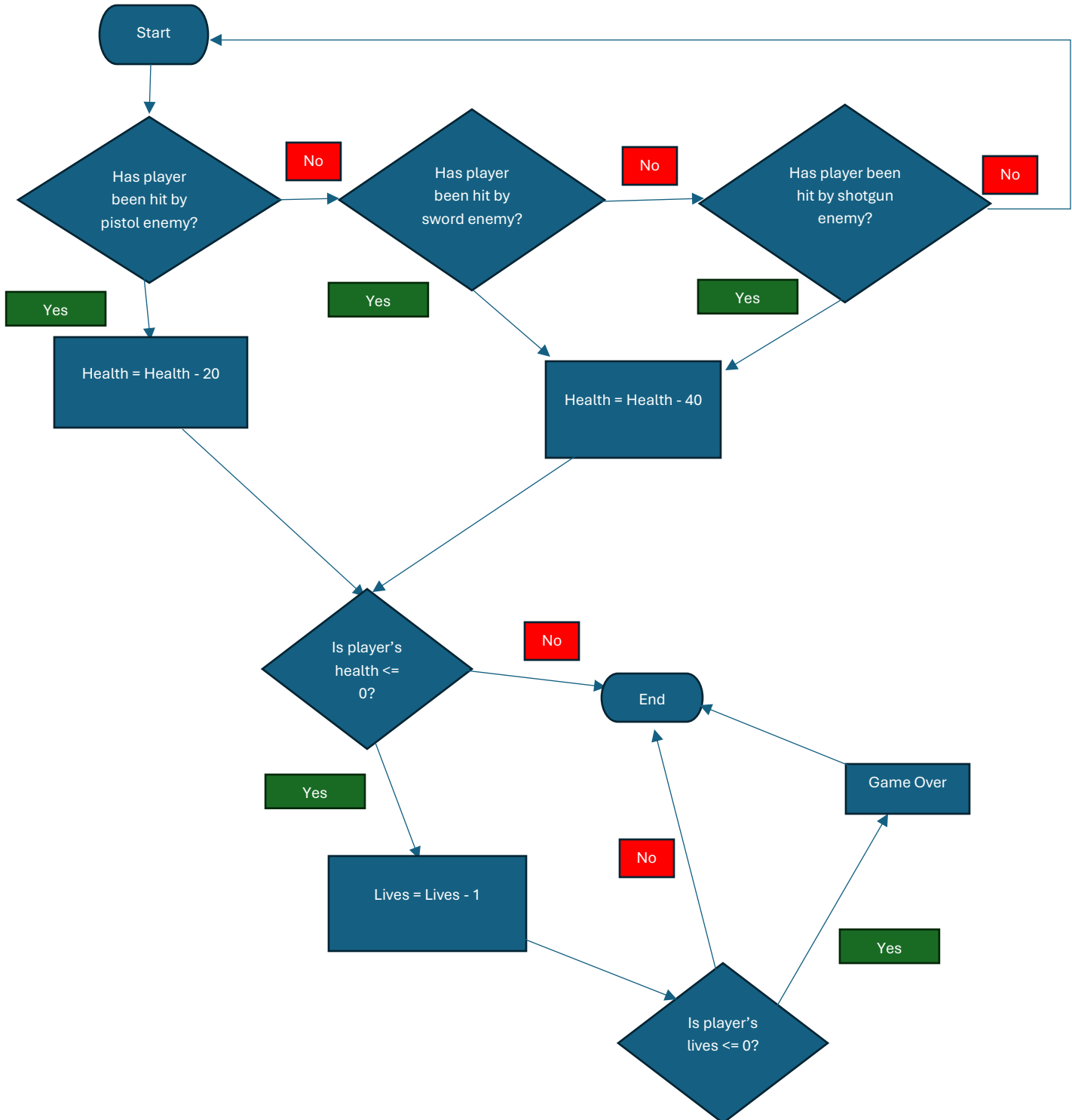
Scoring (Positive)



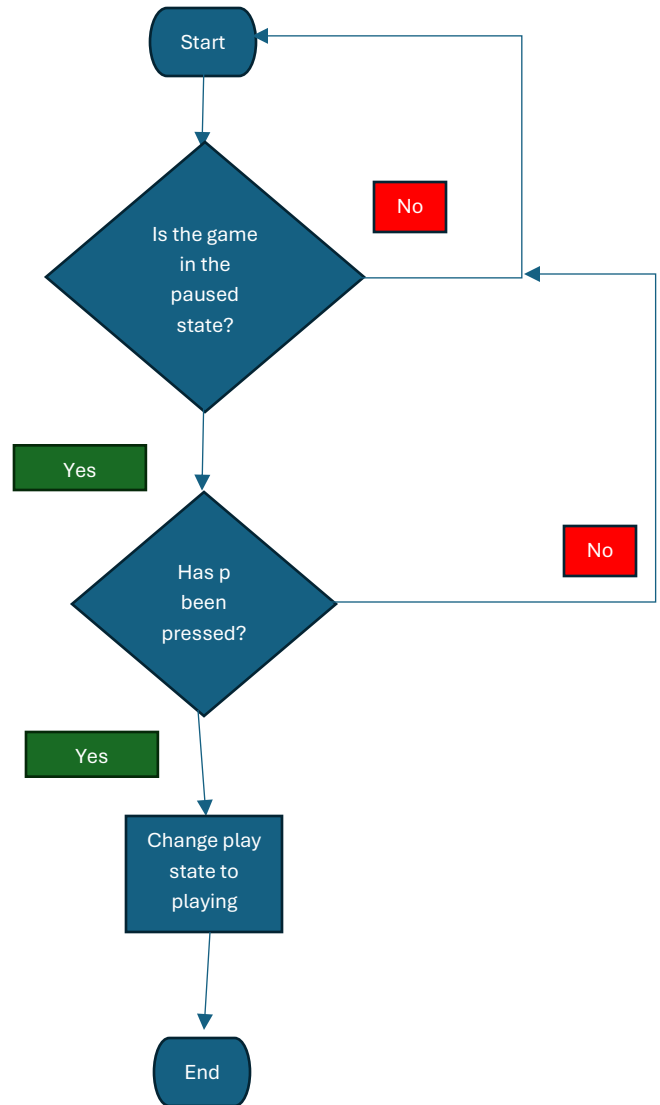
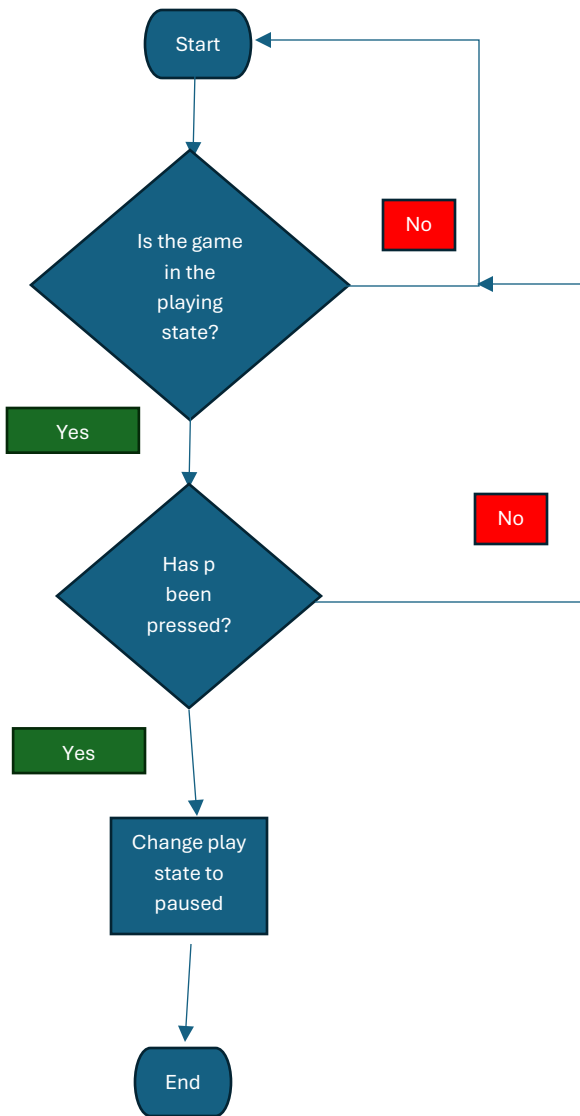
Scoring (Negative)



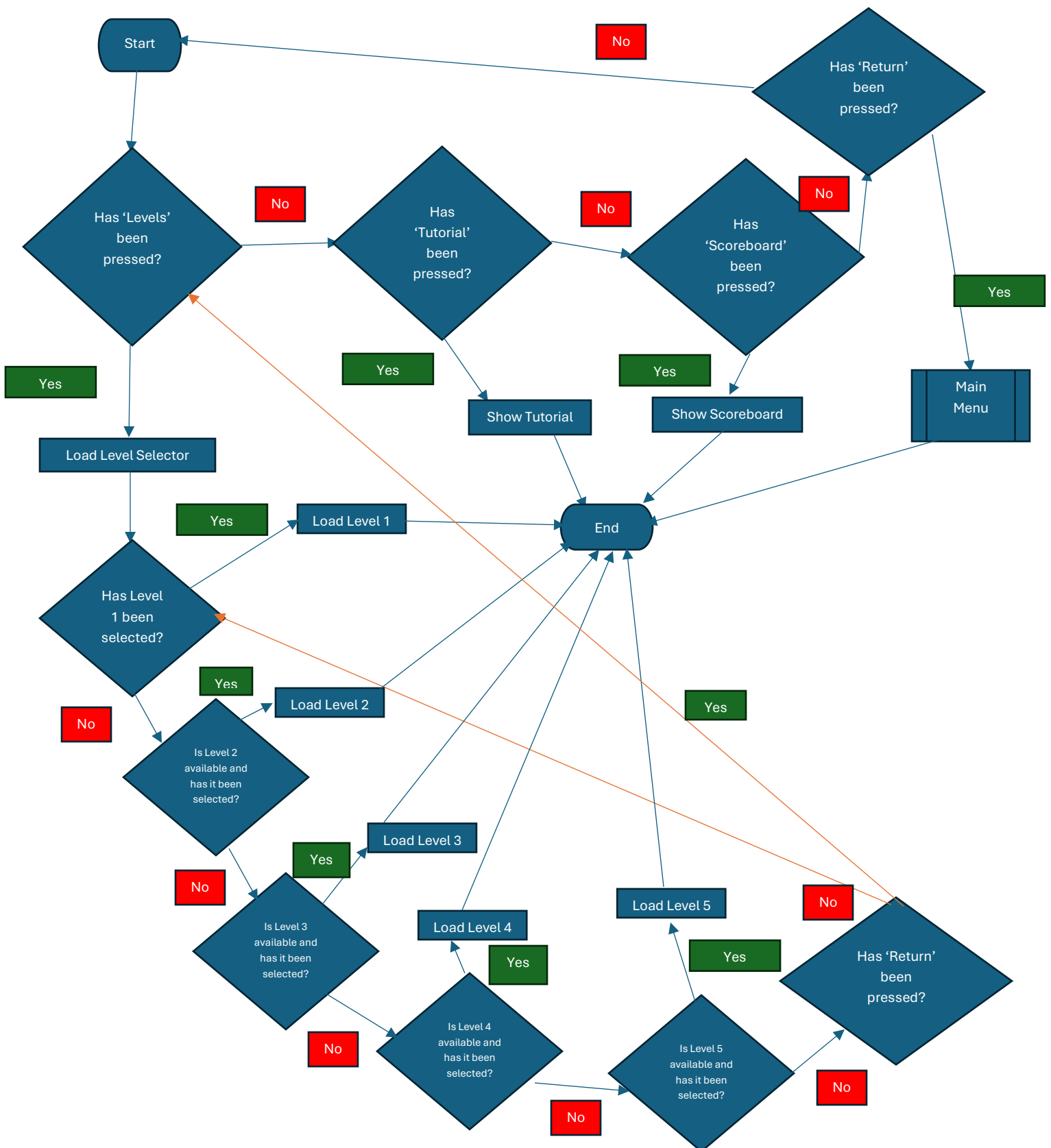
Health and Lives



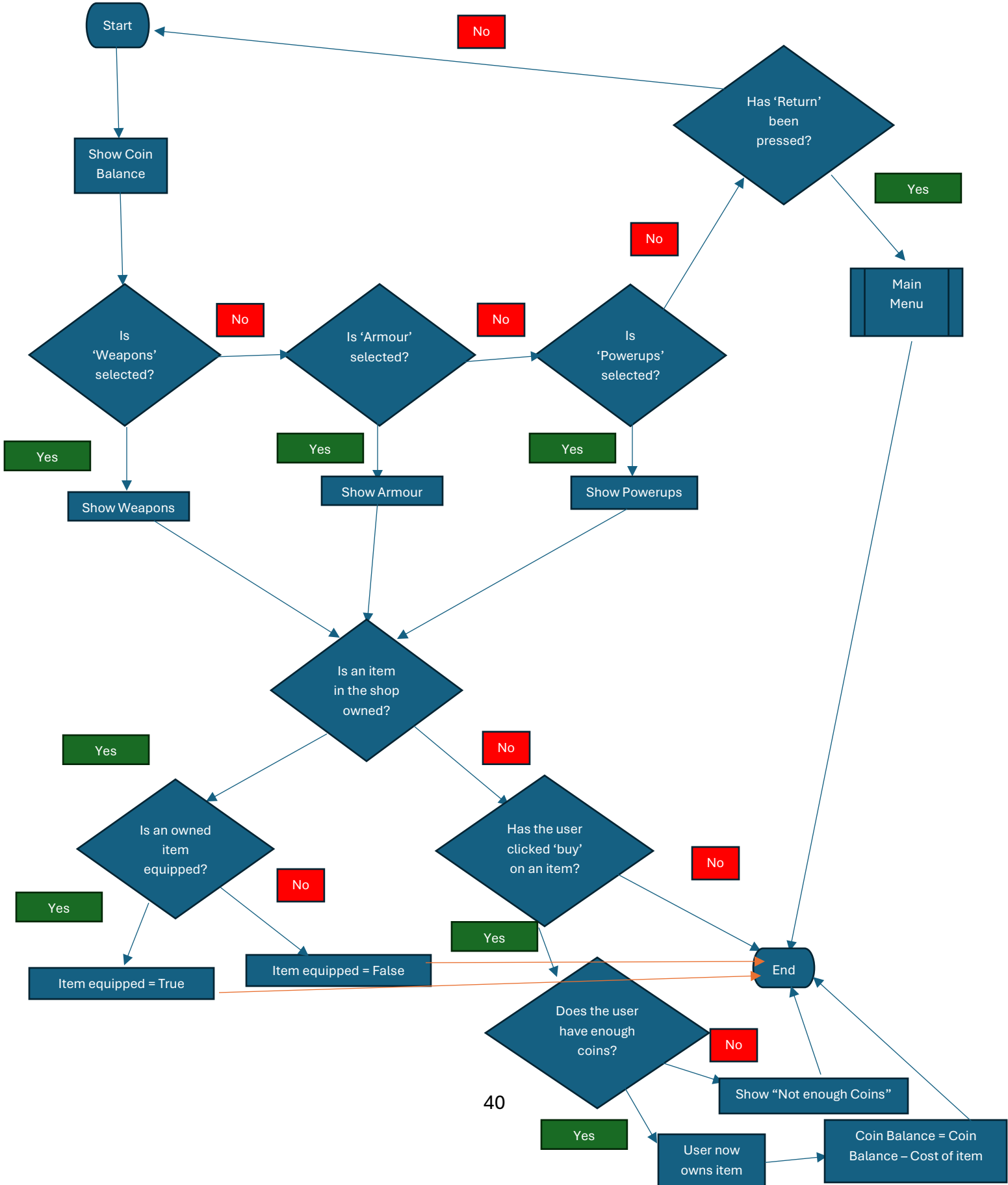
Paused/Playing-



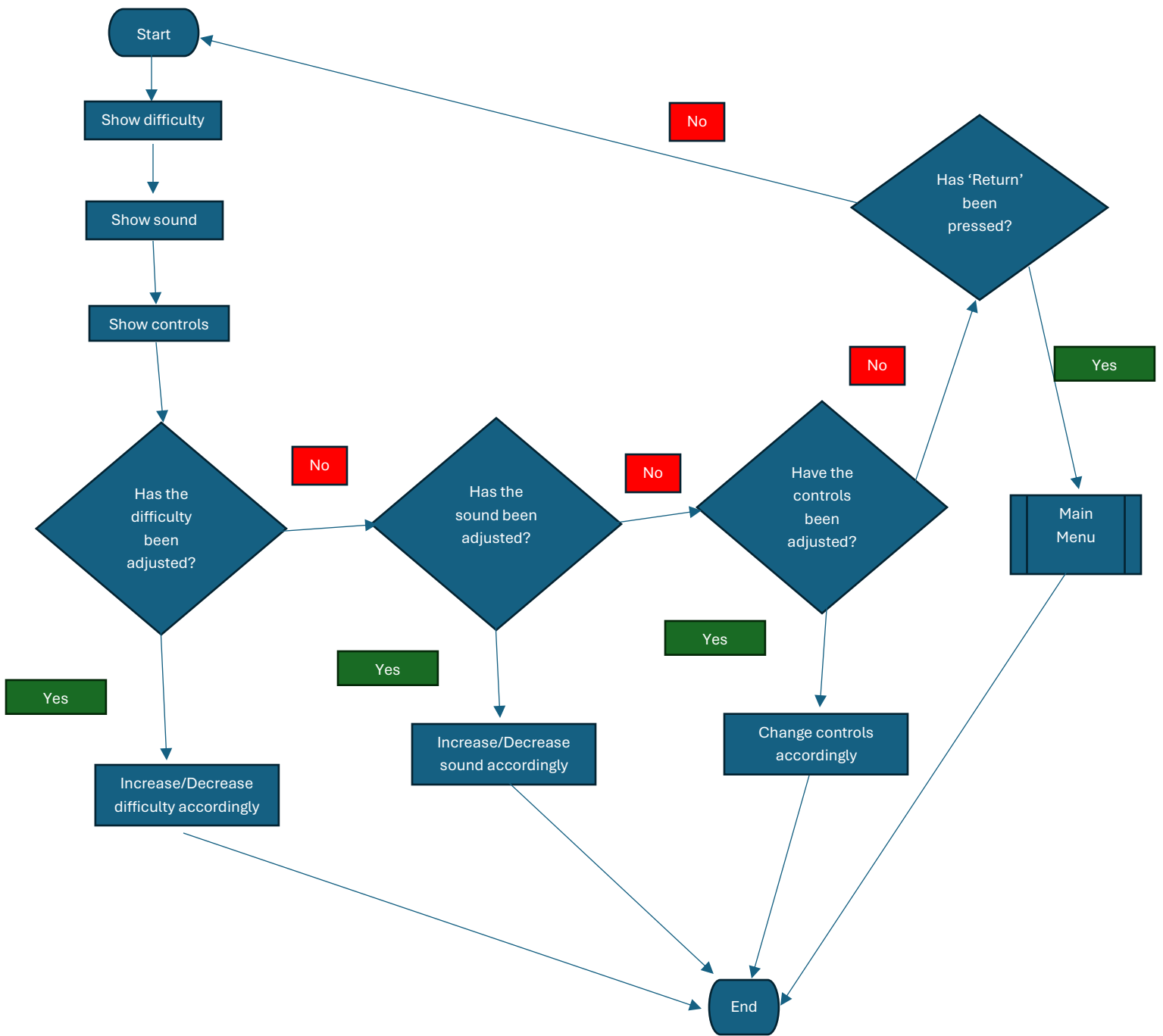
Play Menu-



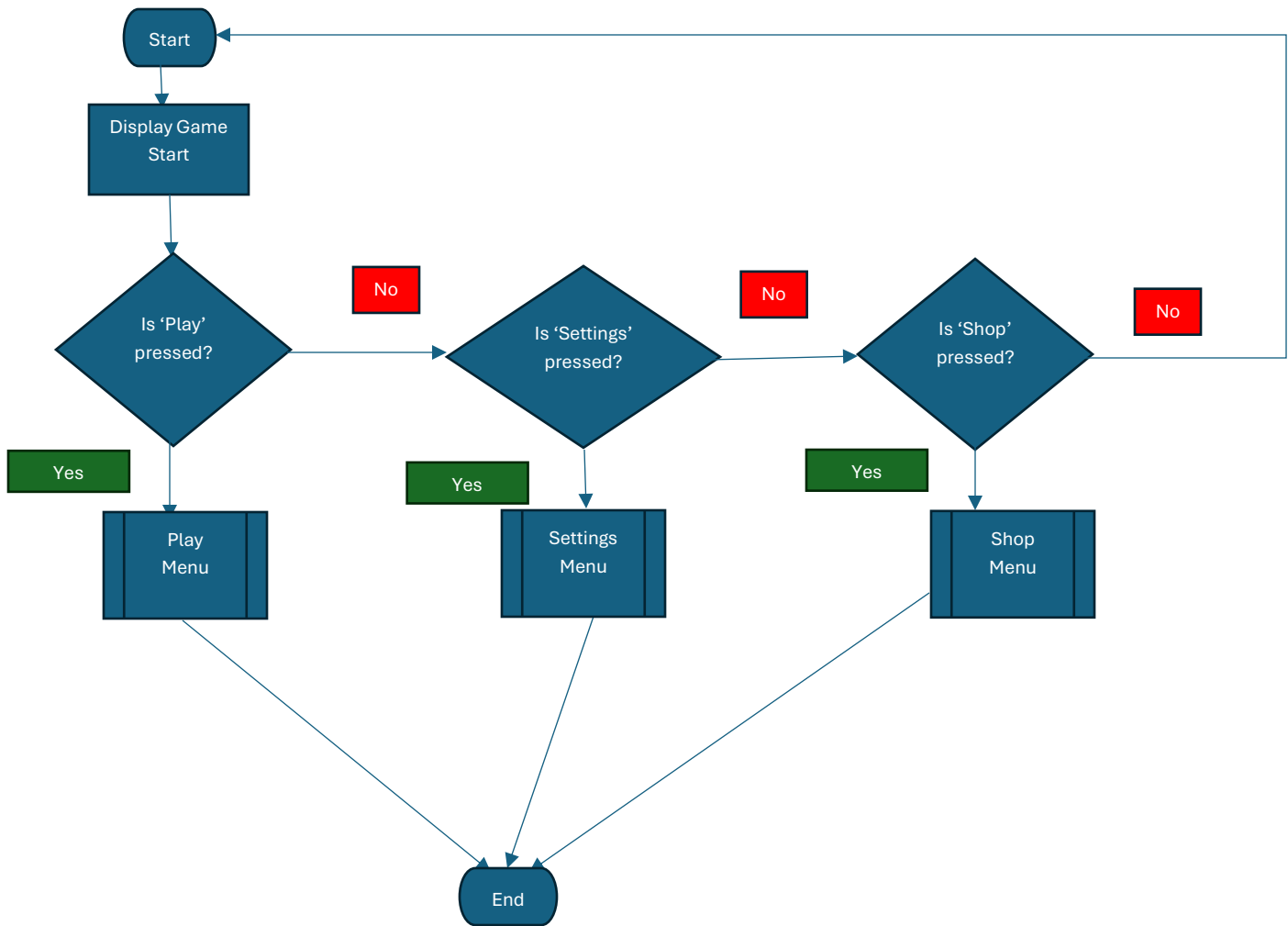
Shop Menu



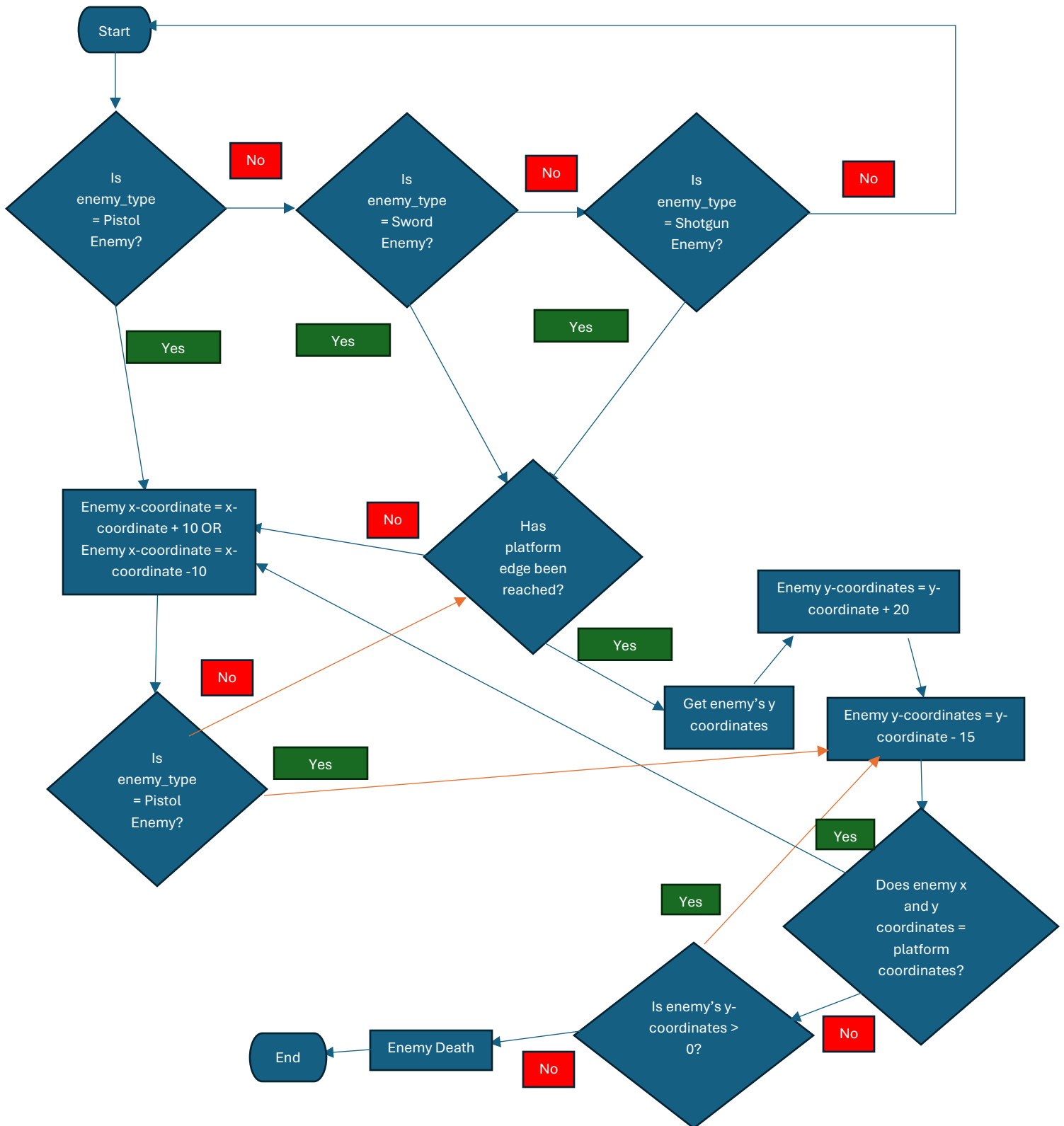
Settings Menu



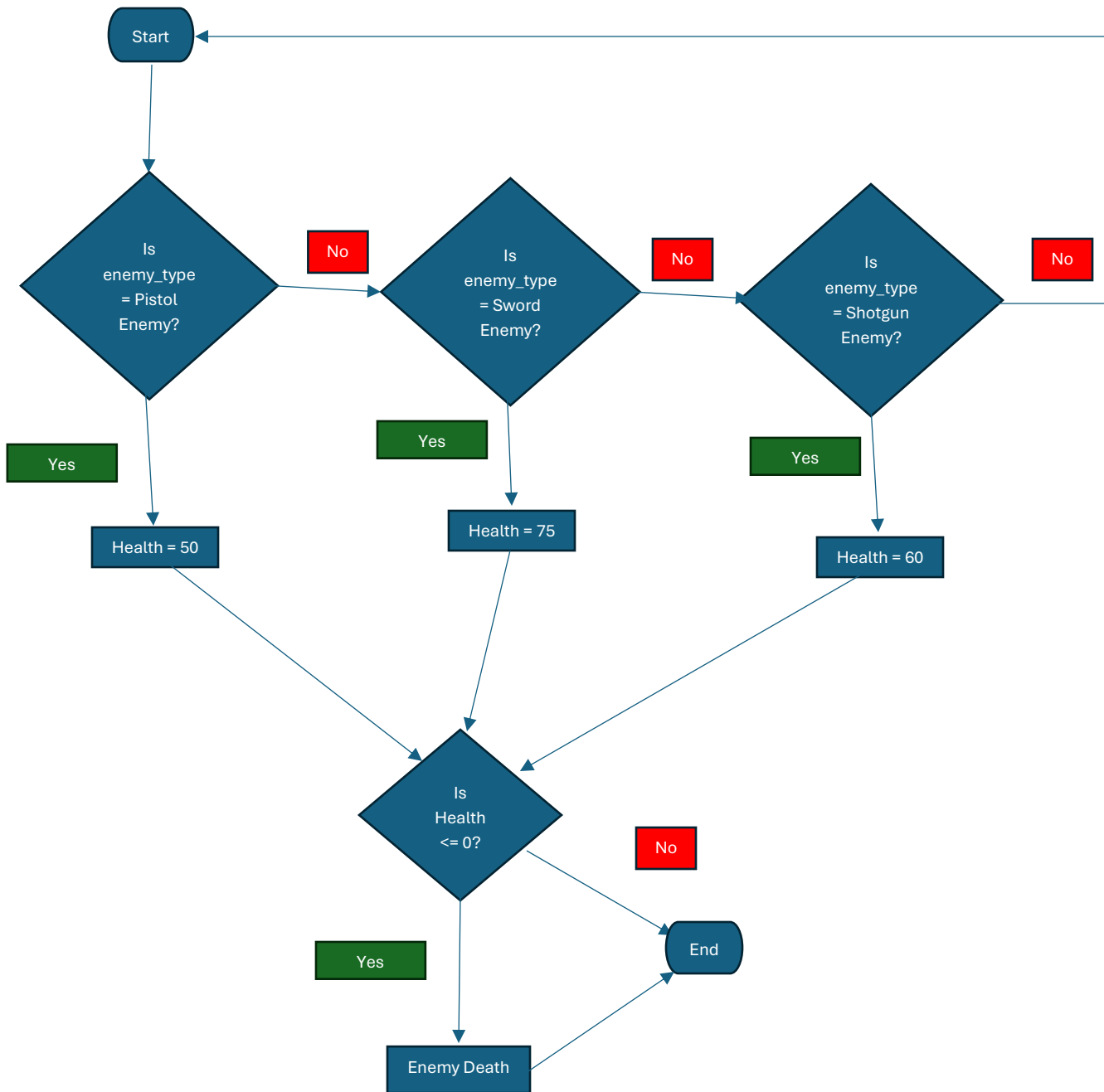
Main Menu



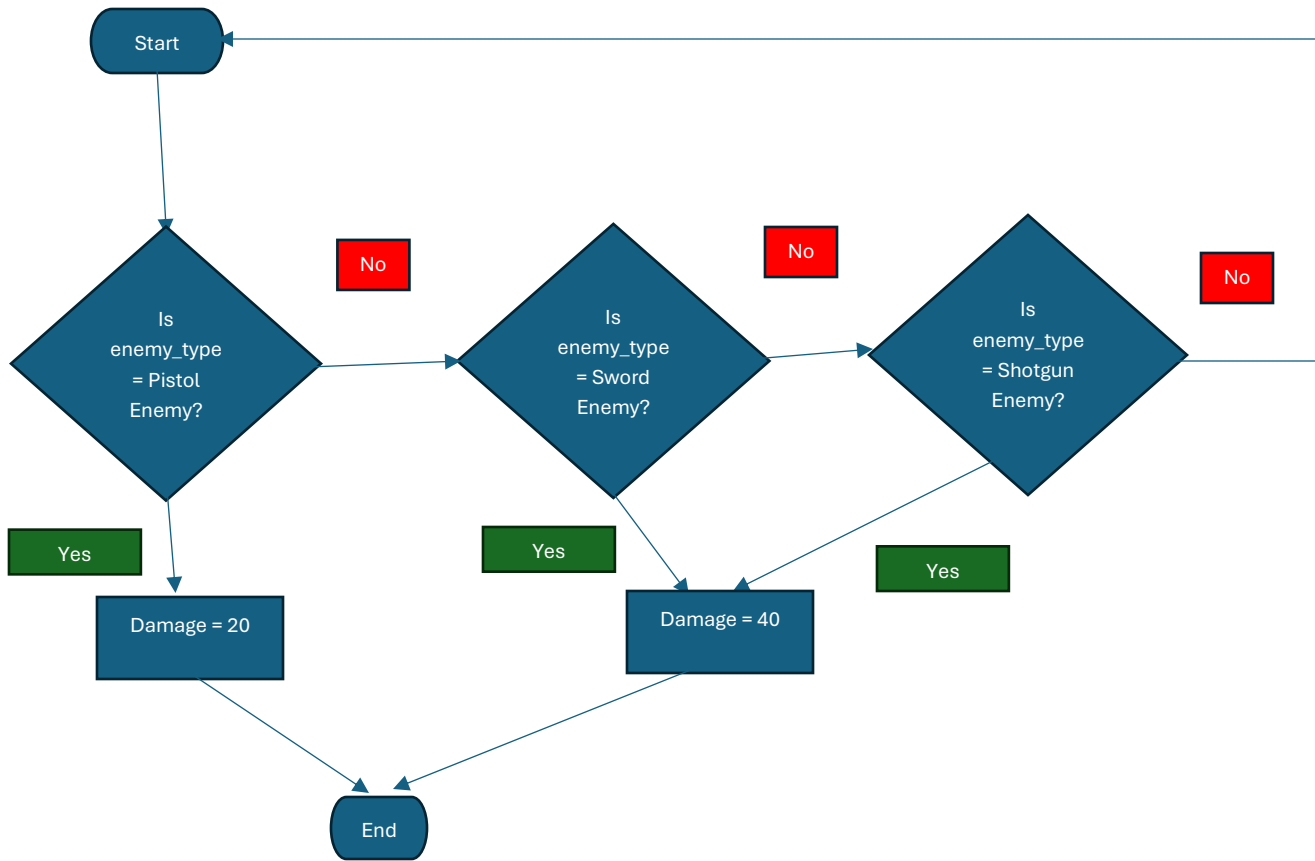
Enemy Movement



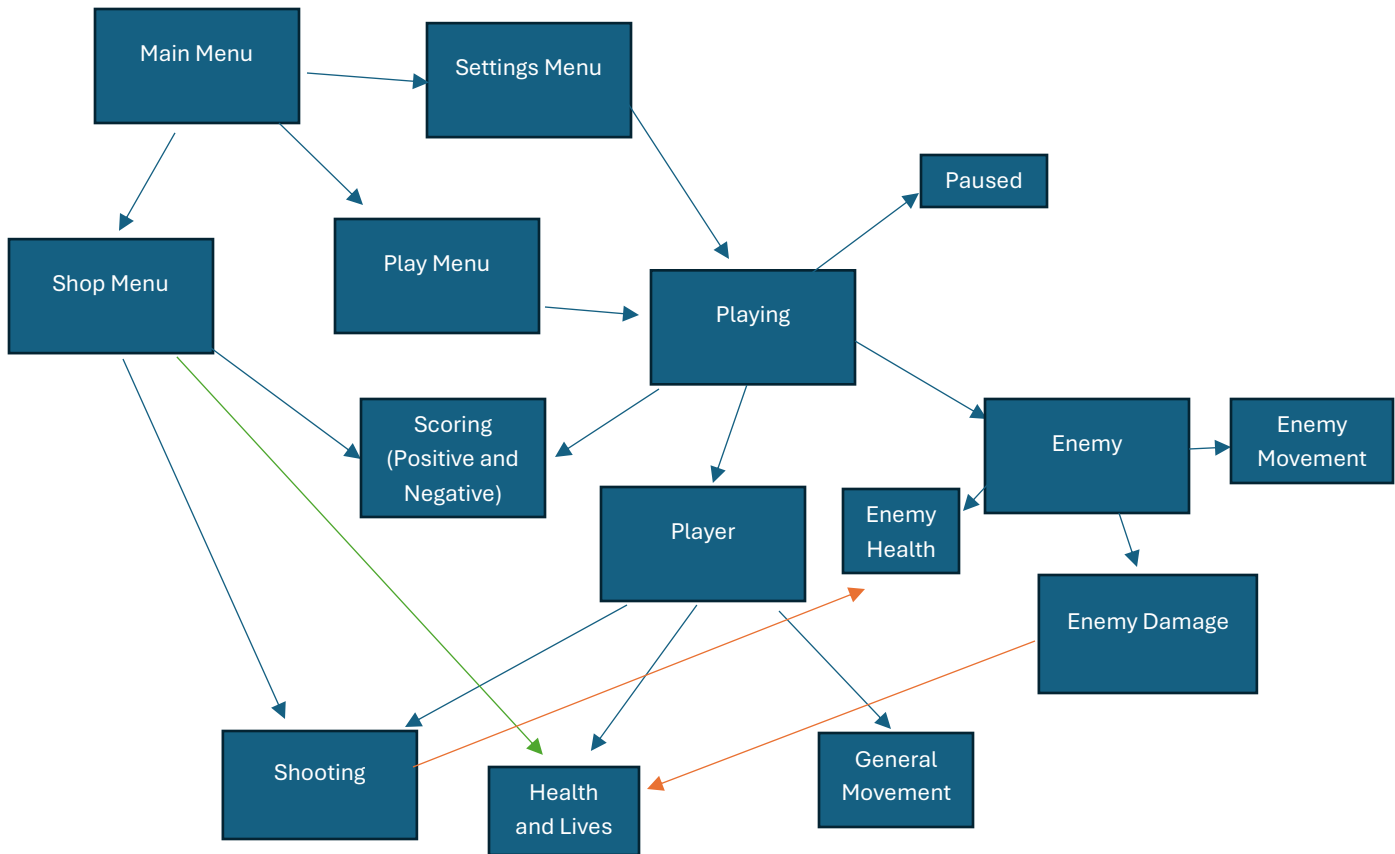
Enemy Health



Enemy Damage



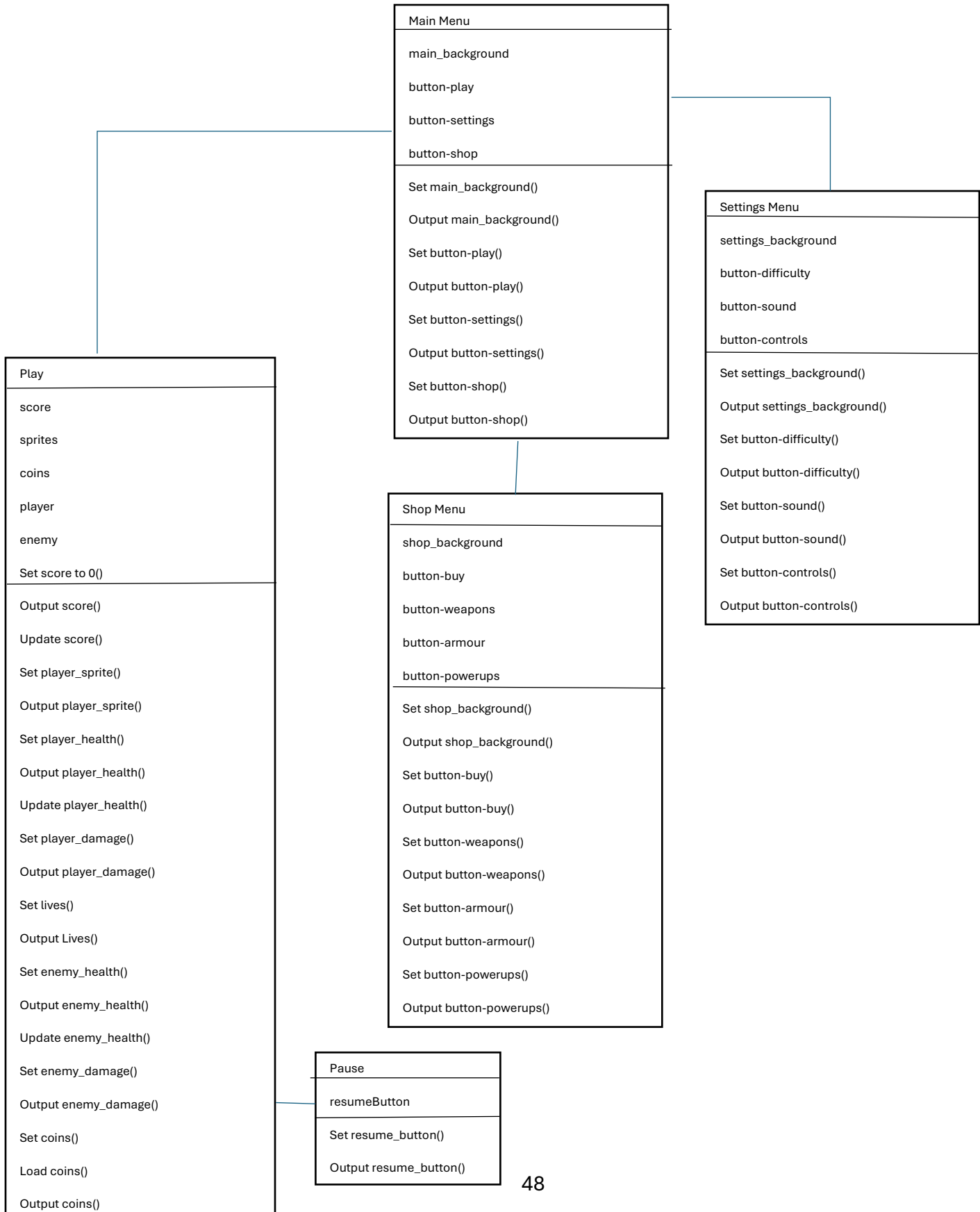
Linking the algorithms together



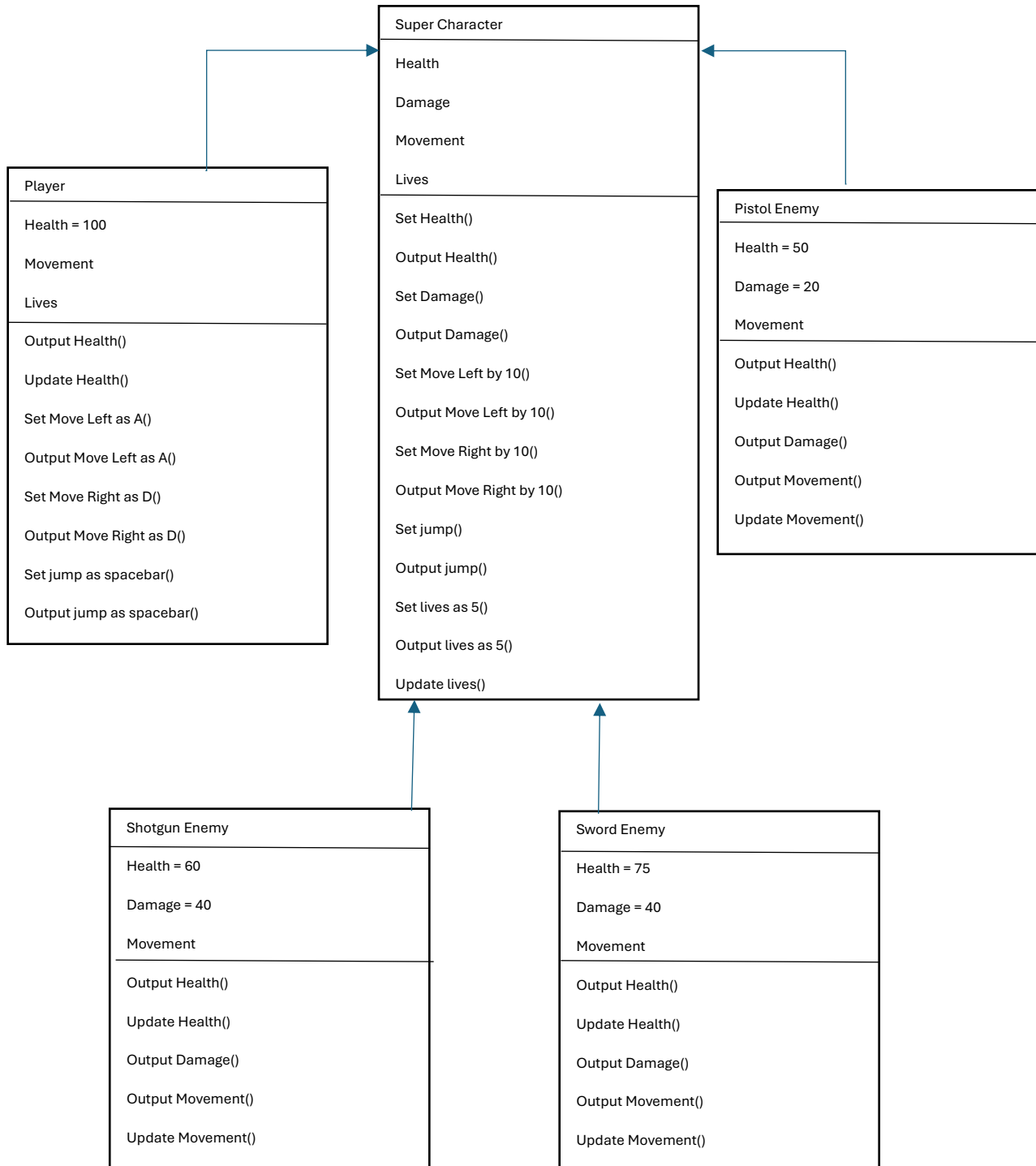
Class Diagrams

For this project, I decided to use Object Oriented Programming (OOP) to develop the solution. This is because OOP is much more suited for my project compared to alternative programming paradigms such as procedural programming. OOP will involve the creation of instances from classes as well as methods such as inheritance, which will be useful for my project as I make use of sprites for the players, enemies and the coins. To help set out what I plan to do for this project, I have made a Class Diagram which outlines the relationships between the different classes.

Menus-



Characters-



Test Data – Iterative Development

These tests shown below will make up the testing that will be conducted during the iterative development phase. This is done to help reduce the number of bugs that I may encounter as I develop the game, as well as help to reduce the number of bugs that may be spotted during the post development testing. In addition, it should also reduce the number of bugs that Ben will encounter when he uses the program.

In game Movement-

Test Data	Expected Result
W	Invalid
A	Valid
S	Invalid
D	Valid
Spacebar	Valid
Up Arrow	Invalid
Down Arrow	Invalid
Left Arrow	Invalid
Right Arrow	Invalid

Other in game Controls-

Test Data	Expected Result
Left Mouse Click	Valid
R	Valid
G	Valid
P	Valid
H	Invalid
J	Invalid
L	Invalid

Menu Controls

Test Data	Expected Result
Left Mouse Click	Valid
W	Invalid
S	Invalid
Escape key	Valid

Scoreboard validation – New high Score

Test Data	Expected Result
new_score > current_high_score	Valid
new_score < current_high_score	Invalid

Positive Scoring

Test Data	Expected Result
Coin Pickup	Valid
Enemy Killed	Valid
Moving	Invalid
Dying	Invalid
Falling	Invalid

Negative Scoring

Test Data	Expected Result
Dying	Valid – score decreases by 30
Using Powerups	Valid
Spending More Time	Valid
Killing Enemies	Invalid
Coin Pickup	Invalid
Moving	Invalid
Falling	Invalid

Timer

Test Data	Expected Result
Timer is on 3:59	Valid
Timer is on 164:45673	Invalid
Timer is on 0:00	Borderline

Game endings

Test Data	Expected Result
“Level Complete” when the user succeeds	Valid
“Level Failed” when the user fails	Valid
“Level Failed” when the user succeeds	Invalid
“Level Complete” when the user fails	Invalid

Paused

Test data	Expected Result
p	Valid
Left Click	Valid
h	Invalid

Shooting

Test Data	Expected Result
Every shot removes 1 from the total ammo count	Valid
Shots that hit the enemy impacts their health depending on the damage	Valid
Shots that don't hit any enemies can still damage nearby enemies	Invalid
Shots can go through objects	Invalid

Player Health and Lives

Test Data	Expected Result
Health = 100	Valid
Lives = 5	Valid
When player health = 0, lives go down by 1 and health resets back to 100 unless they're out of lives	Valid
Player doesn't lose a life when health = 0	Invalid
Player loses a life and respawns when they fall off a platform	Valid

Test Data – Post Development Testing

These tests will be used in the post development phase to ensure that all of the components work together and the entire coded solution works as intended. This ensures that the solution can be showcased to Ben in the best way possible.

Part A – Controls

Test Number	Test	Action	Expected Result
A1	Player Movement	VALID – A is pressed INVALID – J is pressed	VALID – Pressing A moves the player to the left INVALID – Pressing J will do nothing
A2	Player Movement	VALID – D is pressed INVALID – V is pressed	VALID – Pressing D moves the player to the right INVALID – Pressing V does nothing
A3	Player Movement	VALID – Spacebar is pressed INVALID – W is pressed	VALID – Pressing spacebar causes the user to jump INVALID – Pressing W does nothing
A4	Attacking	VALID – Left Mouse Click is pressed INVALID – Z is pressed	VALID – Pressing the Left Mouse Click causes the player to shoot INVALID – Pressing Z does nothing
A5	Pausing	VALID – P is pressed INVALID – Esc is pressed	VALID – Pressing P pauses/unpauses the game INVALID – Pressing Esc does nothing
A6	Using Powerups	VALID – G is pressed INVALID – M is pressed	VALID – Pressing G activates a powerup INVALID – Pressing M does nothing
A7	Menu Navigation	VALID – Left Mouse Click is pressed when hovering over a button	VALID – Pressing the Left Mouse Click allows user to access other parts

		INVALID – W is pressed	of the menu when selecting buttons INVALID – Pressing W does nothing
A8	Menu Navigation	VALID – Esc is pressed when not on the main menu INVALID – Esc is pressed when on the main menu	VALID – Pressing Esc when not on the main menu allows the user to return back to the main menu INVALID – Pressing Esc when in the main menu does nothing as there is nothing to return to

Part B – Collisions

Test Number	Test	Action	Expected Result
B1	Platforms	The player will move across the different platforms	The player should be able to walk on all platforms as intended
B2	Movement	The player will attempt to go through an enemy	The player should be unable to go through any enemy
B3	Player Attacks	The player will shoot at an enemy	The shot should hit the enemy and deal damage to it
B4	Enemy Attacks	The enemy will shoot at the player	The shot should hit the player and deal damage to them
B5	Coins	The player will move over a coin	The player should pick up a coin, it should add 1 to their total coin balance, and the coin should disappear.

Part C – Game States

Test Number	Test	Action	Expected Result
C1	Level Selection	VALID – Clicking an unlocked level INVALID – Pressing U in the level selector	VALID – This should load the level that has been selected by the user INVALID – This will do nothing as U is an invalid input in the menus
C2	Pausing	VALID – Pressing P while the game is playing INVALID – Pressing O whilst the game is playing	VALID – The game should become paused INVALID – Nothing will happen
C3	Unpausing	VALID – Pressing P while the game is paused INVALID – Pressing A whilst the game is paused.	VALID – Pressing P while the game is paused will resume the game INVALID – Nothing will happen
C4	Transition to shop menu	VALID – The “Shop” option is pressed INVALID – W is pressed	VALID – The game loads up the shop menu INVALID – Nothing happens as W is not a valid input
C5	Transition to settings	VALID – The “Settings” option is pressed INVALID – The “Play” option is pressed	VALID – The Settings menu loads up INVALID – The Play menu loads up
C6	“Level Failed” Screen	VALID – Fail the level INVALID – Press P while the game is running	VALID – The “Level Failed” Screen should show INVALID – The game should pause
C7	Closing the game	VALID – Pressing the X button at the top right of the screen	VALID – Closes the game INVALID – Loads the play menu

		INVALID – Pressing the “Play” option	
--	--	---	--

Part D – Scoring

Test Number	Test	Action	Expected Result
D1	The score will be null at the start of each level	The specific level will be loaded	The score will start at 0
D2	Coins	A coin will be picked up	The score will increase by 50
D3	Enemies	An enemy will be killed	The score will increase by 100
D4	Dying	The player will die	The player’s score will be reduced by 30
D5	Powerups	The player will use a powerup	The player’s score will be reduced by 10 for each powerup that is used
D6	Timer	VALID – The timer shows that there is less than 3 minutes remaining INVALID – The timer shows that there is more than 3 minutes remaining	VALID – The player’s score will be reduced by 30 INVALID – The player’s score will not be impacted
D7	Scoreboard Updating	VALID – The player will achieve a higher score than their current highest score on a specific level VALID – The player achieves a score that’s lower than their high score on a specific level	VALID – The scoreboard will update to show the new score as the best score and the previous high score will drop down a place as will any lower scores. VALID – The high score will remain where it is. If the new score is in the top 5 personal scores earned, then

			it will be placed accordingly in the top 5. Otherwise, it won't be saved.
--	--	--	---

Development

For this part of the project, I will be writing down what I do whenever I work on the development of the project. I figured that it would be best if I worked on each section of the project simultaneously e.g. I could work on gameplay and sort out the menus at any time. At the end of this development phase, I will conduct testing to help my evaluation later.

19/11/2024 – The Start

I wanted to start my project off by making the start menu, which is also the main menu. I wanted to incorporate my own buttons into this with the hand-drawn designs shown earlier. To create these buttons, I used Microsoft Paint to create these buttons, which would include the buttons to access the play, settings, and shop menus. Below is a screenshot of what I've done to start this project off. It already has comments to show what parts of the code do.

```
import pygame

pygame.init()
#initialises pygame
SCREEN_WIDTH = 1000#sets the screen width in pixels
SCREEN_HEIGHT = 800#sets the screen height in pixels

screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))#creates a 1000 by 800 screen
pygame.display.set_caption("Treasure Chase")#gives the window the name "Treasure Chase"

play_img = pygame.image.load('play_button.png').convert_alpha()
shop_img = pygame.image.load('shop_button.png').convert_alpha()
settings_img = pygame.image.load('settings_button.png').convert_alpha()
#loads their respective image onto the program, these are saved in the same folder
class Button():#Creates the button class
    def __init__(self, x, y, image):
        self.image = image
        self.rect = self.image.get_rect()#sets the image as a rectangle
        self.rect.topleft = (x, y)#gets the coordinate of the top left pixel of the image

    def draw(self): #allows the buttons to be drawn onto the screen
        screen.blit(self.image, (self.rect.x, self.rect.y))

play_button = Button(360, 150, play_img)
shop_button = Button(360, 300, shop_img)
settings_button = Button(360, 450, settings_img)
#Initialises the Button class
font = pygame.font.SysFont("Comic Sans MS",40)
#sets a font that I may want to use - will be used with the text class
class Text():#Creates the Text class
    def __init__(self, screen):
        self.screen = screen

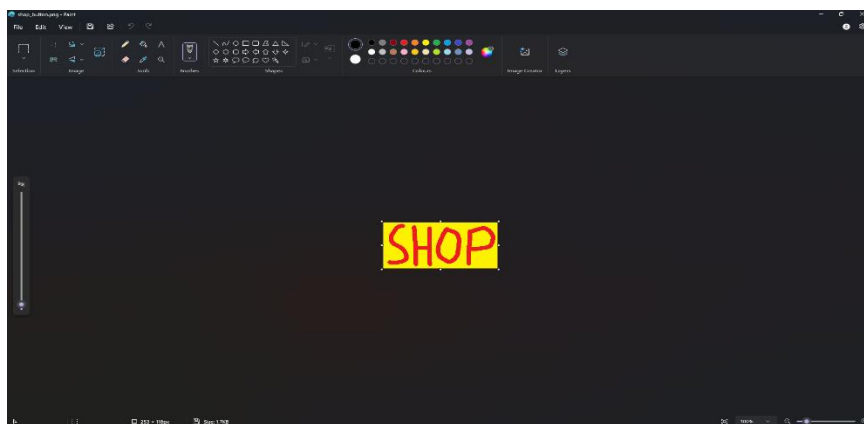
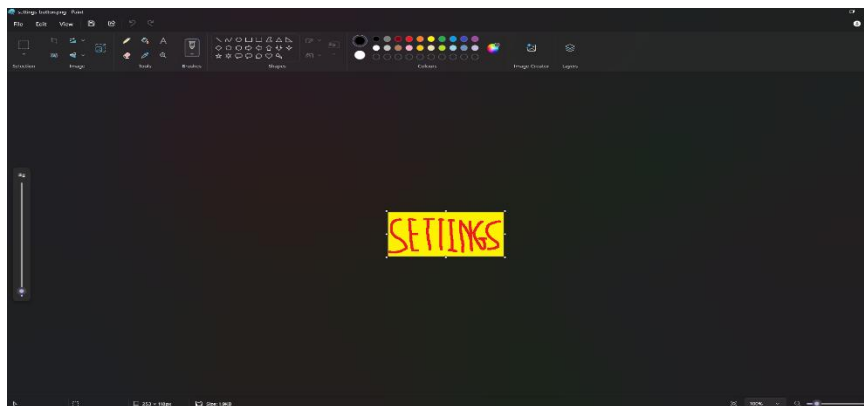
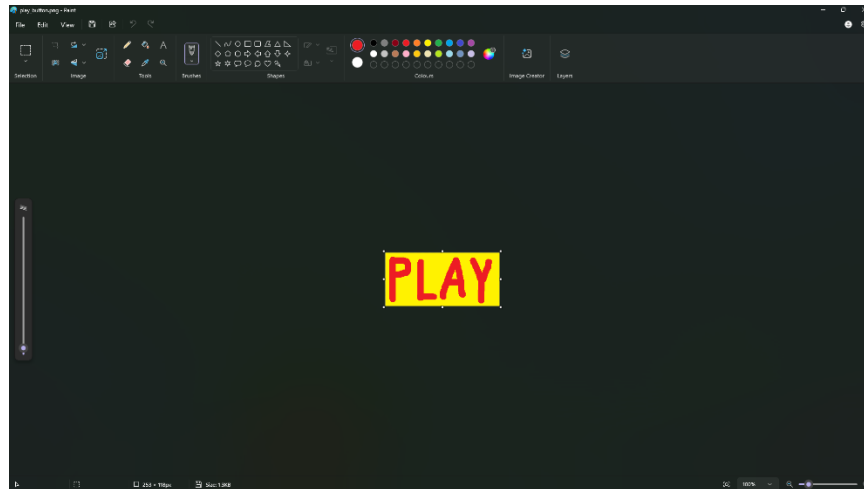
    def draw_text(self, text, font, text_col, x, y):#Allows the text to be drawn on the screen
        img = font.render(text, True, text_col)
        self.screen.blit(img, (x, y))

text_drawer = Text(screen)
#Initialises the Text class
run = True
while run:#Starts a while loop that'll run the program

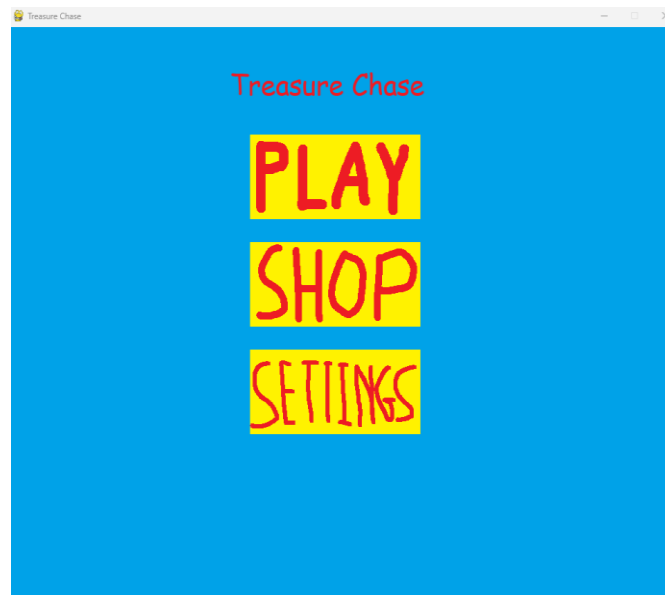
    screen.fill((0,162,232))#Fills the screen in Light Blue
    play_button.draw()
    shop_button.draw()
    settings_button.draw()
    #Draws the buttons
    text_drawer.draw_text("Treasure Chase", font, (237,28,36), 330, 50)
    #Draws text based on the parameters given
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            run = False
    #If the program notices that the user has pressed the x button at the top right of the screen, the program will close
    pygame.display.update()

pygame.quit()
```

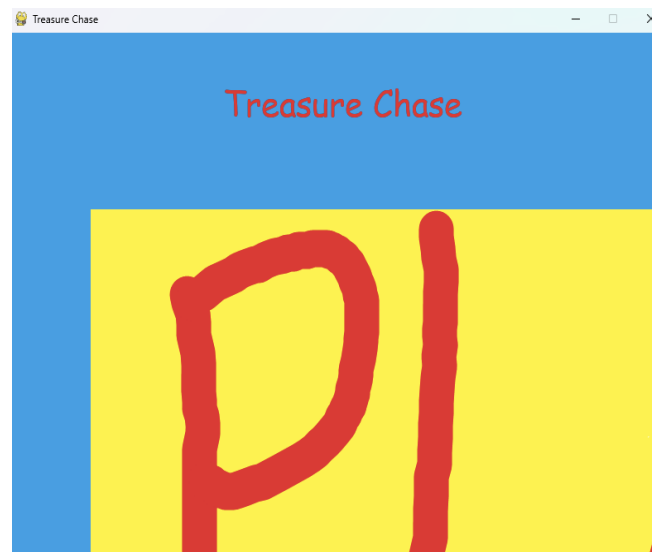
Below are also screenshots of the play, settings and menu buttons that I made on MS Paint:



So far, all of these aspects have lead to the program to look like this when it's running:



Whilst it does look basic, I'm fine with the way it looks for now. This is because my main priority was trying to get the buttons on the screen first. In the screenshots of the buttons above, you can see in the bottom left that the buttons are 253 x 118, which is quite small. However, I made this decision to make the button smaller as when I ran the program with the buttons at roughly over 1000 x 700, the following would happen:



As you can see with the screenshot shown, the button was way too big and not suitable at all for the project. In the future, I will work on the functionality of the actual buttons so they can load separate menus.

21/11/2024 – Giving the buttons functionality

The main objective of today is to attempt to allow the buttons to detect mouse movements and mouse clicks, so I can build up the menu. To do this, I planned on creating a new method called 'detect', which would detect mouse movements. I created the following code to attempt to complete this:

```
def detect(self):  
    pos = pygame.mouse.get_pos()  
    if self.rect.collidepoint(pos):  
        print ("Hello")
```

Detects the position of the mouse

If the mouse hovers over the button, it will print "Hello"

Before I attempt to register mouse clicks, I must first check that the buttons can detect when the user's mouse is hovering over the buttons.



Button Mouse
Hover Detection.mp

This here is an attached video showing myself testing this method. From the video, we can see that this does work and the program is now recognising when I hover over the buttons. Now I just need to add the ability to click the buttons. To do this, I can use `mouse.get_pressed()` to allow the program to accept mouse clicks. The method 'detect' now looks like this:

```
def detect(self):  
    pos = pygame.mouse.get_pos()  
    if self.rect.collidepoint(pos):  
        if pygame.mouse.get_pressed()[0] == 1:  
            print ("Hello")
```

Detects if a left mouse click has happened

If a left mouse click happens, it will print "Hello"

Below this is another attached video showing the testing of the updated method:

Test Data	Expected Result	Actual Result
Left Mouse Click	Valid	Valid
W	Invalid	Invalid



Button Mouse Click
Detection.mp4

Despite the clear success of the updated method in its ability to detect clicks, one major problem, which was shown in the video, is that one click would register as multiple clicks, which could prove to be a problem. However, this can be solved by setting True and False statements, shown in the screenshot below:

```
class Button():#Creates the button class
    def __init__(self, x, y, image):
        self.image = image
        self.rect = self.image.get_rect()#sets the image as a rectangle
        self.rect.topleft = (x, y)#gets the coordinate of the top left pixel of the image
        self.clicked = False

    def draw(self): #allows the buttons to be drawn onto the screen
        screen.blit(self.image, (self.rect.x, self.rect.y))

    def detect(self):
        pos = pygame.mouse.get_pos()
        if self.rect.collidepoint(pos):
            if pygame.mouse.get_pressed()[0] == 1 and self.clicked == False:
                self.clicked = True
                print ("Hello")
            if pygame.mouse.get_pressed()[0] == 0:
                self.clicked = False
```

This statement will be used later to prevent the program from registering multiple clicks

If the button is clicked while the check is false, the check will become true and Hello will be printed

When the button isn't clicked, the check automatically goes back to false – this is so if the user wants to press a button multiple times they can do that

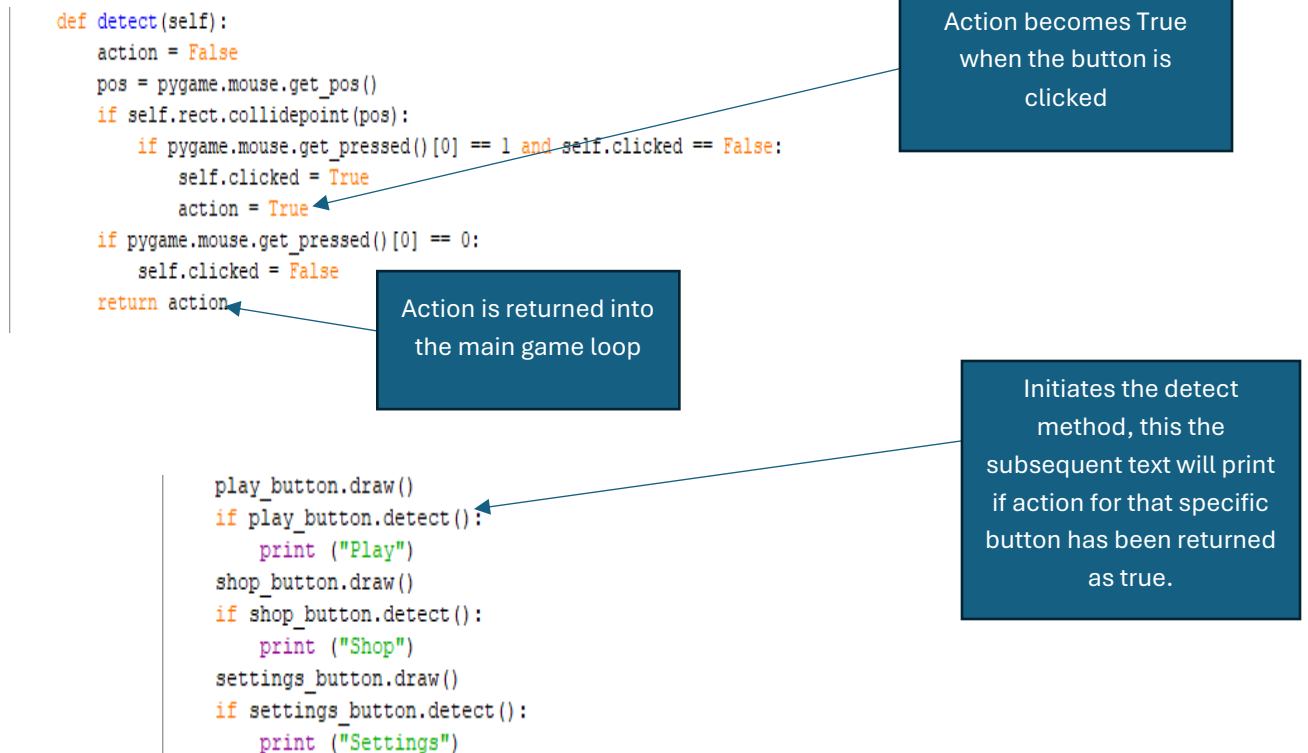
Here's another screen recording showing how the code works now:



Button Correct
Recognition.mp4

As shown in the recording, now when I click the button, “Hello” only appears once, indicating that the buttons are now registering the correct number of presses, which means that the buttons are almost working as intended. The only real problem now is that the buttons will need to show different outputs as all of the buttons have different intended

outputs. They currently do the exact same thing and differentiation is required with all of the buttons. I can create another variable, this time called action, which will once again be a Boolean variable. The screenshots below shows more:



Here's also a screen recording that shows how the detect method now works.

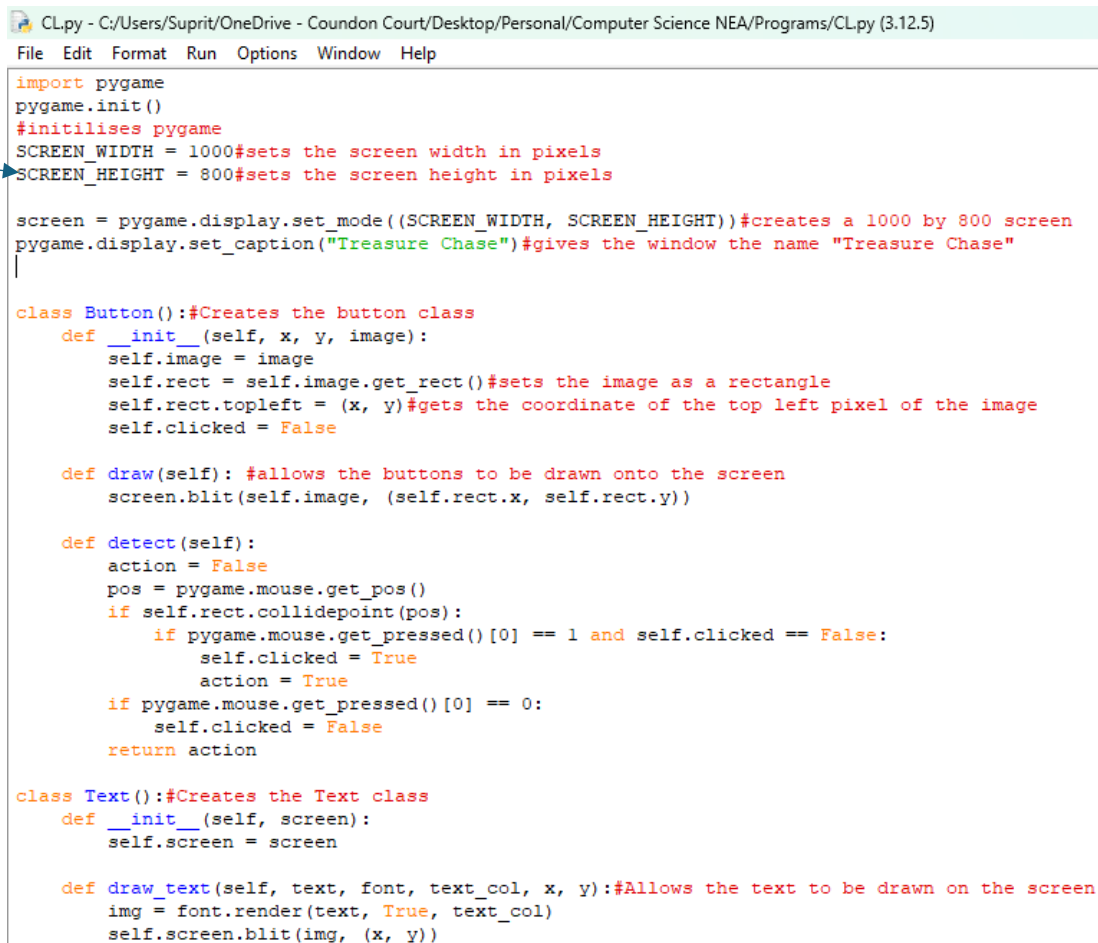


Unique Button
Recognition.mp4

As you can see, I have successfully managed to give the buttons their own functionality, so now each button can do their own unique thing and they can also be clicked using the left mouse click.

22/11/2024 – Creating gameplay

Today, I wanted to work on the gameplay and move away from the menus for a bit. This is because the gameplay is the main aspect of this program and is something that I should prioritise over the menus. The gameplay that I intend to create should provide a base for all of the levels. I will need to use the classes from the main program to do this. The first thing I chose to do is to place the classes in its own program. This is so all of the programs can access the classes without any problem. I created a separate file called “CL” to store all of the classes that I will use from now on.



```
CL.py - C:/Users/Suprit/OneDrive - Coundon Court/Desktop/Personal/Computer Science NEA/Programs/CL.py (3.12.5)
File Edit Format Run Options Window Help

import pygame
pygame.init()
#initilises pygame
SCREEN_WIDTH = 1000#sets the screen width in pixels
SCREEN_HEIGHT = 800#sets the screen height in pixels

screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))#creates a 1000 by 800 screen
pygame.display.set_caption("Treasure Chase")#gives the window the name "Treasure Chase"

class Button():#Creates the button class
    def __init__(self, x, y, image):
        self.image = image
        self.rect = self.image.get_rect()#sets the image as a rectangle
        self.rect.topleft = (x, y)#gets the coordinate of the top left pixel of the image
        self.clicked = False

    def draw(self): #allows the buttons to be drawn onto the screen
        screen.blit(self.image, (self.rect.x, self.rect.y))

    def detect(self):
        action = False
        pos = pygame.mouse.get_pos()
        if self.rect.collidepoint(pos):
            if pygame.mouse.get_pressed()[0] == 1 and self.clicked == False:
                self.clicked = True
                action = True
            if pygame.mouse.get_pressed()[0] == 0:
                self.clicked = False
        return action

class Text():#Creates the Text class
    def __init__(self, screen):
        self.screen = screen

    def draw_text(self, text, font, text_col, x, y):#Allows the text to be drawn on the screen
        img = font.render(text, True, text_col)
        self.screen.blit(img, (x, y))
```

The screen is still here as the classes need it to run correctly

Importing the classes made it much easier to create the pause button, which I created in the same way as all of the other buttons I’ve previously created. The screenshot below shows how I’ve imported the classes:


```

play.py - C:/Users/Suprit/OneDrive - Coundon Court/Desktop/Personal/Computer Science N...
File Edit Format Run Options Window Help

import pygame
import CL
pygame.init()

SCREEN_WIDTH = 1000
SCREEN_HEIGHT = 800

screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
pygame.display.set_caption("Treasure Chase")

pause_img = pygame.image.load('pause.png').convert_alpha()
pause_button = CL.Button(0,150,pause_img)
run = True
while run:

    screen.fill((0,162,232))
    pause_button.draw()
    if pause_button.detect():
        print("PAUSE")
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            run = False
        pygame.display.update()

pygame.quit()

```

This import statement imports all of the classes from the CL program

Here's a recording that shows that the pause button works:

Test data	Expected Result	Actual Result
p	Valid	Invalid
Left Click	Valid	Valid



Pause Button.mp4

Now that the pause button basically works (minus the menu of course) I wanted to create the player. To complete this I created a new class called Player to draw the player onto a specific position on the screen.

```

class Player():
    def __init__(self, image, x, y):
        self.image = pygame.image.load(image)
        self.x = x
        self.y = y

    def draw_player(self, screen):
        screen.blit(self.image, (self.x, self.y))

```

The class gets the sprite and the desired position of the sprite

This method draws the sprite onto the screen at the desired position

The code that moves the player is within the main game loop in the play program, as shown in the screenshot below:

```

player.draw_player(screen)
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        run = False
    elif event.type == pygame.KEYDOWN:
        if event.key == pygame.K_a:
            player.x -= 10
        elif event.key == pygame.K_d:
            player.x += 10
        elif event.key == pygame.K_SPACE:
            player.y -= 10
player.draw_player(screen)
pygame.display.update()

```

Player moves by 10 to the left when the A key is pressed

Player moves by 10 to the right when the D key is pressed

Player jumps up by 10 when the spacebar is pressed

Shown below is a recording that shows player movement:

Test Data	Expected Result	Actual Result
W	Invalid	Invalid
A	Valid	Valid
S	Invalid	Invalid
D	Valid	Valid
Spacebar	Valid	Valid
Up Arrow	Invalid	Invalid
Down Arrow	Invalid	Invalid
Left Arrow	Invalid	Invalid
Right Arrow	Invalid	Invalid



First Player Movement.mp4

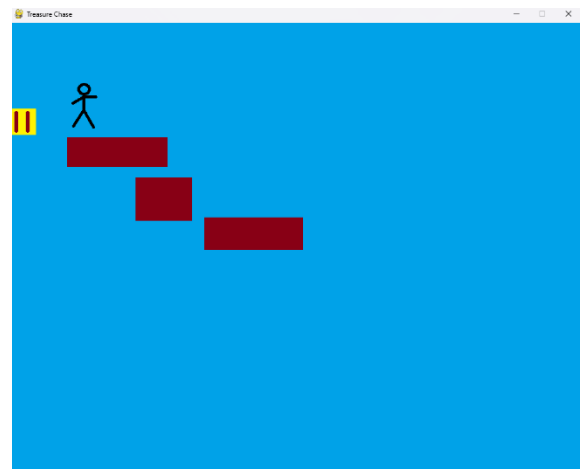
After playing around with this, I decided to change all of the values to 20. This was because I believed that the movement felt a bit sluggish and way too slow for the game, so I've increased the speed. Another obvious problem right now is that there is a clear lack of platforms within the game, which is bad as without these, the game will be unplayable for its main intention. To do this, I have created another class called Platform, which draws in the platforms. The code and result are shown below:

```

class Platform():
    def __init__(self, image, x, y):
        self.image = pygame.image.load(image)
        self.x = x
        self.y = y

    def draw_platform(self, screen):
        screen.blit(self.image, (self.x, self.y))

```



Whilst there are platforms, there is still no collisions or gravity, so I will need to add these next. This is because the player should be falling when they're not stood on a platform and the player can currently move through the platforms.

28/11/2024 – Adding Gravity and Collisions

I wanted to add gravity to the game first. To do this, I first created a new method called `update`, which would add the effect of gravity onto the player. I also updated the entire `Player` class, so that it used `.rect`, just like the buttons:

```
class Player():
    def __init__(self, image, x, y):
        self.image = image
        self.rect = self.image.get_rect()
        self.rect.topleft = (x, y)
        self.vel_y = 0
    def update(self):
        self.vel_y += GRAVITY
        self.rect.y += self.vel_y
    def draw(self):
        screen.blit(self.image, (self.rect.x, self.rect.y))
```

The velocity of the player falling is set at the same rate as the gravity and continuously increases

The y coordinate of the player updates with the gravity

I set the gravity value to 0.5 for a start, although I may amend this later down the line. Here's a recording showing how the code works now:



Gravity and
Collisions 1.mp4

As you can see, the gravity does take effect. However, the glaring problem is that the player cannot make any contact with the platforms, so I will need to find out how to do this. I think one way to do this is to amend the `Platform` class, so just like the `Player` class, it uses `.rect`. In addition, I will reduce the effect of the jump for now. This is because, as shown in the video, it's very easy to lose control of the character's jumping.



Attempt to fix
jumping and collision

The jumping in this video is much more controlled compared to the previous recording. However, the `.rect` didn't seem to fix the collisions. Therefore, I'll have to fix this. To do this,

I have created another new method within the Player class called `check_collision`, and this would help to ensure that the player can move along the platforms:

```
def check_collision(self, platforms):  
    for platform in platforms:  
        if self.rect.colliderect(platform.rect):  
            if self.vel_y > 0: # falling down  
                self.rect.bottom = platform.rect.top  
                self.vel_y = 0
```

Creates a loop that checks all of the platforms

Checks if the user has made contact with the platforms

In the main game loop, I've also created a for loop there that draws the platforms onto the screen. I've done this as it helps to reduce the lines of code that are in the program. I also added a clock that manages the frame rate. In the main game loop, I have set the frame rate of the game to 60.

```
platforms = [  
    CL.Platform (plat_1_img,100,200),  
    CL.Platform (plat_2_img,220,270),  
    CL.Platform (plat_3_img,340,340)  
]  
run = True  
while run:  
  
    clock.tick(60)  
    screen.fill((0,162,232))  
    pause_button.draw()  
    if pause_button.detect():  
        print("PAUSE")  
    player.draw()  
    player.update()  
    player.check_collision(platforms)  
    for platform in platforms:  
        platform.draw_platform()
```

The list where all the instances of platforms are stored

Runs the game at 60 frames per second (60fps)

Calls the method to check for collisions with the platforms

Draws all of the platforms in with a for loop (much more efficient compared to previous method of writing multiple draw methods)

Here's a recording showing how the game now works:



Fixed collisions.mp4

As you can see, the problem of no collisions with the platforms has been officially fixed, and now the player can move as intended.

29/11/2024 – Adding coins to the gameplay

Today, my main objective is to add coins onto the gameplay. Picking up the coins will be a contributor to the scoring, which I'll work on later down the line. To do this, I believe that I would have to make the class similarly to how I made the platforms. This is so there are collisions so the player can make contact with the coin.

```
class Coin():
    def __init__(self, image, x, y):
        self.image = image
        self.rect = self.image.get_rect()
        self.rect.topleft = (x, y)
    def draw_coin(self):
        screen.blit(self.image, (self.rect.x, self.rect.y))
    def check(self, player):
        return self.rect.colliderect(player.rect)
```

Similar
initiation and
draw methods
to platform
class

Check which
returns when the
player has made
contact with a coin

Similarly to the platforms, I'm using a list to store all instances of the coins and then in the actual game loop, I'm using a for loop that draws all of the coins onto the screen. Within the for loop, the method for checking if the player has directly contacted the coin is embedded within it, and the coin will be removed if the player has made contact with it.

```
coins = [
    CL.Coin(coin_img, 150, 100),
    CL.Coin(coin_img, 300, 150),
    CL.Coin(coin_img, 450, 200)
]

for coin in coins:
    coin.draw_coin()
    if coin.check(player):
        coins.remove(coin)
```

Draws all
of the
coins onto
the screen

Removes the coin
when the player
makes contact with it

All instances of coins
will be put into this
list

Here's a recording showing how the code works now and how the collisions of the coins work with this new code.



Coins.mp4

As shown in the video, the coins disappeared as they should when the player made contact with them. Even though the program was instructed to draw 3 coins, when the game is booted up, only 2 are showing. This is because the first coin is very close to the player, so it doesn't show up. Something I want to test is if it would be possible to use another for loop to create the coins. This is because I will probably not need to measure the positioning of the coins, which is something that I must do with the platforms, hence I will not do this for the platforms. However, when I ran the code now, the following error would come up:

```
ce NEA\Programs\play.py", line 49, in <module>
    coin.draw_coin()
AttributeError: 'list' object has no attribute 'draw_coin'
```

This error meant that the array couldn't work with the draw_coin method. I tried another way where all of the values would just be put into one list, but that also showed the same problem. As a result, I'm not sure how I can fix this and I'll just revert the code back to how it was when it was working. If I have time, I might come back to this and try to figure out another way to do this. For now, I'll start working on the contribution of the coins to the final score. To do this, I created a score variable and added a line where the collision detection happens where the score increases by 50 when the player makes contact with a coin. I've also added a line where the program shows on the screen the live score of the user.

Here's a recording showing how the scoring works:

```
for coin in coins:
    coin.draw_coin()
    if coin.check(player):
        coins.remove(coin)
        score += 50

score_text = f"Score: {score}"
text_drawer.draw_text(score_text, font, (237,28,36), 800, 750)
```

I've created score_text to output the result. This is because the draw_text method would have too many inputs if I put this within the brackets.

Test Data	Expected Result	Actual Result
Coin Pickup	Valid	Valid



Coins contributing
to score.mp4

As shown in the recording, the program is correctly picking up the coins and the score is being recorded correctly.

3/12/2024 – Adding lives to the game

I am planning to add lives to the game. To do this, I'll create a separate attribute called "lives" for the player class. I will set this to 5 as that's how many lives I want to have in the game, and I will add an extra line to the update method within the player class that takes away a life whenever the player makes contact with the ground.

```
lives_text = f"Lives: {player.lives}"
text_drawer.draw_text(lives_text, font, (237,28,36), 1550, 850)
if player.lives <= 0:
    print("Game Over")
    run = False
```

Text used to inform the user of how many remaining lives they have

Ends the game loop when the player has no more lives

```
class Player():
    def __init__(self, image, x, y):
        self.image = image
        self.rect = self.image.get_rect()
        self.rect.topleft = (x, y)
        self.vel_y = 0
        self.lives = 5
    def update(self):
        self.vel_y += GRAVITY
        self.rect.y += self.vel_y
        if self.rect.bottom >= SCREEN_HEIGHT:
            self.rect.bottom = SCREEN_HEIGHT
            self.vel_y = 0
            self.lives -= 1
```

The player's lives are set to 5

The player's lives are reduced by 1

Here's a recording showing how this new code works now:

Test Data	Expected Result	Actual Result
Player loses a life and respawns when they fall off a platform	Valid	Invalid
Lives = 5	Valid	Valid



Lives.mp4

As shown above, the lives are in the game. However, the player cannot respawn. So when they touch the ground, they end up losing many lives and end up dying. Therefore, I'll need to find a way to make the user respawn back on the first platform so they can still play the game as intended.

6/12/2024 – Adding a respawn feature

I believe that the best way to fix the problem of the game instantly ending when the user hits the floor is by adding a respawn feature that respawns the player back onto the first platform when they die until they run out of lives. To do this, I've created a method for the player class called `respawn()`, and this class would be responsible for respawning the player back at the starting position. I've also created 2 other attributes – `start_x` and `start_y`. These variables would be responsible for storing the starting position of the player, so when the player does die, they actually spawn back at the start. This method is called within the `update()` method, which is where the number of lives are also controlled.

```
def respawn(self):  
    self.rect.topleft = (self.start_x, self.start_y)  
    self.vel_y = 0
```

Respawns the
player back at the
start

Here's an updated recording showing how the respawn works in-game:

Test Data	Expected Result	Actual Result
Player loses a life and respawns when they fall off a platform	Valid	Invalid



Respawn.mp4

As shown in the recording, the player can now respawn, so the lives system has been sorted for now.

7/12/2024 – Creating the weapon for the user

With the lives sorted, the next things I plan to implement is the weapon and the bullets that will come out of the weapon. This can be done by creating separate classes for both the gun and the bullets that are released. The gun class consists of draw and shoot methods, whilst the bullet class consists of an update method instead, which updates the screen to keep the bullet moving.

```
class Bullet():
    def __init__(self, image, x, y):
        self.image = image
        self.rect = self.image.get_rect()
        self.rect.center = (x,y)
        self.speed = BULLET_SPEED

    def update(self):
        self.rect.x += self.speed

    def draw(self, screen):
        screen.blit(self.image, (self.rect.x, self.rect.y))

class Gun():
    def __init__(self, image, player, bullet_image):
        self.image = image
        self.player = player
        self.bullet_image = bullet_image
        self.cooldown = 20
        self.last_shot = 0
        self.rect = self.image.get_rect()
        self.rect.topleft = (170, 125)
        self.initial_position_set = False

    def draw(self, screen):
        if not self.initial_position_set:
            self.rect.topleft = (170, 125)
            self.initial_position_set = True
        else:
            self.rect.x = self.player.rect.centerx + 60
            self.rect.y = self.player.rect.top + 20
            screen.blit(self.image, self.rect.topleft)

    def shoot(self, current_frame):
        if current_frame - self.last_shot > self.cooldown:
            weapon = Bullet(self.bullet_image, self.player.rect.centerx + 60, self.player.rect.centery - 5)
            self.last_shot = current_frame
            return weapon
        return None
```

Allows gun object to move with player

Sets a cooldown between shots

Sets the speed of the bullet

Controls the rate at which the player shoots the gun

In the draw method, it may seem that random numbers had been used for the second part of the selection. However, these numbers were tactically chosen so the gun appears in the player's hand. If I didn't adjust this, what would've happened instead is that the gun would just appear on the player's head. Interestingly, in all of this, the bullets worked as intended.

```
bullets = []
gun = CL.Gun(img, player, bullet_img)
run = True
current_frame = 0
while run:
    clock.tick(60)
    screen.fill((0,162,232))
    pause_button.draw()
    if pause_button.detect():
        print("PAUSE")

    player.draw()
    player.update()
    player.check_collision(platforms)

    gun.draw(screen)
    for platform in platforms:
        platform.draw_platform()

    for coin in coins:
        coin.draw_coin()
        if coin.check(player):
            coins.remove(coin)
            score += 50

    for bullet in bullets:
        bullet.update()
        bullet.draw(screen)
        if bullet.rect.x > SCREEN_WIDTH:
            bullets.remove(bullet)

    score_text = f"Score: {score}"
    text_drawer.draw_text(score_text, font, (237,28,36), 150, 850)
    lives_text = f"Lives: {player.lives}"
    text_drawer.draw_text(lives_text, font, (237,28,36), 1550, 850)
    if player.lives <= 0:
        print("Game Over")
        run = False
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            run = False
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_a:
                player.rect.x -= 20
            elif event.key == pygame.K_d:
                player.rect.x += 20
            elif event.key == pygame.K_SPACE:
                player.vel_y = -10
        elif event.type == pygame.MOUSEBUTTONDOWN:
            if event.button == 1:
                bullet = gun.shoot(current_frame)
                if bullet:
                    bullets.append(bullet)

    player.draw()
    pygame.display.update()
    current_frame += 1
```

Stores all the bullet instances that take place

For loop which activates when there is an instance in 'bullets'

Test Data	Expected Result	Actual Result
Left Mouse Click	Valid	Valid

Here's a recording showing how the shooting now works:



Start Weapon.mp4

As shown in the video, the shooting now successfully works, so I can focus elsewhere for now.

10/12/2024 – Creating the Enemy

Now, I need to create an enemy class that attacks and can kill the player. It should be able to move randomly around the map and the player. I have created a new class called Enemy, which will allow the enemies to be drawn on as well as allow the enemies to move around the map.

```
class Enemy():
    def __init__(self, image, x, y, speed):
        self.image = image
        self.rect = self.image.get_rect()
        self.rect.topleft = (x, y)
        self.speed = speed
        self.vel_y = 0
        self.on_platform = False
    def update(self, platforms):
        if not self.on_platform:
            self.vel_y += GRAVITY
            self.rect.y += self.vel_y
            self.rect.x += self.speed
            self.on_platform = False
            for platform in platforms:
                if self.rect.colliderect(platform.rect):
                    if self.vel_y > 0:
                        self.rect.bottom = platform.rect.top
                        self.vel_y = 0
                        self.on_platform = True
            if self.rect.right >= SCREEN_WIDTH or self.rect.left <= 0:
                self.speed = -self.speed
    def draw(self, screen):
        screen.blit(self.image, self.rect.topleft)
```

Boolean statement which is false if the enemy is not on a platform

Sets the speed of the enemy

Draws the enemy

Initially sets the speed of the enemy falling to 0

Enemy falls at the rate of gravity that was set

For loop which runs if an enemy falls until they either die or until they land on another platform.

Enemy bounces off edges of screen

The other screenshot below represents the iteration which draws the enemies in. It also contains a check to see if an enemy has fallen off and has failed to land on a platform.

Draws the enemy onto the screen

```
for enemy in enemies:
    enemy.update(platforms)
    if enemy.rect.top > SCREEN_HEIGHT:
        enemies.remove(enemy)
    else:
        enemy.draw(screen)
```

Removes enemy if they fall off

Here's a recording which

shows how the code works now:



First Enemies.mp4

As shown in the video, whilst the enemies do spawn in, they don't attack the player at all and they run away from the player and fall to their deaths, so I'll have to fix this.

13/12/2024 – Fixing the enemy class (Part 1)

To fix this problem, I've edited the update method so the enemy calculates the location of the player based off the player's central x-coordinate and the enemy moves towards the player accordingly.

```

def update(self, player, platforms):
    if not self.on_platform:
        self.vel_y += GRAVITY
        self.rect.y += self.vel_y
        #calculate direction towards player
        direction_x = player.rect.centerx - self.rect.centerx
        if direction_x != 0:
            direction_x = direction_x // abs(direction_x)
        #move towards player
        self.rect.x += direction_x * self.speed

```

Finds difference between x-coordinate of player's centre and x-coordinate of enemy

Normalises direction_x into -1 (move left) if negative, 1 (move right) if positive, or 0 (no movement) if 0

Here's a recording that shows how the code works now (for now, I've intentionally added a fourth platform and kept one of the enemies there intentionally):

Test Data	Expected Result	Actual Result
Shots can go through objects	Invalid	Valid



Enemy and Gun Interaction.mp4

As the video shows, the enemy now goes towards the player. The enemy on the other side was spawned in to show that the enemies do still die when they fall. However, there are still a few problems. Firstly, as there is no spawn protection of any sort, when the enemy does reach the platform where the player respawns, the player instantly dies. In addition, the bullets deal no damage to the enemies and instead go through the enemies. I've got another screen recording which unfortunately highlights a separate problem. I've slightly adjusted the code to highlight this issue:



No Enemy
Jump.mp4

As shown, because the enemy cannot actually jump, there could be situations where the enemy cannot reach the player at all and it creates this situation where the enemy does not move at all.

22/12/2024 - Fixing the enemy class (Part 2)

The first thing I want to fix is the fact that the bullets do not make contact with the enemies. To do this, I added a method to the bullet class which would check for collisions with enemies.

```
def check_collision(self, enemies):
    for enemy in enemies:
        if self.rect.colliderect(enemy.rect):
            return enemy
    return None
```

Checks if there was any
contact between a bullet
and any enemy

```
for bullet in bullets:
    bullet.update()
    enemy_hit = bullet.check_collision(enemies)
    if enemy_hit:
        enemies.remove(enemy_hit)
        bullets.remove(bullet)
        score = score + 100
    elif bullet.rect.x > SCREEN_WIDTH:
        bullets.remove(bullet)
    else:
        bullet.draw(screen)
```

Removes the bullet
and the enemy when
it gets hit

Adds 100 to the
player's score

Here's a screen recording which shows how the updated code works:

Test Data	Expected Result	Actual Result
Shots can go through objects (Shooting)	Valid	Valid
Enemy Killed (Positive Scoring)	Valid	Valid



Enemy and Gun
Interaction 2.mp4

Checks if there are any platforms in front of the enemy

As shown in the video, the enemies can now be killed and killing them contributes to the score. The next thing I wanted to do was give the enemies the ability to jump, so if there is a platform that is too far away for the enemies to walk to, they can jump to reach those platforms. To do this, I changed the update method for the enemy slightly so that the enemy could detect when there wasn't a platform in front of them and they would be able to jump to attempt to reach another platform with the risk of dying.

```
Check if the player is at a higher position or there's no platform in front
if not any(platform.rect.collidepoint(self.rect.centerx + direction x * self.speed, self.rect.bottom + 1) for platform in platforms):
    if self.on_platform and self.vel_y == 0: # Jump only if on platform and not already jumping/fall
        self.vel_y = -10 # Jump velocity
```

Enemy jumps when there is no platform and they cannot double jump

Here's a recording showing how the game works now.



Jumping Enemies.mp4

As shown in the recording, the enemies can now jump across gaps to get to platforms so they can attempt to get to the player.

28/12/2024 – Creating final few features (Part 1)

I wanted to implement both a timer and the finish flag into the game. I created a class for the finish flag which involves one method to draw it onto the screen and another method to check if the player has made any contact with the flag.

```
class FinishLine():
    def __init__(self, image, x, y):
        self.image = image
        self.rect = self.image.get_rect()
        self.rect.topleft = (x, y)
    def draw(self, screen):
        screen.blit(self.image, self.rect.topleft)
    def check(self, player):
        return self.rect.colliderect(player.rect)

finish_line.draw(screen)

if finish_line.check(player):
    print("You Win!")
    run = False
```

Draws the finish flag onto the screen

Checks to see if the player makes contact with the flag

Draws the flag onto the screen

Prints "You Win!" when the player finishes and closes the game.

Here's a recording which shows how the game works now:



Finish Line.mp4

As shown in the recording, the game now ends when the player touches the flag and the game recognises that the player completed the level.

The next feature that I wanted to add was a timer that would count down from 5 minutes and when the timer reached 0, the player would fail the level and the game would end. To do this, I will use the `time.get_ticks()` method to track the time. I will initially format it into milliseconds, then convert it into minutes and seconds. The game will automatically fail when the timer hits 0.

```
# Timer setup
start_ticks = pygame.time.get_ticks() # Get the initial time
total_time = 5 * 60 * 1000 # 5 minutes in milliseconds

timer_rect = pygame.Rect(5,5,65,35)

seconds = (pygame.time.get_ticks() - start_ticks)//1000
remaining_time = total_time - (seconds * 1000)

minutes = remaining_time // 60000
seconds = (remaining_time % 60000)//1000

pygame.draw.rect(screen, (0,0,0), timer_rect)
time_text = f"{minutes}:{seconds:02d}"
text_drawer.draw_text(time_text, font2, (237,28,36), 10, 10)

if remaining_time <= 0:
    print ("Time's up! You failed!")
    run = False
```

Converts
remaining
time to
minutes
and
seconds

Rectangle which acts
as a background for
the timer.

Calculates elapsed time

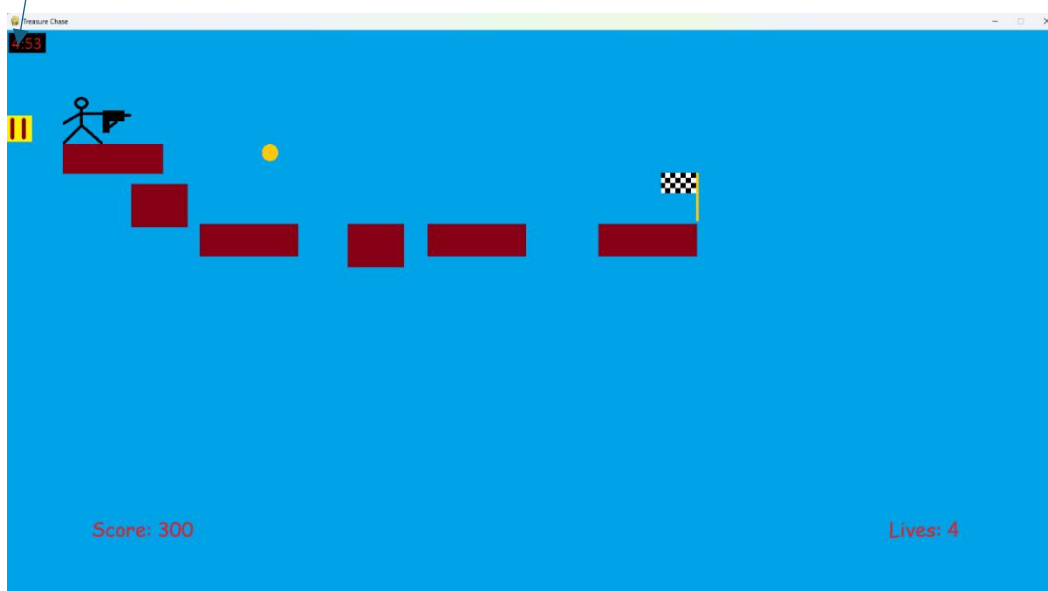
Calculates
remaining time in
milliseconds

Draws black rectangle
on screen

Draws the timer
onto the screen

Displays message when the
time runs out.

Timer showing
remaining time



```

Python 3.12.5 (tags/v3.12.5:ff3bc82, Aug 6 2024, 20:45:27) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\Suprit\OneDrive - Coundon Court\Desktop\Personal\Computer Science NEA\Programs\play.py
pygame 2.6.0 (SDL 2.28.4, Python 3.12.5)
Hello from the pygame community. https://www.pygame.org/contribute.html
Time's up! You failed!

```

Message for when the user runs out of time

Test Data	Expected Result	Actual Result
Timer is on 3:59	Valid	Valid
Timer is on 164:45673	Invalid	Invalid
Timer is on 0:00	Borderline	Borderline

As shown with the screenshots, the timer works as intended, counts the time correctly, and shows the correct message when there is no time left.

30/12/2024 – Creating final few features (Part 2)

The next thing that I wanted to do was to fix the pause button. Whilst the user can definitely interact with it, it doesn't actually pause the game. To make it pause the game, I plan to create another screen that would go on top of the current screen. I've created a function which shows the separate screen, and this function will be executed either when the user presses the pause button or when they press 'P'.

```

def pause_screen():
    pause = True
    font = pygame.font.SysFont("Comic Sans MS", 50)
    while pause:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                quit()
            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_p: # Press 'p' to resume
                    pause = False

    screen.fill((0, 0, 0))
    text = font.render("Paused", True, (255, 255, 255))
    screen.blit(text, (SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 - text.get_height() // 2 - 400))
    text = font.render("Press 'P' to resume", True, (255, 255, 255))
    screen.blit(text, (SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 + text.get_height() // 2 - 100))
    pygame.display.update()
    clock.tick(5)

```

Draws the text onto the screen

Here's a screen recording showing how this works now:

Test data	Expected Result	Actual Result
p	Valid	Valid
Left Click	Valid	Valid
h	Invalid	Invalid



Pause function.mp4

As shown, the pause button works and successfully pauses the game. However, if the pause button was pressed to pause the game, when the game resumes, the player automatically shoots. I'm currently not sure what could be causing this as the left mouse click is not being pressed when I resume the game. After looking into the problem, it seems that because I use the mouse to pause the game the same way the player would use it to shoot, the game thinks the player is shooting. I'm not too familiar with this problem, so I'll leave it for now and I will attempt to resolve this issue later.

Next, I wanted to add a screen that shows up when you finish the level. This would include the options to reset the level and close the game as well as show the score.

```
def end_screen(score, text_drawer, screen, clock):
    end = True
    font = pygame.font.SysFont("Comic Sans MS", 50)
    small_font = pygame.font.SysFont("Comic Sans MS", 30)

    while end:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                quit()
            elif event.type == pygame.MOUSEBUTTONDOWN:
                mouse_pos = pygame.mouse.get_pos()
                if replay_button_rect.collidepoint(mouse_pos):
                    return True # Replay the game
                elif quit_button_rect.collidepoint(mouse_pos):
                    pygame.quit()
                    quit()

        screen.fill((0, 0, 0))
        text_drawer.draw_centered_text(f"Your Score: {score}", font, (255, 255, 255), SCREEN_WIDTH, SCREEN_HEIGHT - 200)
        text = font.render("You Won!", True, (255, 255, 255))
        screen.blit(text, (SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 - text.get_height() // 2 - 400))

        replay_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2, 300, 50)
        quit_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2 + 100, 300, 50)

        pygame.draw.rect(screen, (0, 255, 0), replay_button_rect)
        text_drawer.draw_centered_text("Replay", small_font, (0, 0, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 50)

        pygame.draw.rect(screen, (255, 0, 0), quit_button_rect)
        text_drawer.draw_centered_text("Quit", small_font, (0, 0, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 250)

        pygame.display.update()
        clock.tick(60)
```

2 different fonts for the screen

Allows user to either replay or close the game.

Draws 2 buttons onto the screen which allows the user to play again or close the game.

Here's a recording showing how it now works:

Test Data	Expected Result	Actual Result
"Level Complete" when the user succeeds	Valid	Valid

"Level Failed" when the user succeeds	Invalid	Invalid
---------------------------------------	---------	---------



Finish Screen and
Replay.mp4

As shown, the functionality works as intended and there were no apparent problems with the implementation.

10/01/2025 – Adding the remaining ending screens

The remaining end screens that are left are for when the user runs out of lives and when the user runs out of time. I can approach this in a very similar way to how I approached the end screen with a function.

```
def time_screen(score, text_drawer, screen, clock):
    time = True
    font = pygame.font.SysFont("Comic Sans MS", 50)
    small_font = pygame.font.SysFont("Comic Sans MS", 30)

    while time:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                quit()
            elif event.type == pygame.MOUSEBUTTONDOWN:
                mouse_pos = pygame.mouse.get_pos()
                if replay_button_rect.collidepoint(mouse_pos):
                    return True # Replay the game
                elif quit_button_rect.collidepoint(mouse_pos):
                    pygame.quit()
                    quit()

        screen.fill((0, 0, 0))
        text_drawer.draw_centered_text(f"Your Score: {score}", font, (237, 28, 36), SCREEN_WIDTH, SCREEN_HEIGHT - 200)
        text = font.render("You ran out of time!", True, (237, 28, 36))
        screen.blit(text, (SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 - text.get_height() // 2 - 400))

        replay_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2, 300, 50)
        quit_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2 + 100, 300, 50)

        pygame.draw.rect(screen, (255, 0, 0), replay_button_rect)
        text_drawer.draw_centered_text("Replay", small_font, (0, 0, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 50)

        pygame.draw.rect(screen, (255, 0, 0), quit_button_rect)
        text_drawer.draw_centered_text("Quit", small_font, (0, 0, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 250)

        pygame.display.update()
        clock.tick(60)
```

```
def live_screen(score, text_drawer, screen, clock):
    time = True
    font = pygame.font.SysFont("Comic Sans MS", 50)
    small_font = pygame.font.SysFont("Comic Sans MS", 30)

    while time:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                quit()
            elif event.type == pygame.MOUSEBUTTONDOWN:
                mouse_pos = pygame.mouse.get_pos()
                if replay_button_rect.collidepoint(mouse_pos):
                    return True # Replay the game
                elif quit_button_rect.collidepoint(mouse_pos):
                    pygame.quit()
                    quit()

        screen.fill((0, 0, 0))
        text_drawer.draw_centered_text(f"Your Score: {score}", font, (237, 28, 36), SCREEN_WIDTH, SCREEN_HEIGHT - 200)
        text = font.render("You died!", True, (237, 28, 36))
        screen.blit(text, (SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 - text.get_height() // 2 - 400))

        replay_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2, 300, 50)
        quit_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2 + 100, 300, 50)

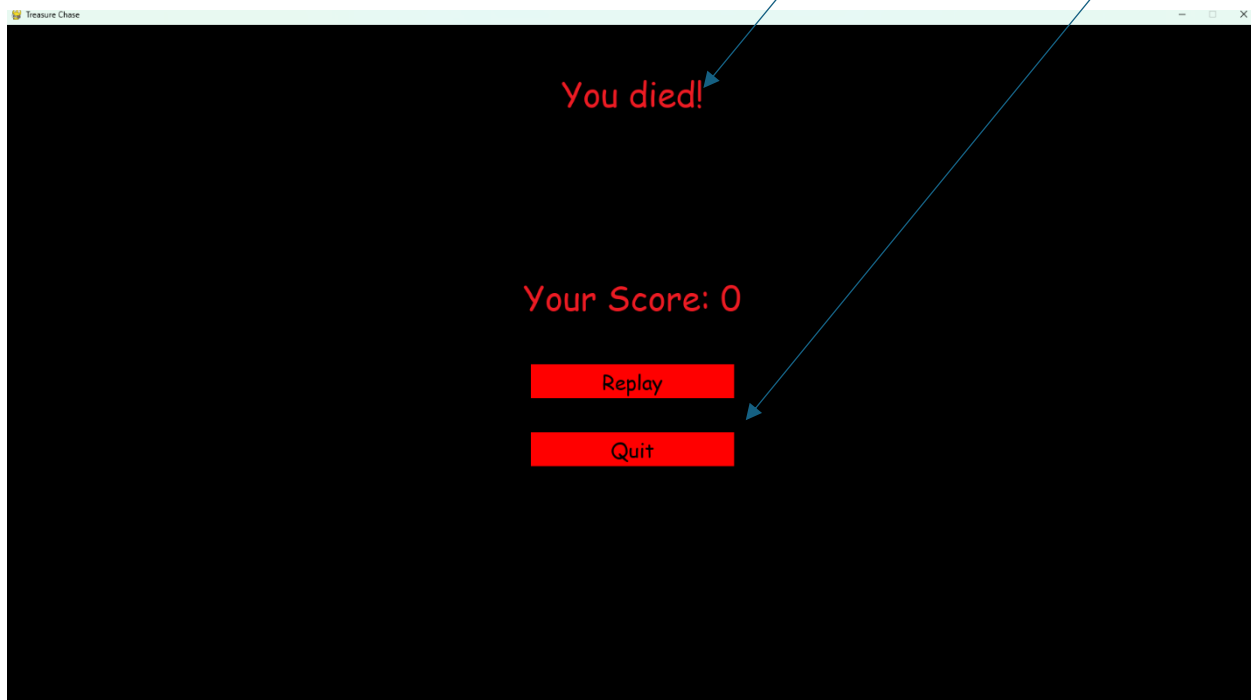
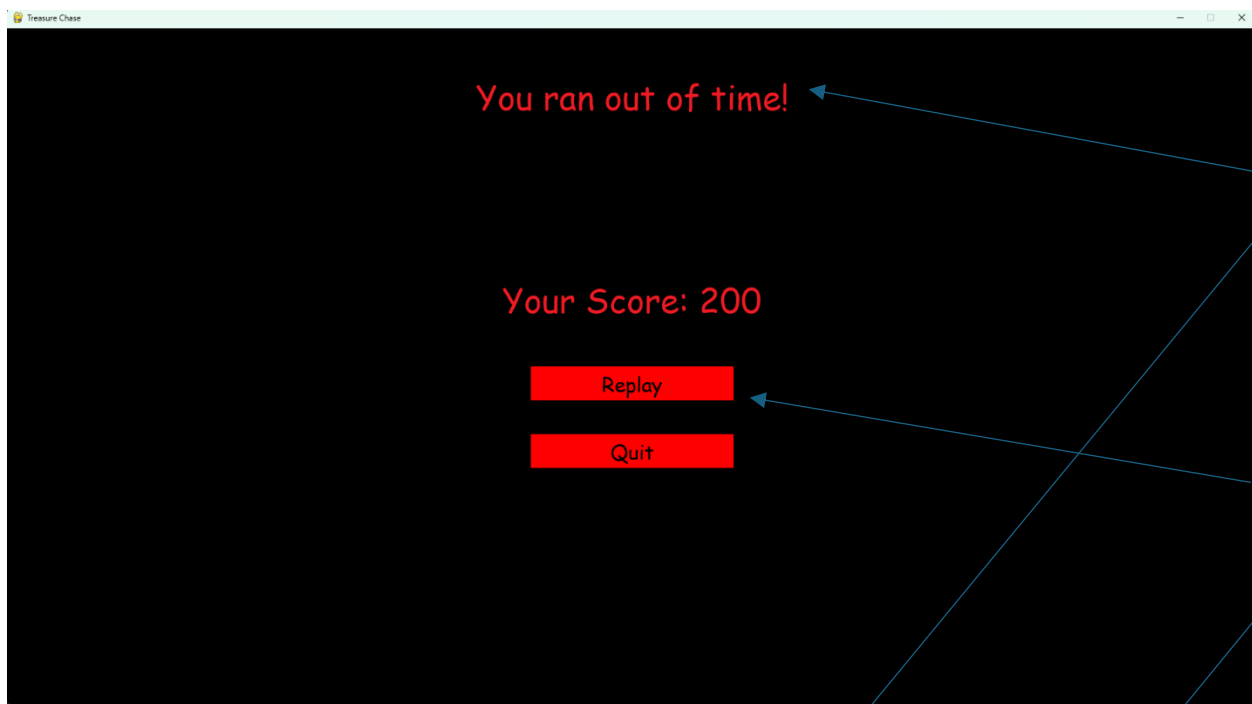
        pygame.draw.rect(screen, (255, 0, 0), replay_button_rect)
        text_drawer.draw_centered_text("Replay", small_font, (0, 0, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 50)

        pygame.draw.rect(screen, (255, 0, 0), quit_button_rect)
        text_drawer.draw_centered_text("Quit", small_font, (0, 0, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 250)

        pygame.display.update()
        clock.tick(60)
```

Adjusted the message and the colours to suit the ending

Gives the user the option to either play again or completely close the game.



Test Data	Expected Result	Actual Result
"Level Failed" when the user fails	Valid	Valid
"Level Complete" when the user fails	Invalid	Invalid

10/01/2025 – Adding and fixing some gameplay features

Now that the vast majority of the playable game works as intended, I now wanted to adjust the code in a few areas to change the way they worked. Once sorting that out, I would want to adjust the positioning of many things on the map and add more coins, enemies, and platforms to the game. The first thing I wanted to do was to adjust the movement of the player. This is because when you move the player left or right, you cannot hold down the corresponding key, so the user has to constantly tap the key until they have gotten to the place where they want to go. This will also ensure that the player's movement is much more smooth compared to how it is currently. To do this, I've taken all of the keys away from the 'for event' loop as it would mean that the keys couldn't be held down.

```
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        run = False
keys = pygame.key.get_pressed()
if keys[pygame.K_a]:
    player.rect.x -= 5
if keys[pygame.K_d]:
    player.rect.x += 5
if keys[pygame.K_p]:
    pause_screen()
if keys[pygame.K_SPACE]:
    player.vel_y = -10
if event.type == pygame.MOUSEBUTTONDOWN:
    if event.button == 1:
        bullet = gun.shoot(current_frame)
        if bullet:
            bullets.append(bullet)
```

Changed the amount the player moves by as they go flying with the original values of 20

Allows keys to be assigned with a key press

Because this code is outside the 'for event' loop, instead of updating every event, it now updates every loop, which is much faster and allows the user to hold down the keys to move.

The next thing I wanted to do was fix the jumping. Before and after the changes I made above had occurred, the user could just spam the spacebar and it could just cause them to fly, which isn't ideal for any platformer. To do this, I've added a Boolean statement that will change depending on whether the player has made contact with any platforms.

```
def check_collision(self, platforms, enemies, score):
    self.on_ground = False #resets every time this function takes place
    for platform in platforms:
        if self.rect.colliderect(platform.rect):
            if self.vel_y > 0: # falling down
                self.rect.bottom = platform.rect.top
                self.vel_y = 0
                self.on_ground = True #player is on solid ground
```

Only true when the player is on a platform

```

if keys[pygame.K_SPACE]:
    if player.on_ground: #only jumps if player is on solid ground
        player.vel_y = -10

```

'if' statement checks if the player is on a platform

Because of this additional check, if the player ever attempts to jump when mid-air, nothing will happen now as the addition of the Boolean statement blocks any jumping from taking place.

Next, I wanted to slightly adjust the scoring. Earlier, it was discussed that when the player loses a life, their score should be decreasing by 30. However, when the player dies, they don't lose anything. To do this, I've created a new method for the player class called 'die', which checks when the player has lost a life. This method is called in both the 'update' and 'check_collision' methods. It removes a life from the player and calls the 'respawn' method to respawn the player.

```

def update(self):
    self.vel_y += GRAVITY
    self.rect.y += self.vel_y
    if self.rect.bottom >= SCREEN_HEIGHT:
        self.rect.bottom = SCREEN_HEIGHT
        self.vel_y = 0
        if self.die():
            return True
    return False

def die(self):
    self.lives -= 1
    self.respawn()
    return True

def draw(self):
    screen.blit(self.image, (self.rect.x, self.rect.y))

def check_collision(self, platforms, enemies):
    self.on_ground = False #resets every time this function takes place
    for platform in platforms:
        if self.rect.colliderect(platform.rect):
            if self.vel_y > 0: # falling down
                self.rect.bottom = platform.rect.top
                self.vel_y = 0
                self.on_ground = True #player is on solid ground
    for enemy in enemies:
        if self.rect.colliderect(enemy.rect):
            self.lives -= 1
            if self.die():
                return True
            else:
                return False

```

Happens when the player loses a life and executes the method

```

player.draw()

if player.update() or player.check_collision(platforms, enemies):
    score -= 30

```

Because the check within the methods depends on if self.die is true or false, I can write a check like this which deducts the player's score by 30 when they die.

Here's a recording showing the testing of this feature:

Test Data	Expected Result	Actual Result
Dying	Valid – score decreases by 30	Valid – score decreases by 30



Score deduction via
lives.mp4

As shown, this works as intended and there are no problems with it.

16/01/2025 – Linking the Main Menu and level together

The next thing I wanted to do was link the Main Menu and the actual game that I've created together. To do this, I created a subprogram called 'start_game' and placed pretty much the entire code that makes up the main game in this subprogram. This is because I want to be able to access this from the code that makes up the main menu. The next thing I did was create 2 more functions, this time in the file where my main menu was located. One of the functions 'pl', which activates when pressing the play button, opens a sub menu, which works as a level selector, with 2 buttons, one which returns the user back to the previous menu and a button called 'Levels'. Once this button is clicked, the second function, called 'lvl', loads up another sub-menu, where you can once again return back to the main menu, or you can press 'Level 1', which loads up the gameplay.

```

def pl():
    pl = True
    font = pygame.font.SysFont("Comic Sans MS", 40)
    small_font = pygame.font.SysFont("Comic Sans MS", 30)
    while pl:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                quit()
            elif event.type == pygame.MOUSEBUTTONDOWN:
                mouse_pos = pygame.mouse.get_pos()
                if level_button_rect.collidepoint(mouse_pos):
                    lvl()
                elif return_button_rect.collidepoint(mouse_pos):
                    return False
        screen.fill((0, 162, 232))
        text = font.render("Play Menu", True, (255,255,255))
        screen.blit(text, (SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 - text.get_height() // 2 - 400 ))

        level_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2, 300, 50)
        return_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2 + 100, 300, 50)

        pygame.draw.rect(screen, (0, 255, 0), level_button_rect)
        text_drawer.draw_centered_text("Levels", small_font, (255, 242, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 50)

        pygame.draw.rect(screen, (255, 0, 0), return_button_rect)
        text_drawer.draw_centered_text("Return", small_font, (255, 242, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 250)

        pygame.display.update()
        clock.tick(60)

def lvl():
    lvl = True
    font = pygame.font.SysFont("Comic Sans MS", 40)
    small_font = pygame.font.SysFont("Comic Sans MS", 30)
    while pl:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                quit()
            elif event.type == pygame.MOUSEBUTTONDOWN:
                mouse_pos = pygame.mouse.get_pos()
                if level_button_rect.collidepoint(mouse_pos):
                    play.start_game()
                    return True
                elif return_button_rect.collidepoint(mouse_pos):
                    return False
        screen.fill((0, 162, 232))
        text = font.render("Play Menu", True, (255,255,255))
        screen.blit(text, (SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 - text.get_height() // 2 - 400 ))

        level_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2, 300, 50)
        return_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2 + 100, 300, 50)

        pygame.draw.rect(screen, (0, 255, 0), level_button_rect)
        text_drawer.draw_centered_text("Level 1", small_font, (255, 242, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 50)

        pygame.draw.rect(screen, (255, 0, 0), return_button_rect)
        text_drawer.draw_centered_text("Return", small_font, (255, 242, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 250)

        pygame.display.update()
        clock.tick(60)

```

Loads the level selector when the appropriate button is pressed

Determines size and position of buttons

Loads the main gameplay when the appropriate button is pressed

Determines size and position of buttons

Here's a screen recording which shows how this new version of the whole program works:

Test Data	Expected Result
Left Mouse Click	Valid
W	Invalid
S	Invalid
Escape key	Valid



Updated Version of the game.mp4

As shown in the video, the menus and gameplay link quite well together and can help improve the user experience of this game.

17/01/2025 – Creating a database to store scores and coins that the player gets

The player needs to be able to see their highest scores in their own personal leaderboard. In addition, the player also needs to earn coins from the levels to be able to buy better equipment from the shop. In order to achieve this, I'll need to create a database that'll store all of the player's scores and the total number of coins they have. It must be able to detect when the player has achieved a higher score and update the scoreboard appropriately. In addition, it must also be able to detect when the player has spent coins and deduct the correct amount of coins from the player, as well as reward the player appropriately for the number of coins they pick up by increasing the number of coins that they have to spend. I am planning to use sqlite3 for this as I already have some familiarity with SQL.

```
import sqlite3
con = sqlite3.connect("treasureChase_data.db")
cur = con.cursor()
cur.execute("CREATE TABLE high_scores(score)")
cur.execute("CREATE TABLE coin_total(coins)")
```

Connects (or creates it if it doesn't exist) to a database called treasureChase_data

Creates 2 separate tables to store the number of coins and all of the scores that the player achieves.

After running this once, it creates the tables and the database and there is no need to run lines 04 and 05 again as the tables now exist. If I try to run this exact program again, this error pops up:

```
= RESTART: C:/Users/Suprit/OneDrive - Coundon Court/Desktop/Personal/Computer Science NEA/Programs/database.py
Traceback (most recent call last):
  File "C:/Users/Suprit/OneDrive - Coundon Court/Desktop/Personal/Computer Science NEA/Programs/database.py", line 4, in <module>
    cur.execute("CREATE TABLE high_scores(score)")
sqlite3.OperationalError: table high_scores already exists
```

Because the table already exists, the program recognises that and prints out an error.

Because of this, I will import this database file into the other files that will interact with the database.

I wanted to try this with the score in the main game loop. This would involve me importing the database file to that program and when the game would end, the score should be added to the database.

```
while live:
    database.cur.execute("""
        INSERT INTO high_score VALUES
            (score)
    """)
```

Happens during the while loop which loads each ending

Should insert the final score into the database.

However, when the program runs, if I don't remove the 'CREATE TABLE' line, the error from earlier understandably pops up. However, when I reach the end screen, this unexpected error occurs:

```
pygame 2.6.0 (SDL 2.28.4, Python 3.12.5)
Hello from the pygame community. https://www.pygame.org/contribute.html
Traceback (most recent call last):
  File "C:\Users\Suprit\OneDrive - Coundon Court\Desktop\Personal\Computer Science NEA\Programs\testing.py", line 96, in <module>
    pl()
  File "C:\Users\Suprit\OneDrive - Coundon Court\Desktop\Personal\Computer Science NEA\Programs\testing.py", line 39, in pl
    lvl()
  File "C:\Users\Suprit\OneDrive - Coundon Court\Desktop\Personal\Computer Science NEA\Programs\testing.py", line 70, in lvl
    play.start_game()
  File "C:\Users\Suprit\OneDrive - Coundon Court\Desktop\Personal\Computer Science NEA\Programs\play.py", line 231, in start_game
    result = live_screen(score, text_drawer, screen, clock) # Call the end screen function and check if replay is chosen
  File "C:\Users\Suprit\OneDrive - Coundon Court\Desktop\Personal\Computer Science NEA\Programs\play.py", line 126, in live_screen
    database.cur.execute("""
sqlite3.OperationalError: no such table: high_score
```

This is very bad, as the program doesn't recognise that the table actually exists.

This is a problematic situation, as there doesn't seem to be a middle ground solution where I can call the table and the program actually recognises it without any issues.

In addition, I currently do not know how I can fix these issues therefore I will have to do extra research to ensure that I can figure out what is the cause of this issue. One way I was thinking about solving this issue could be to have a check which could create the table if the program claims it doesn't exist.

After doing some research, I came up with a solution. The lines where a table would be created would be reworded to "CREATE TABLE IF NOT EXISTS", which would create the table if the program didn't recognise the existence of a table. I also created 2 subprograms which would insert the coins and the score into the database. I haven't quite finished on the subprogram for the coins as I would have to add or subtract the amount of coins with the amount in the database.

```
import sqlite3
con = sqlite3.connect("treasureChase_data.db")
cur = con.cursor()
cur.execute("CREATE TABLE IF NOT EXISTS high_scores(score INT)")
cur.execute("CREATE TABLE IF NOT EXISTS coin_total(coins INT)")

def insert_score(score):
    cur.execute("INSERT INTO high_scores (score) VALUES (?)", (score,))
    con.commit()
```

If the table doesn't exist, a new version is created and it stores the score as an int

Ensures that the changes are reflected in the database

Inserts the score into the table

I also slightly adjusted the main game loop so that the insert_score subprogram is called just before the function displaying the end screen shows. I ran the program twice, where I

earned -90 by killing myself until I ran out of lives. I then performed a 'SELECT' statement to find all of the scores that were stored in the 'high_score' table. This was the result of it:

```
= RESTART: C:/Users/Suprit/OneDrive - Coundon Court/Desktop/Personal/Computer Science NEA/Programs/database.py  
(-90,) ←  
(-90,)
```

Both of the scores that I got
are shown

As shown in the screenshot, it correctly displays the contents of the table high_score, which stores the scores.

18/01/2025 – Creating the personal leaderboard

Today, I wanted to use the database I made to make the leaderboard. To do this, I created 2 functions. One would retrieve the 5 highest scores earned by the player, and the other would display the top 5 scores earned by the player.

```
def get_top_5_scores():  
    cur.execute("SELECT score FROM high_scores ORDER BY score DESC LIMIT 5")  
    scores = cur.fetchall()  
    return scores  
  
def display_top_5_scores(screen, text_drawer):  
    leader = True  
    font_2 = pygame.font.SysFont("Comic Sans MS", 40)  
    while leader:  
        for event in pygame.event.get():  
            if event.type == pygame.QUIT:  
                pygame.quit()  
                quit()  
            elif event.type == pygame.MOUSEBUTTONDOWN:  
                mouse_pos = pygame.mouse.get_pos()  
                if return_button_rect.collidepoint(mouse_pos):  
                    return False  
        screen.fill((0, 162, 232))  
        top_scores = get_top_5_scores() ←  
        y_offset = 300  
        text_drawer.draw_text("Top 5 High Scores:", font, (255, 242, 0), 780, 150)  
        for rank, score in enumerate(top_scores, start=1):  
            text_drawer.draw_text(f"{rank}. {score[0]}", font, (255, 242, 0), 800, y_offset) ←  
            y_offset += 50  
        return_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2 + 100, 300, 50)  
        pygame.draw.rect(screen, (255, 0, 0), return_button_rect)  
        text_drawer.draw_centered_text("Return", font_2, (255, 242, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 250)  
        pygame.display.update()  
        clock.tick(60)
```

Calls function to
get scores

Selects the 5
highest scores
from the highest to
lowest

Prints the rank
and the score
onto the screen

'enumerate' adds a counter to all
of the data. It's told that the first
item should be ranked as 1

Top 5 High Scores:

1. 350
2. 350
3. 350
4. 320
5. 300

Return

Shows the 5 highest scores that the user has gotten

Button to return to play menu

Test Data	Expected Result	Actual Result
new_score > current_high_score	Valid	Valid
new_score < current_high_score	Invalid	Invalid

23/01/2025 – Creating a counter to show the coins

To create the shop, I would need to make the game so that it could increase the

```
def display_coin_total(screen, text_drawer, font_2):
    total_coins = database.get_coin_total()
    coin_text = f"{total_coins}"
    text_drawer.draw_text(coin_text, font, (255, 201, 14), 1700, 30)
```

coin total every time the player picks up a coin. I already had the function to insert the coins into the program. I just applied this into the 'check' method within the

Retrieves the total amount of coins in the database

Prints the total amount of coins in the database

coin class, so the program could increase the number of coins in the database by 1. Next, I created another function that would show the number of coins in the database.

Here's a recording showing how this works.



coin_balance.mp4

As shown in the video, the program can successfully recognise when the user has picked up a coin. I've placed this in the main menu for now as I haven't created the shop yet.

24/01/2025 – Creating the Shop (Part 1)

The shop would include items which can be brought to make the player stronger. Firstly, I

```
def shop():
    shop = True
    while shop:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                quit()
            elif event.type == pygame.MOUSEBUTTONDOWN:
                mouse_pos = pygame.mouse.get_pos()
                if return_button_rect.collidepoint(mouse_pos):
                    return False

        screen.fill((136,0,21))
        pygame.draw.rect(screen, (136,0,21), coin_rect)
        text = font.render("Shop", True, (255,242,0))
        screen.blit(text, (SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 - text.get_height() // 2 - 400))
        text = font.render("Buy anything you desire!", True, (255,242,0))
        screen.blit(text, (SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 + text.get_height() // 2 - 300))

        coins = [CL.Coin(shop_coin_img, 1650, 45)]
        for coin in coins:
            coin.draw_coin()
        display_coin_total(screen, text_drawer, font_2)

        return_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 880, SCREEN_HEIGHT // 2 + 330, 120, 45)

        pygame.draw.rect(screen, (0,0,0), return_button_rect)
        text_drawer.draw_centered_text("Return", font_2, (255,242,0), 200, 1700)

        pygame.display.update()
        clock.tick(60)
```

The return button is the only button that currently works for now

wanted to draw up the base first to get this process out of the way first. There currently is no way to buy any items as of now, although that should be sorted out later.

This is currently what the shop looks like:



As the screenshot shows, it's currently lacking the main feature to actually buy items using the coins. I might have to do some additional research to figure out the best way to approach this.

The first thing I wanted to focus on was showing the different weapons. I went onto paint and drew up 2 images which can be used to show the 2 different weapons that will be in the game for now. I used the Button class to draw them onto the screen. I haven't used the detect method yet, but I might use it so that the user can click on the weapon and view additional information and buy the weapon.

```
start_showcase_img = pygame.image.load('start_showcase.png').convert_alpha()
start_show = CL.Button(600,350,start_showcase_img)
start_show.draw()
relic_showcase_img = pygame.image.load('relic_showcase.png').convert_alpha()
relic_show = CL.Button(923,350,relic_showcase_img)
relic_show.draw()
```

Loads and
draws both
of the
weapons
onto the
screen

This is what it looks like now:



The next thing I wanted to do was add the detect method so that the user could press the weapon to view it's details.

```
start_showcase_img = pygame.image.load('start_showcase.png').convert_alpha()
start_show = CL.Button(600,350,start_showcase_img)
start_show.draw()
if start_show.detect():
    start()
relic_showcase_img = pygame.image.load('relic_showcase.png').convert_alpha()
relic_show = CL.Button(923,350,relic_showcase_img)
relic_show.draw()
if relic_show.detect():
    relic()
```

Detects when
the user presses
the weapons

However, when the code runs, the game doesn't work as intended. The information screen for the weapon called start shows up when the user enters the shop, which shouldn't happen. The worst part is that when the return button is pressed, the program doesn't go to the shop menu. Here's a video showing this problem in action:



shop_not_working.
mp4

To potentially fix this problem, I moved the lines where the image is called and the Button class is called to outside the while loop to attempt to fix this.

```
def shop():
    shop = True
    start_showcase_img = pygame.image.load('start_showcase.png').convert_alpha()
    start_show = CL.Button(600,350,start_showcase_img)
    relic_showcase_img = pygame.image.load('relic_showcase.png').convert_alpha()
    relic_show = CL.Button(923,350,relic_showcase_img)
    while shop:
```



shop_partially_wor
king.mp4

This attached video shows that the user can now access the shop and the separate weapon pages. However, when the shop button is clicked, the weapon page for the start weapon pops up, even though this shouldn't happen. One potential fix that I hadn't tried yet was to change the way the game would detect mouse clicks. Currently, I was using the .detect() method from my Button class to detect when the mouse made contact with the button. I figured this might've been the cause of the problem, and I changed the code

so that the buttons would use `.rect.collidepoint()` to detect collisions, just like the return button and a lot of the other buttons.

```
elif event.type == pygame.MOUSEBUTTONDOWN:
    mouse_pos = pygame.mouse.get_pos()
    if start_show.rect.collidepoint(mouse_pos):
        start()
    elif relic_show.rect.collidepoint(mouse_pos):
        relic()
```

Detects the position of the mouse to determine when the mouse makes contact with the button

Here's a video showing how the code works now:



no more incorrect
shop screens.mp4

As shown, the screens don't randomly show up anymore and the shop is almost working as intended.

26/01/2025 – Creating the shop (Part 2)

Because I want to create a second weapon for the shop, I will need to create the second weapon, To do this, I've made it so the weapon class is now a super class and the 2 weapons will be subclasses that inherit the attributes and methods from the main weapon class.

```
class StartWeapon(Weapon):
    def __init__(self, image, player, bullet_image):
        super().__init__(image, player, bullet_image)
        self.cooldown = 20

class RelicWeapon(Weapon):
    def __init__(self, image, player, bullet_image):
        super().__init__(image, player, bullet_image)
        self.cooldown = 15

    def shoot(self, current_frame):
        if current_frame - self.last_shot > self.cooldown:
            bullet = Bullet(self.bullet_image, self.player.rect.centerx + 60, self.player.rect.centery - 5)
            bullet.piercing = True
            self.last_shot = current_frame
            return bullet
        return None
```

Inherits all of these
attributes from the main
class

I also added buttons to equip the 2 weapons. The equip functionality of the Relic weapon will only work once the weapon is brought for 150 coins. Currently, this hasn't been properly tested yet as I don't have the coins to buy the weapon.

```

if start_show.rect.collidepoint(mouse_pos):
    start()
elif start Equip_rect.collidepoint(mouse_pos):
    equip_weapon("Start")
elif relic_show.rect.collidepoint(mouse_pos):
    relic()
elif relic Equip_rect.collidepoint(mouse_pos):
    if purchase_relic():
        equip_weapon("Relic")

```

Checks to see if the player has brought the weapon

```

def purchase_relic():
    current_coins = database.get_coin_total()
    if current_coins >= 150:
        database.insert_coins(-150)
        return True
    else:
        return False

```

Will cost the user 150 coins, if they don't have enough, then the program won't allow them to buy this weapon.

```
player_weapon = "Start"
```

Sets the default weapon to the start weapon

```

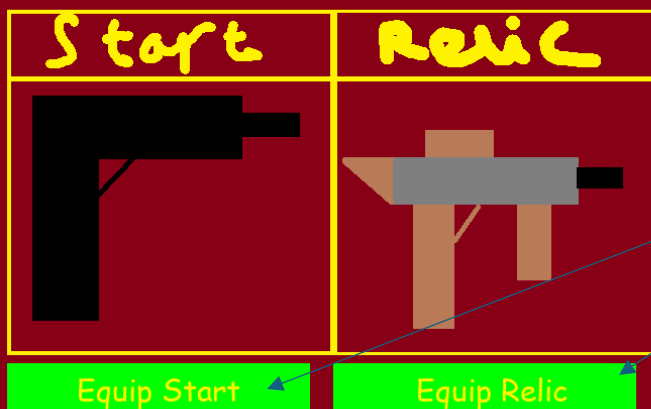
def equip_weapon(weapon_name):
    global player_weapon
    player_weapon = weapon_name

```

Allows the user to equip other weapons

Shop

Buy anything you desire!



Buttons allow the user to equip the different weapons

Due to the fact that there are now 2 different weapons, I had to change the code in the play file slightly so that it could detect which weapon is being equipped.

```
start = CL.StartWeapon(start_img, player, bullet_img)
relic = CL.RelicWeapon(relic_img, player, bullet_img)

if main_menu.player_weapon == "Start":
    current_weapon = start
elif main_menu.player_weapon == "Relic":
    current_weapon = relic
```

Calls the class for both weapons

The equipped weapon becomes the current weapon

To do this, I had to import the main menu file. However, I initially did this so the import statement was at the start. The main menu file also imported from this play file with a statement at the start. This ended up creating a circular import, which meant that both files were relying on each other, so the code couldn't work. I initially placed one of the import statements in the function where it was needed and left the import statement in the main menu how it is, but the problem still persisted.

```
AttributeError: partially initialized module 'play' has no attribute 'start_game' (most likely due to a circular import)
```

This was the error that constantly showed up. The only way to fix this was to put both import statements in the functions where they were actually needed. By doing this, the error was gone and the code was almost working as intended.

After fixing this error, I noticed that there were 2 problems. The first one would be when I attempted to load the level, the program loaded up the shop menu and I would have to go back to the main menu and try a second time. This is when the level would actually load up. However, when the level loaded up, there was another problem. This time, whenever I pressed shoot, instead of shooting one bullet, the program would shoot infinite bullets until I either paused it or ended the level, so I will need to figure out how to fix this.

28/01/2025 – Fixing the new bugs

I wanted to start by fixing the problem where the shop would show up instead of the level. Initially, I placed print statements that would print when a button was pressed to check to see if the buttons were producing the correct outputs. However, when I did this, it was clear that the buttons were doing the right thing and perhaps the program might've unintentionally done this due to the shop being called, perhaps due to one of the import statements. Because, as mentioned above, I cannot move the import statements, I instead created statements to keep track of which screens were being shown. This did involve me having to turn the main menu into a function as well as creating a separate function to constantly track which screen would show.


```
# Define game states
MAIN_MENU = "main_menu"
PLAY_MENU = "play_menu"
LEVEL_SELECTION = "level_selection"
PLAYING = "playing"
SHOP = "shop"
SETTINGS = "settings"

# Initialize the current state
current_state = MAIN_MENU
```

These statements would define each of the game states and mention what the current game state would be when the game is loaded.

```
def main_loop():
    global current_state
    run = True
    while run:
        if current_state == MAIN_MENU:
            current_state = main_menu()
        elif current_state == PLAY_MENU:
            current_state = pl()
        elif current_state == LEVEL_SELECTION:
            current_state = lvl()
        elif current_state == PLAYING:
            play.start_game()
            current_state = MAIN_MENU
        elif current_state == SHOP:
            current_state = shop()
        elif current_state == SETTINGS:
            print("Settings menu")
            current_state = MAIN_MENU

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                run = False

    pygame.quit()
    quit()

if __name__ == "__main__":
    main_loop()
```

Calls the specific subprogram depending on the current state

This statement ensures that the main_loop() subprogram only runs when the program is run directly, so it helps to remove any unintended execution of any other subprograms.

Here's a recording which shows how this works:



NoMore_incorrectS
hopShowings.mp4

As shown, the shop officially no longer shows and the screens now work as intended. Now, I needed to fix the glitch where the player could just shoot infinitely with one click of the shoot button.

To fix this, I've added a flag which is False when the player hasn't shot and becomes True when the player has shot. In addition to this, I've also added another line that makes it so that when the player doesn't press the shoot button, the flag becomes false.

```

elif event.type == pygame.MOUSEBUTTONDOWN:
    if event.button == 1 and not is_shooting:
        bullet = current_weapon.shoot(current_frame, bullet_img)
    if bullet:
        bullets.append(bullet)
    is_shooting = True
elif event.type == pygame.MOUSEBUTTONUP:
    if event.button == 1:
        is_shooting = False

```

Only shoots when
is_shooting is false

If the user doesn't
press the shoot
button for a bit, then
they can shoot again

Here's a recording which shows how it now works:



shoot now truly
works.mp4

As shown, the shoot now works as intended and these unexpected bugs have gone.

29/01/2025 – Another Big Problem

To check if the shop truly works, I changed the value for picking up coins from 1 to 100, so it would be much easier to gain the coins needed to buy the Relic weapon. After gaining the coins, I changed the value of the coins back to 1, and I brought the weapon. The shop worked fine, and this did take place yesterday (I just forgot to mention this). I tried it again however, and now all of a sudden, while the program is taking the coins, it's not actually equipping the weapon, which is very problematic. I thought the problem might've been the player_weapon statement outside the shop, which turned out to be true as when I changed it from "Start" to "Relic", the relic weapon would load up and the start weapon couldn't load up. There were also other problems (these were found earlier), such as:

- Relic could've been brought multiple times
- If you brought Relic and switched back to Start, you would need to buy the Relic again
- Whenever you closed and reopened the program, you would no longer own the Relic and you would have to purchase it again

I suspect that these are due to the fact that the purchase may not be permanently stored in the database. However, before fixing these, I had to fix the current problem at hand.

I initially tried to fix this by adding main_menu.player_weapon statements where the user would buy these weapons so that the global player_weapon variable would update, although this didn't work. I'm currently doing more research into this to find a solution. I

first ran some print statements in certain areas to gain a better understanding of how the program is processing the change in the weapons. After doing this, I can safely conclude that my initial suspicion is incorrect and the actual problem at hand is the play program resetting the weapon back to the Start weapon and the shop actually updates the weapon correctly.

After doing some research, I figured out how to fix this. I created a new table for the database which would store the currently equipped weapon. I also created 2 new functions that would set the weapon to be used and get the weapon that was set.

```
def set_weapon(weapon_name):
    cur.execute("DELETE FROM player_data") # Clear previous selection
    cur.execute("INSERT INTO player_data (weapon) VALUES (?)", (weapon_name,))
    con.commit()

def get_weapon():
    cur.execute("SELECT weapon FROM player_data")
    result = cur.fetchone()
    return result[0] if result else "Start" # Default to "Start" if no value is stored

def equip_weapon(weapon_name):
    global player_weapon
    player_weapon = weapon_name
    database.set_weapon(weapon_name)
    print (f"weapon equipped: {player_weapon}")

import main_menu
main_menu.player_weapon = database.get_weapon()
current_weapon = relic if main_menu.player_weapon == "Relic" else start
```

Inserts the selected weapon into the database

Gathers the current weapon from the table

Takes the equipped weapon and places it in the set_weapon function

Calls the get_weapon function to load the correct weapon

Here's a recording showing this corrected version:



Different Weapons
can now be equipped

There's another recording that I made immediately after. Here it is below:



Program remembers
last weapon when c

As this video shows, not only was the big problem of not being able to switch weapons fixed, but the fix also fixed the problem where the program would reset the weapon back to

the Start weapon when it closed. However, despite fixing this problem, the other problems that were initially encountered are still present. Just a reminder that these issues are:

- Relic can be brought multiple times
- If you brought Relic and switched back to Start, you would need to buy the Relic again.

Once these issues are ironed out, we can move on to the settings.

30/01/2025 – Fixing the remaining shop bugs

Now, I wanted to fix the problems that were mentioned above. To do this, I created a new table to store the weapons which the player owned and I also created 2 functions, one would be used to purchase the relic and the other would be used to check if the user has purchased the relic.

```
def purchase_weapon(weapon_name):  
    #stores purchased weapon into database  
    cur.execute("INSERT OR IGNORE INTO owned_weapons (weapon) VALUES (?)", (weapon_name,))  
  
def has_weapon(weapon_name):  
    #checks if weapon is already purchased  
    cur.execute("SELECT weapon FROM owned_weapons WHERE weapon = ?", (weapon_name,))  
    return cur.fetchone() is not None # Returns True if the weapon is owned, False otherwise
```

Adds weapon
into the table
if it's not
already there

Checks if the table contains
the inputted weapon

```
elif relic equip_rect.collidepoint(mouse_pos):  
    if database.has_weapon("Relic"):  
        equip_weapon("Relic")  
    elif purchase_relic():  
        database.purchase_weapon("Relic")  
        equip_weapon("Relic")
```

If Relic is in the database, it equips it
when the user presses the button
without using any coins

If Relic is not in the database and the user
has the coins, the game will buy Relic using
the player's coins and equip it automatically

```
def purchase_relic():  
    if database.has_weapon("Relic"):#Checks if Relic is already purchased  
        print ("Relic is already equipped!")  
        return True #Allows user to equip without using coins  
  
    current_coins = database.get_coin_total()  
    if current_coins >= 150:  
        database.insert_coins(-150)  
        database.purchase_weapon("Relic")  
        return True  
    else:  
        return False
```

Stores Relic into the database so the
game remembers that the Relic
weapon has been brought

```

if main_menu.player_weapon == "Relic" and database.has_weapon("Relic"):
    current_weapon = relic
else:
    current_weapon = start
    main_menu.player_weapon = "Start" # Ensure the default is Start if relic isn't owned

```

Equips relic only when the user has equipped it and if the database knows it's been brought

Here's a recording showing how the program now works:



Shop fully works now.mp4

As shown, the shop now works perfectly as the user can now switch weapons without any issues and they cannot unexpectedly lose money when they want to switch weapons when they already own both of them.

05/02/2025 – Creating weapon sounds

To create the settings menu, I will need to make something that can may not be necessary but can be toggled on or off by the user. In this case, I want to add sound effects for the weapons which can be toggled on or off in the settings so the user can experience the game with and without these sounds. In addition, sound effects are also something that Ben wanted to see in the game, so it's necessary to add them where possible.

To create the sound effects, I used an application called 'BandLab', which is a piece of software dedicated to making music. In my case, I used the different drums they had amongst other instruments to make these sound effects. When downloading them, they showed up as a .m4a sound file, which isn't supported by pygame as it only seems to support either .mp3 or .wav sound files. Therefore, I used an online converter to convert these files into .mp3 files. Before creating the settings to allow the user to toggle on/off this feature, I had to get these files implemented into the game. I saved them in their own folder labelled 'sounds', which thinking about it now could be a great idea to clean up the folder which stores all of the programs and assets. To add these sound effects, I had to use a method called pygame.mixer.Sound, which allows the program to retrieve the sounds. After this, I slightly adjusted the code where the program registers the user's left mouse click as a shot, and adjusted it so that the weapon sound effect is made as the weapon is shot depending on what the weapon is.

```

elif event.type == pygame.MOUSEBUTTONDOWN:
    if event.button == 1 and not is_shooting:
        bullet = current_weapon.shoot(current_frame, bullet_img)
        if bullet:
            bullets.append(bullet)
            if current_weapon == start:
                start_weapon_sound.play()
            elif current_weapon == relic:
                relic_weapon_sound.play()
    is_shooting = True

```

Uses a certain sound effect depending on the weapon

However, when I initially ran it, whilst the sounds were being produced, the ending screens were no longer working. The problem was with when I was defining the buttons, as they were actually defined after they would've been used. What is quite interesting is that this error had never existed from when I initially made these ending screens and have only shown now when the sound effects have been added. To fix this, I moved where I defined these buttons to before they were used, which fixed this error.

```

def end_screen(score, text_drawer, screen, clock):
    end = True
    font = pygame.font.SysFont("Comic Sans MS", 50)
    small_font = pygame.font.SysFont("Comic Sans MS", 30)

    replay_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2, 300, 50)
    return_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2 + 100, 300, 50)

    while end:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                quit()
            elif event.type == pygame.MOUSEBUTTONDOWN:
                mouse_pos = pygame.mouse.get_pos()
                if replay_button_rect.collidepoint(mouse_pos):
                    return 'replay'
                elif return_button_rect.collidepoint(mouse_pos):
                    return 'menu'

```

The buttons are now being called here

The buttons are being used here



Gun SFX.mp4

As the video shows, the sound effects work quite well in the game. I might come back to add more in the future, but for now I need to make the settings menu and allow the user to turn these sounds off if they want to.

08/02/2025 – Creating the settings menu to toggle the sounds on or off

To attempt to do this, I created a separate file to manage the sounds. This is a python file that would save the settings that have been set.

```
import json

SETTINGS_FILE = "settings.json"

# Default settings
default_settings = {
    "sound": True,
    "weapon_sounds": {
        "Start": "sounds/startShot.mp3",
        "Relic": "sounds/relicShot.mp3"
    }
}

# Load settings
def load_settings():
    try:
        with open(SETTINGS_FILE, "r") as file:
            return json.load(file)
    except (FileNotFoundError, json.JSONDecodeError):
        # Return default settings if the file is missing or corrupted
        save_settings(default_settings)
        return default_settings

# Save settings
def save_settings(settings):
    with open(SETTINGS_FILE, "w") as file:
        json.dump(settings, file, indent=4)

settings = load_settings() # Load settings at the start
```

The settings are saved in a .json file, which is used to store and exchange data, such as the settings that are set by the user. This file can be opened in Notepad to view the data.

This is the default settings for the sounds that will be stored

Subprograms to load/save the settings – if the .json file isn't recognised, then the default settings will be returned when load_settings() is called

```
app_settings = settings.load_settings()
# Load settings from JSON file
def load_settings():
    try:
        with open("settings.json", "r") as file:
            return json.load(file) # Load settings
    except FileNotFoundError:
        return {"sound": True, "weapon_sounds": {}} # Default settings if file is missing

app_settings = load_settings()
pygame.mixer.init() # Initialize sound system

# Load weapon sounds if sound is enabled
weapon_sounds = {}
if app_settings["sound"]:
    for weapon, sound_file in app_settings["weapon_sounds"].items():
        weapon_sounds[weapon] = pygame.mixer.Sound(sound_file) # Load sound

print("Settings loaded:", app_settings)
```

Gives the function to load the settings a new name to not confuse it with the subprogram for the settings menu

Loads the settings

```

def settings_menu():
    settings_open = True
    font = pygame.font.SysFont("Comic Sans MS", 40)
    toggle_sound_button = pygame.Rect(750, 400, 350, 50)

    while settings_open:
        screen.fill((0, 162, 232))

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                quit()

            elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
                mouse_pos = pygame.mouse.get_pos()
                if toggle_sound_button.collidepoint(mouse_pos):
                    app_settings["sound"] = not app_settings["sound"] # Toggle sound
                    settings.save_settings(app_settings)
                    print(f"Sound Toggled: {app_settings['sound']}") # Debugging output
                elif event.type == pygame.MOUSEBUTTONDOWN:
                    mouse_pos = pygame.mouse.get_pos()
                    if return_button_rect.collidepoint(mouse_pos):
                        return False

        # Draw the button
        pygame.draw.rect(screen, (255, 0, 0), toggle_sound_button)
        sound_status = "ON" if app_settings["sound"] else "OFF"
        text_drawer.draw_centered_text(f"Weapon Sounds: {sound_status}", font, (255, 255, 255), SCREEN_WIDTH, SCREEN_HEIGHT + 50)

        return_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2 + 100, 300, 50)

        pygame.draw.rect(screen, (255, 0, 0), return_button_rect)
        text_drawer.draw_centered_text("Return", font_2, (255, 242, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 250)

        pygame.display.update()
        clock.tick(60)

def toggle_sound():
    app_settings["sound"] = not app_settings["sound"] # Toggle True/False

    # Save the new setting
    with open("settings.json", "w") as file:
        json.dump(app_settings, file, indent=4)

    print(f"Sound {'enabled' if settings['sound'] else 'disabled'}")

```

Changes the setting
when contact is made
with the button

Saves the
change in
the .json file

There are a few problems with this code. One of them was that the return button didn't work, but that was easily fixed by moving the code for the detection before the code where the program detects if the toggle_sound_button has been pressed. I also got rid of the line "elif event.type == ... and event.button == 1:" and I instead added an elif where the button is being detected. Here's a video that shows another problem:



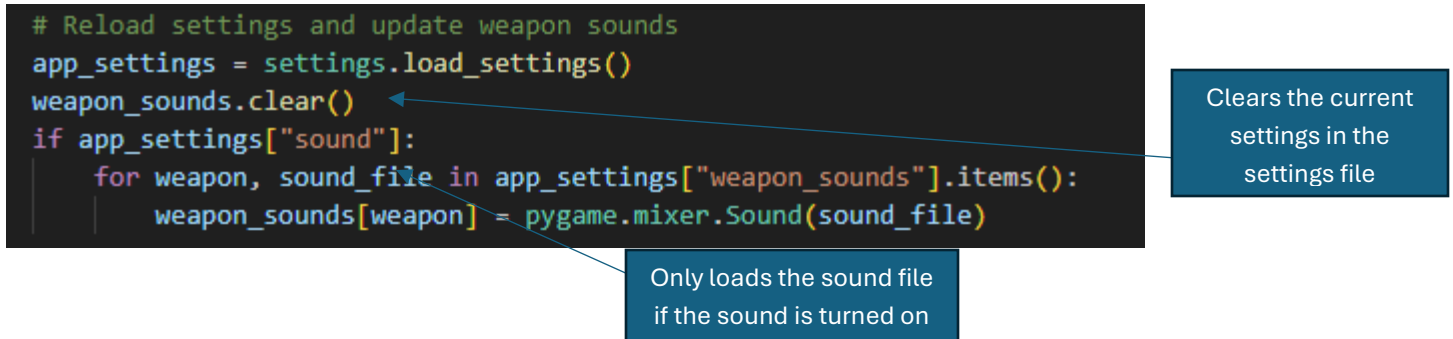
Toggle not
working.mp4

Luckily, the fix doesn't seem to be complicated. It's just that the subclasses don't seem to have the attribute name, so I can add that and the problem should be fixed.

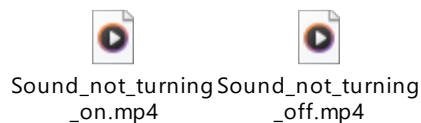
After fixing this issue, another one popped up where current_frame wasn't properly defined in the shoot_weapon function within the play.py file. I fixed this by making it so current_frame is passed into the shoot_weapon function(), which fixed this error. While there are no error messages anymore, it turns out that the toggle wasn't working this entire time at all as it doesn't disable the sounds when the user tells it to disable sounds, even though the message to do so shows.

14/02/2025 – Fixing the toggle

To try to fix this, I added an extra bit of code to the `settings_menu()` which should update the sound options when the user changes them accordingly. The screenshot below shows how this works. I switched to Visual Studio Code as it was becoming difficult to track all of my different files and it allows me to access all of my files in one place.



However, when I ran this, the toggle was only partially working. This is because whether the sound was on or off depended on what the setting was when the game started up. If the game sounds were on, they'll always be on and vice versa. This means that if the user does change the settings, they'll need to restart the game to see the changes, which is very annoying. These videos show me testing the feature



As these videos show, whether the sound works or not depends on the state of the toggle. The first thing I thought to do was to add debug prints to see what is wrong with the code. After doing this, it seems that the problem right now is that the `main_menu` file recognises the change with the settings, however the change isn't being reflected in the play file. This means that I'll have to find where in the play file this change isn't being reflected and why this is the case.

To ensure that this was the case, I ran the code again, but this time I changed the setting before I played the level, and by some chance, it actually worked as intended. However, after I played the first time, when I switched the setting again and then played the level again, the same issues are showing, so it's evident that to some extent it actually does work, but it stops working after running the game for the first time, so I'm going to have to find out why this is the case.

After lots of time experimenting with the code, I think I figured out what the problem was. The problem was actually linked to my initial problem, except that it wasn't being updated in the main game loop itself. To fix this problem, I added the following to fix this:

```
run = True
while run:
    app_settings = settings.load_settings()
    if main_menu.app_settings["sound"] == False:
        app_settings["sound"] = False
```

Sets the sound depending on what the file has saved

Sets the sound to False if the main menu equivalent is False

When I ran this with the setting on, it worked as intended as shown below:



Working with on
start.mp4

However, this is how it ran when the setting was off:



Not working with
off start.mp4

I'm not really sure why this is the case as it should work like how the other video showed it working to some extent.

Once again, after playing around with the code for a bit longer, I found the problem. The problem was the fact that I put False instead of True in the last 2 lines. I figured this out by randomly changing them and then running the code and I was surprised that the problem was just that I put the incorrect Boolean in the code. I wanted to test it again just to be sure, and all of a sudden the same problem showed up where it wasn't switching. I genuinely do not have a clue regarding how this can possibly be fixed, as when I implemented the fix it was working for a bit, but it then randomly stops working.

I've managed to find a fix that properly works now. This fix involves calling the sound files later in this part of the code. I also got rid of the `app_settings["sound"] = True/False` line as this line might've been overriding the default settings, which would explain why the code only worked in certain situations.

```

run = True
while run:
    app_settings = settings.load_settings()
    if app_settings["sound"]:
        start_weapon_sound = pygame.mixer.Sound("sounds/startShot.mp3")
        relic_weapon_sound = pygame.mixer.Sound("sounds/relicShot.mp3")

```

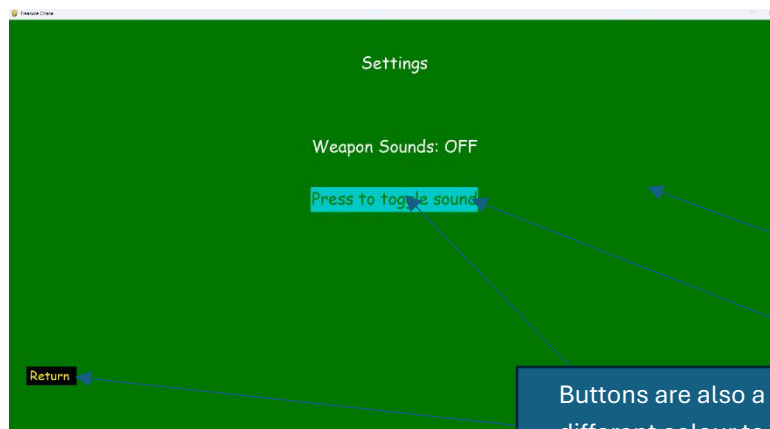
Only loads the sounds when the sounds are toggled on

Here's a video which shows how the code works:



Toggle now working.mp4

As shown, this feature now works as intended. It's going to be difficult to add more features to the project, although there is still a bit that I wanted to add. Now, I'm going to remove any print statements that I had for debugging and if there are any glaring bugs remaining when I have another look I might try to fix them if possible. I am also going to change the look of the settings menu so it looks a bit better. After doing a bit of experimenting, I think this is a decent way to present the settings menu:



Different theme to differentiate from other menus

Button is now annotated so users can clearly see what it does.

Buttons are also a different colour to help avoid confusion

I think that this colour combination looks quite good and as mentioned earlier, it could help differentiate the look of this menu compared to other menus.

16/02/2025 – Bug with the lives

I ran the game again and noticed that whenever I fall off the map and die, the lives system works as intended. However, when I get killed by the enemy, the game removes 2 lives instead of one, which shouldn't be happening. This was happening because it turns out that I was calling the die() method twice within the player class, specifically in the update() method and the check_collision() method, which probably caused the game to remove 2

lives instead of the one it should've removed initially. To fix this, I slightly adjusted the code so the die() method in the check_collision() method is removed, and instead I replaced it with the following:

```
for enemy in enemies:
    if self.rect.colliderect(enemy.rect):
        self.lives -= 1
        self.respawn()
    return True
```

Removes a life and respawns the player when they are hit by an enemy

Because both update() and check_collision() are running at the same time, this caused die() to run multiple times when the player was hit by the enemy. By writing the code like this, the game will only call die() when the player falls off the map now, so the code is now working as intended and the player only loses one life when they die as they should.

With this bug fixed, that will be the end of this development section due to time limitations, so I cannot do anymore now and I will evaluate what has been done.

Evaluation

With everything that I could possibly program in the limited timeframe complete, I will now evaluate how this project went. I will perform one final set of testing – Post Development Testing – and show the stakeholder what I have done and receive feedback from him. I will also compare what I did to the success criteria outlined earlier to determine just how much of the success criteria I have met.

Post Development Testing

Now that I've added as much to the program as possible within the limited time that I had, now it's time to conduct the post development testing. These tests will involve the post development tests mentioned in the 'Test Data – Post Development Testing'. I will include video evidence showing these tests being conducted where possible.

Part A – Controls

These tests focus on the controls and how the user can interact with the program. Because I couldn't complete the project fully due to time constraints, some tests may not be possible to conduct.

Test Number	Test	Action	Expected Result	Actual Result	Video Evidence
A1	Player Movement	VALID – A is pressed INVALID – J is pressed	VALID – Pressing A moves the player to the left INVALID – Pressing J will do nothing	VALID – Pressing A does move the player to the left INVALID – Pressing J does nothing	 Gameplay.mp4
A2	Player Movement	VALID – D is pressed INVALID – V is pressed	VALID – Pressing D moves the player to the right INVALID – Pressing V does nothing	VALID – Pressing D does move the player to the right INVALID – Pressing V does nothing	 Gameplay.mp4
A3	Player Movement	VALID – Spacebar is pressed INVALID – W is pressed	VALID – Pressing spacebar causes the user to jump INVALID – Pressing W does nothing	VALID – Pressing spacebar does cause the user to jump INVALID – Pressing W does nothing	 Gameplay.mp4

A4	Attacking	VALID – Left Mouse Click is pressed INVALID – Z is pressed	VALID – Pressing the Left Mouse Click causes the player to shoot INVALID – Pressing Z does nothing	VALID – Pressing the Left Mouse Click causes the player to shoot INVALID – Pressing Z does nothing	 Gameplay.mp4
A5	Pausing	VALID – P is pressed INVALID – Esc is pressed	VALID – Pressing P pauses/unpauses the game INVALID – Pressing Esc does nothing	VALID – Pressing P pauses/unpauses the game INVALID – Pressing Esc does nothing	 Gameplay.mp4
A6	Using Powerups	VALID – G is pressed INVALID – M is pressed	VALID – Pressing G activates a powerup INVALID – Pressing M does nothing	Powerups were not added due to time constraints.	N/A
A7	Menu Navigation	VALID – Left Mouse Click is pressed when hovering over a button INVALID – W is pressed	VALID – Pressing the Left Mouse Click allows user to access other parts of the menu when selecting buttons INVALID – Pressing W does nothing	VALID – Pressing the Left Mouse Click allows user to access other parts of the menu when selecting buttons INVALID – Pressing W does nothing	 Menu Navigation.mp4
A8	Menu Navigation	VALID – Esc is pressed when not on the main menu INVALID – Esc is pressed when on the main menu	VALID – Pressing Esc when not on the main menu allows the user to return back to the main menu INVALID – Pressing Esc when in the main menu does nothing as there is nothing to return to	This was not added due to time constraints	N/A

Part B – Collisions

These tests help to find out if the collisions are working as intended or if there are still any problems regarding a few of the collisions.

Test Number	Test	Action	Expected Result	Actual Result	Video Evidence
B1	Platforms	The player will move across the different platforms	The player should be able to walk on all platforms as intended	The player can walk on all platforms	 Gameplay.mp4
B2	Movement	The player will attempt to go through an enemy	The player should be unable to go through any enemy	The player cannot go through any enemy	 Gameplay.mp4
B3	Player Attacks	The player will shoot at an enemy	The shot should hit the enemy and deal damage to it	The shot hits and damages the enemy	 Gameplay.mp4
B4	Enemy Attacks	The enemy will shoot at the player	The shot should hit the player and deal damage to them	Enemy shooting was not added due to time constraints	N/A
B5	Coins	The player will move over a coin	The player should pick up a coin, it should add 1 to their total coin balance, and the coin should disappear.	The player picks up the coin, it adds one to the balance, and the coin disappears	 Coins.mp4

Part C – Game States

These tests make sure that when certain buttons are pressed, they load up the correct screens.

Test Number	Test	Action	Expected Result	Actual Result	Video Evidence
C1	Level Selection	VALID – Clicking an unlocked level INVALID – Pressing U in the level selector	VALID – This should load the level that has been selected by the user INVALID – This will do nothing as U is an invalid input in the menus	VALID – This loads the level that has been selected by the user INVALID – This will do nothing as U is an invalid input in the menus	 Level Selection.mp4
C2	Pausing	VALID – Pressing P while the game is playing INVALID – Pressing O whilst the game is playing	VALID – The game should become paused INVALID – Nothing will happen	VALID – The game is paused INVALID – Nothing will happen	 Gameplay.mp4
C3	Unpausing	VALID – Pressing P while the game is paused INVALID – Pressing A while the game is paused	VALID – Pressing P while the game is paused will resume the game INVALID – Nothing will happen	VALID – Pressing P while the game is paused will resume the game INVALID – Nothing will happen	 Gameplay.mp4
C4	Transition to shop menu	VALID – The “Shop” option is pressed INVALID – W is pressed	VALID – The game loads up the shop menu	VALID – The game loads up the shop menu	 Shop Transition.mp4

			INVALID – Nothing happens as W is not a valid input	INVALID – Nothing happens as W is not a valid input	
C5	Transition to settings	VALID – The “Settings” option is pressed INVALID – The “Play” option is pressed	VALID – The Settings menu loads up INVALID – The Play menu loads up	VALID – The Settings menu loads up INVALID – The Play menu loads up	 Settings Transition.mp4
C6	“Level Failed” Screen	VALID – Fail the level INVALID – Press P while the game is running	VALID – The “Level Failed” Screen should show INVALID – The game should pause	VALID – The “Level Failed” Screen show INVALID – The game should pause	 Failure1.mp4  Failure2.mp4
C7	Closing the game	VALID – Pressing the X button at the top right of the screen INVALID – Pressing the “Play” option	VALID – Closes the game INVALID – Loads the play menu	VALID – Closes the game INVALID – Loads the play menu	 Closing.mp4

Part D – Scoring

These tests make sure that the scoring system works as intended. There are some scoring methods that haven’t been added yet. The reasoning is due to time constraints meaning that some features are still not in the game.

Test Number	Test	Action	Expected Result	Actual Result	Video Evidence
D1	The score will be null at the start of each level	The specific level will be loaded	The score will start at 0	The score starts at 0	 Gameplay.mp4

D2	Coins	A coin will be picked up	The score will increase by 50	The score increases by 50	 Gameplay.mp4
D3	Enemies	An enemy will be killed	The score will increase by 100	The score increases by 100	 Gameplay.mp4
D4	Dying	The player will die	The player's score will be reduced by 30	The player's score is reduced by 30	 Dying.mp4
D5	Powerups	The player will use a powerup	The player's score will be reduced by 10 for each powerup that is used	Powerups were not added due to time constraints.	N/A
D6	Timer	VALID – The timer shows that there is less than 3 minutes remaining INVALID – The timer shows that there is more than 3 minutes remaining	VALID – The player's score will be reduced by 30 INVALID – The player's score will not be impacted	The timer's ability to influence scoring was not added due to time constraints	N/A
D7	Scoreboard Updating	VALID – The player will achieve a higher score than their current highest score on a specific level VALID – The player achieves a score that's lower than	VALID – The scoreboard will update to show the new score as the best score and the previous high score will drop down a place as will any lower scores.	VALID – The scoreboard will update to show the new score as the best score and the previous high score will drop down a place as will any lower scores. VALID – The high score will	 Leaderboard.mp4

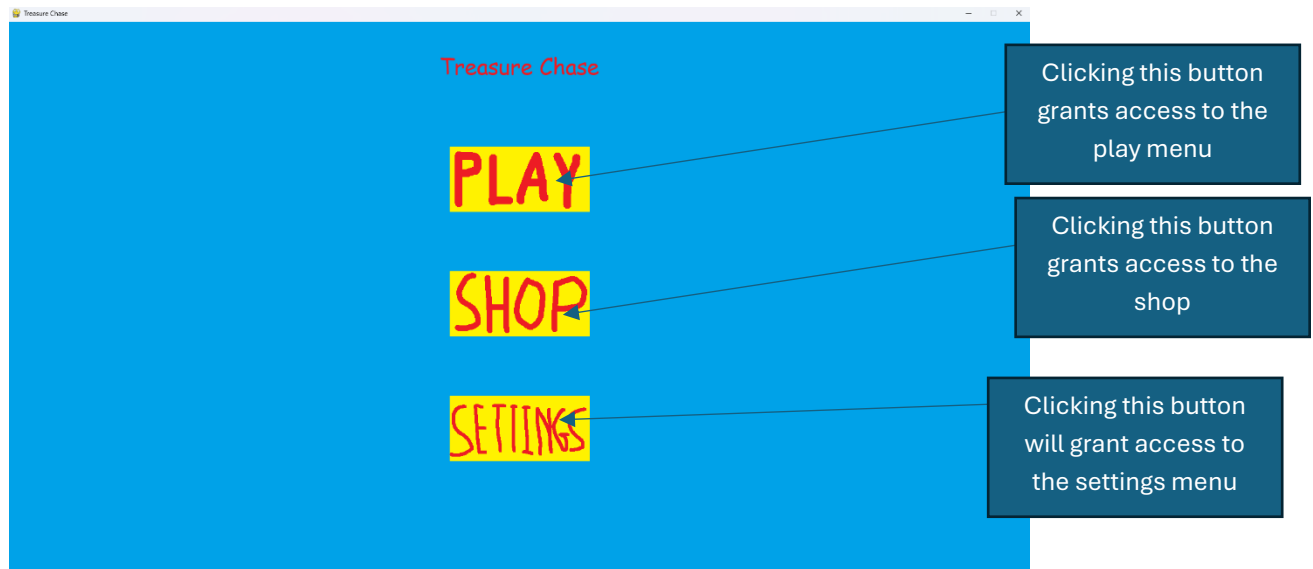
		their high score on a specific level	VALID – The high score will remain where it is. If the new score is in the top 5 personal scores earned, then it will be placed accordingly in the top 5. Otherwise, it won't be saved.	remain where it is. If the new score is in the top 5 personal scores earned, then it will be placed accordingly in the top 5. Otherwise, it won't be saved.	
--	--	--------------------------------------	--	---	--

Going through the usability features with the stakeholder

After conducting these tests, I met up with Ben again to discuss the features that have been implemented into the game and get his thoughts on what had been added. He can also suggest any features that he would want to add.

Menus

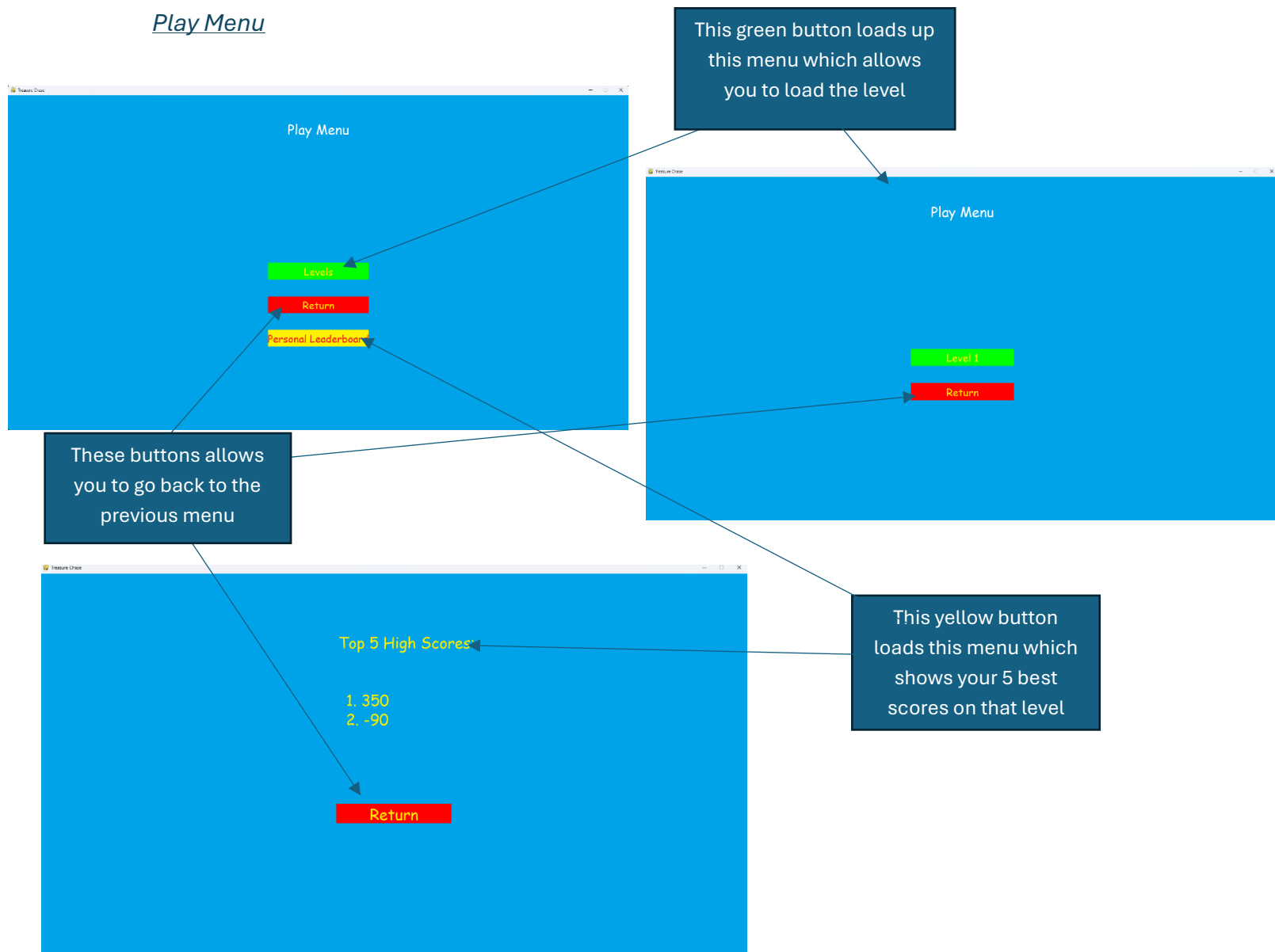
Main Menu



Me: This is the main menu of the game. You can use the mouse to navigate all of the menus, which I personally believed was much better to implement compared to using keys. In addition, I felt that it was necessary to attempt to keep the theme of this menu consistent with the theme within the main gameplay. I also thought that it could be a nice idea to try to draw my own buttons using Microsoft Paint where necessary.

Ben: I really like that you can use the mouse to interact with the menus. I feel that this is very convenient for anyone playing the game. I also like the character of the buttons and the way that they look. I think that they suit this kind of game. The only potential problem I have with this design is that there are unused areas around the buttons and title screens – if there was some decorations there it would look nicer, but this isn't a big concern and overall I like the way it has come out.

Play Menu

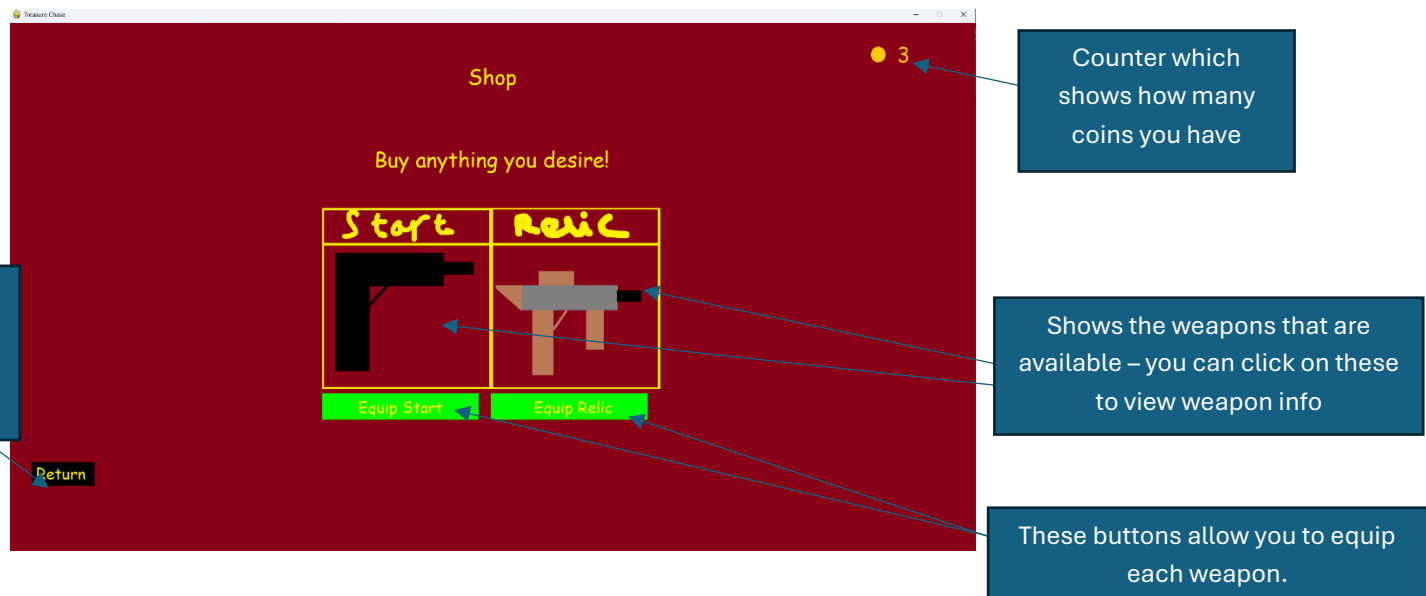


Me: This is the menu that loads when the play button is pressed on the main menu. Due to time constraints, I wasn't able to make a tutorial level for players who may be unfamiliar with the game. You can use the red button labelled 'Return' to go back to the main menu. The 'Levels' option loads up the level selection menu where you can choose the level you want to play. Currently, there is only one level ultimately due to time constraints. The yellow button loads up a menu which shows the 5 best scores you have achieved from the level.

Ben: Once again, I quite like what you have done here. I like how simple, yet effective, these menus are and I like how I can keep track of which scores are the highest achieving scores. My main problem is the lack of a tutorial option, which can help players who are

unfamiliar with this type of game, although you have mentioned why this wasn't possible. Another thing I would like to see is some sort of way to show the controls. There is currently nothing in the game to help the user understand the controls, so to make it more beginner friendly this could also be implemented.

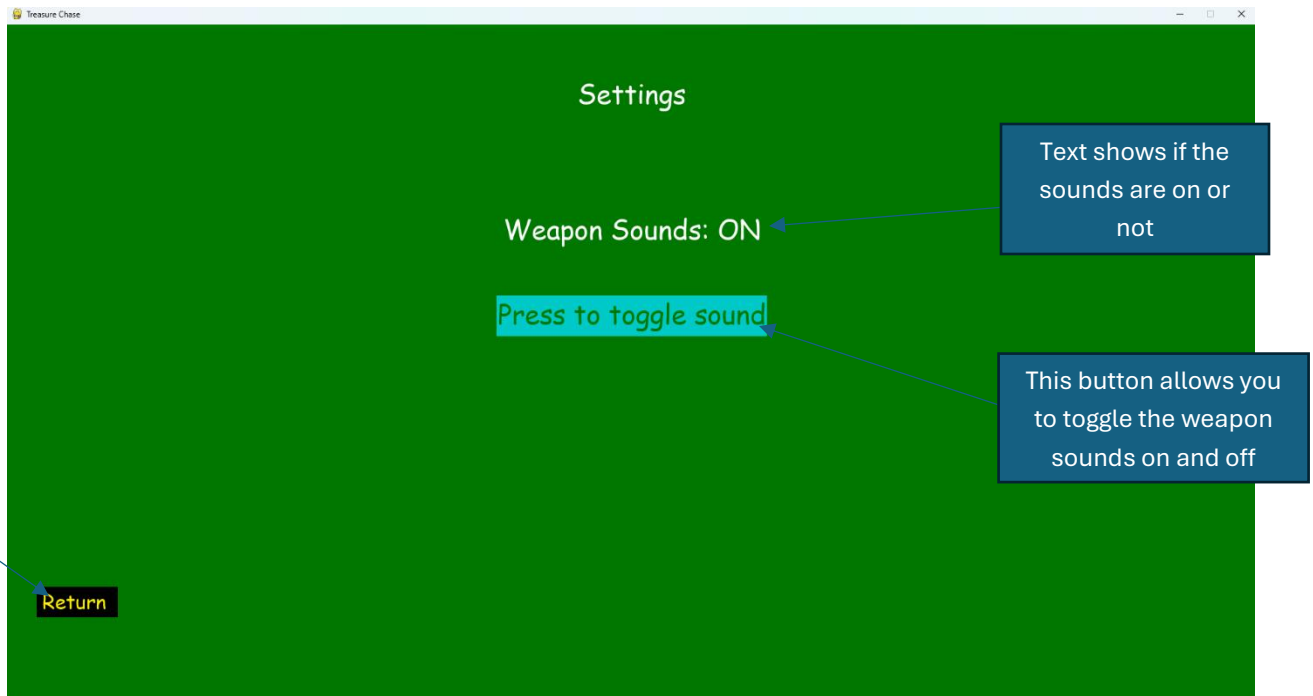
Shop Menu



Me: This is the menu that loads when you press the shop button within the main menu. You can see your total coin balance and equip either of these weapons. The 'Start' weapon is free and is automatically equipped in your inventory. The relic weapon costs 150 coins to buy before you can equip it. Once equipped, you can use it in the game.

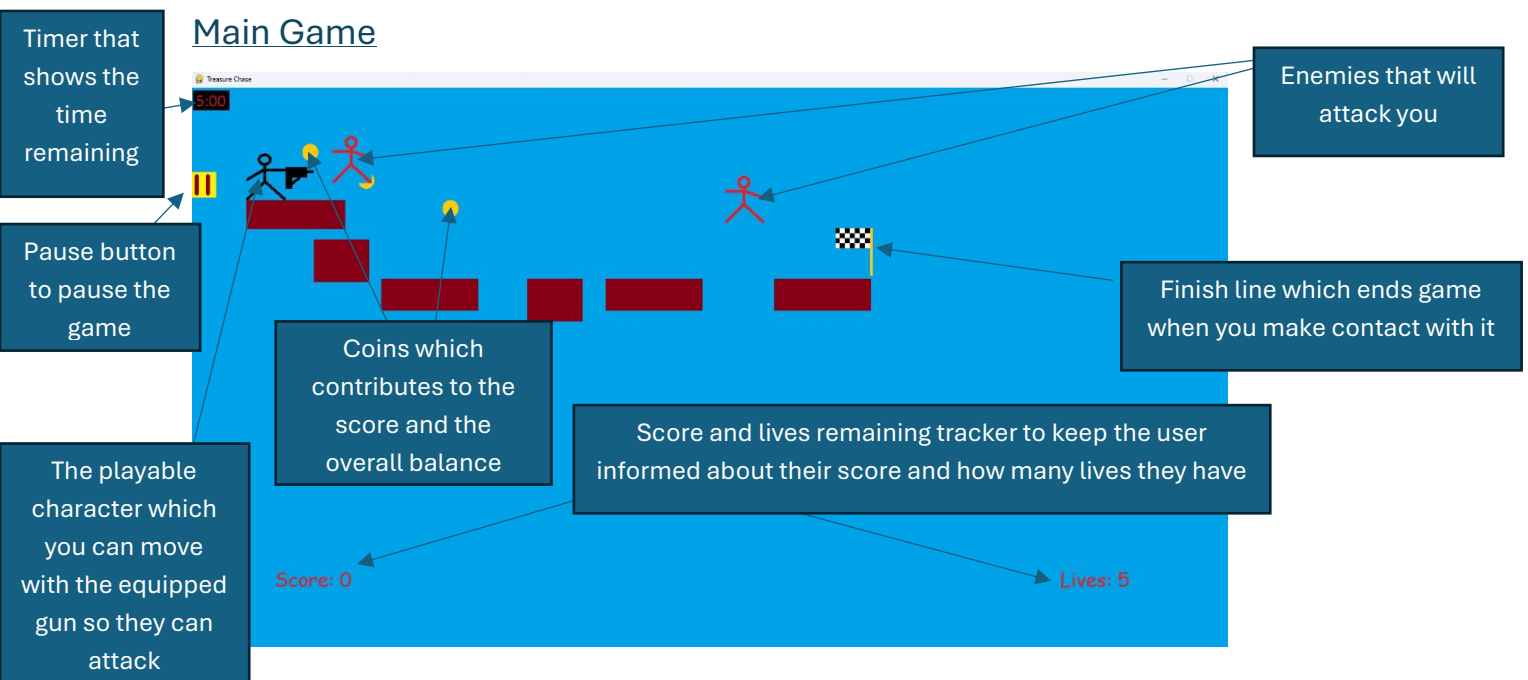
Ben: I like how the background has changed to represent that we are in a different aspect of the game. I also like how you can see what the model of each weapon looks like. I feel like that there should be a separate button that should allow you to buy the relic button only, or a button like that should be there instead of a button that shows that the weapon can be equipped. There should also be a way of some sort to view the price of the Relic until you have brought the weapon. There could also be a message at the bottom showing which weapon has been equipped.

Settings Menu



Me: This is the settings menu, where you can adjust the available settings to your liking. Currently, there is only an option to toggle the weapon sound effects. If the button is clicked, the status of the weapon sounds is changed and that change will be reflected in the text above and in game, where if the sound is off, you won't be able to hear the weapon sounds and if it's on, you'll be able to hear the weapon sounds.

Ben: Similarly to the shop, I like how the background and general theme changes when you go onto different settings. It really helps differentiate each of the options and it doesn't constantly use the same colour theme over and over which is a good thing. I also like this ability to toggle the sound as some people may like the sounds, whereas others may choose to leave it out. One area for improvement could be the fact that the sound is the only setting. Perhaps if there was more time, you could add a slider to change the volume or any other settings such as settings for the controls so people could adjust the settings to their liking.



Me: This is what the playable level looks like. Again, due to time limitations, there is ultimately less than what might've been anticipated. For instance, there is a very small number of coins compared to the 50-100 that you wanted. As you requested, there is a fully working 5 minute timer in the top left corner of the screen which allows you to view how much time you have left. There are 2 counters at the bottom of the screen, one of which tracks your score and the other tracks how many lives you have remaining. There is also a pause button which can be used to pause the game, although you can use P to pause the game. In addition, due to time limitations, I wasn't able to add any powerups of any sort into the game.

Ben: This gameplay looks quite good. I like how there is an option to pause the game, which can be accessed using different methods. I also really like the way the timer looks here – the black and red colour theme almost creates an ominous feeling when you look at it, and in my opinion, it's a great way to pressurise the player to complete the level as fast as possible. I also like how you can see the score and the amount of lives that are remaining – this will be really useful for the player as it shows them critical information that they need to see. I wish that there were more enemies, more coins and more platforms to increase the size of this level, although I understand that there were problems with timings so adding everything was not possible.

Controls

Me: The controls weren't too complicated. This is because I just needed the left mouse click to shoot the weapon and navigate through the menus and a few keys to move the player around. I felt that the use of the mouse needed no explanation, although the use of the keys in hindsight might've needed some explaining, but issues with time have meant that this wasn't possible.

Ben: The controls that were chosen are suitable for this game. There could be an argument to make jump W instead of using the spacebar, but it ultimately makes sense to use either. As you mentioned, if there were no time constraints, it would've been ideal to see a menu of some sort which shows the controls.

Comparing the Solution to the Success Criteria

Requirement	Justification	Has the requirement been met?
At least 5 levels	Having 5 levels is a reasonable number of levels to start with and I feel that this will be sufficient enough.	No
Start Menu has 3 different options	As shown in the draft in the Proposed Solution – there should be 3 different sections of the Start Screen – Play, which allows the user to either start a new game or continue their progress, Shop, where they can buy items, and Settings, where the player can adjust the settings based on their preferences.	Yes
Ability to change difficulty in settings	Can give players an easier/more difficult experience based on what they choose.	No
Having at least 3 different weapons to use	Allows the player to improve their attacks by buying weapons using coins that they earn throughout the levels.	No
Scoring System	There will be a scoring system that will determine the score of the player's run of the level. This will consider how many coins they picked up, how long they took, and if they used power-ups. There will be a points multiplier that is influenced depending on the difficulty, with harder difficulties having bigger multipliers.	Partially
Timer	Ben wanted a timer to be added that should last around 5 minutes, although I may increase the time for harder levels and difficulties.	Yes
Controls	To make the user experience as comfortable as possible, I will use A to move left, D to move right, spacebar to jump, the mouse's left click to shoot, G to activate powerups, P to pause the game, I to interact with objects, and R to reload.	Partially
Sound effects	Sounds that play as actions such as jumping and shooting. Sound output is dependent on the type of input.	Partially
Pause Game	Allows anybody to pause the game should they need to do so. When the game is paused, a menu should show up that gives them the	Yes

	options to return to the game, change some settings, and return to the start menu.	
Show the score	This would allow the player to see how well they're doing according to the scoring system.	Yes
Player starts with 5 lives on each level	Players will start each level with 5 lives, and that will decrease as the player dies.	Yes
Each level has around 50-100 coins that can be found	This is a feature that Ben wanted to see in the game, and I believe that 50-100 coins is a good amount to reward during each level.	No
Reward players with coins and more lives at end of levels	The number of coins and lives will depend on the player scores, with higher scores rewarding more coins and lives, and lower scores rewarding fewer coins and lives.	No
Game ends when all lives are lost	Helps to increase difficulty for the user in the game	Yes
Final Boss Level	On the final level, the user should face a final boss that should also be quite challenging to complete.	No
Levels 2-5 will be locked from the start	The game is meant to be based on a story, so players should be prevented from accessing the later levels early on, and they should progress through the story as I intend them to.	No
All levels can be replayed	If a player wants to play through a level they already completed, then they should have the option to do.	Yes
All levels are displayed in their own menu	This can display a separate area where the levels can be viewed and interacted with.	Yes
Health Bar is shown	Because the player has health, they need to be aware of how much health they have left before they die.	No
Number of lives also shown	This will help the player become aware of how many lives they have remaining.	Yes
Some enemies have ranged attacks whereas others have melee attacks	This helps to diversify the enemies that the player will come up against and will also increase difficulty as the player has to adopt different strategies against different enemies.	No
Player loses life if they fall off a platform	If a player misses a platform and ends up dying, they will respawn, but they'll also lose a life.	Yes
Shop with different weapons and armour	Allows the player to purchase better weapons and armour using the coins they earned to become much more powerful and to aid them in challenging content.	Partially

Harder Difficulties should give better rewards	High Risk – High Reward: It's important that players who play on higher difficulties are rewarded more for playing the harder difficulties.	No
--	---	----

How the criteria was met in some areas

- Start Menu has 3 different options – When you load up the game, the menu shows up and has 3 different buttons which the user can interact with – ‘Play’, which loads up the play menu, ‘Shop’, which loads up the shop, and ‘Settings’, which loads up the settings menu.
- Timer – When the user loads up a level, in the top left of their screen there is a timer counting down for 5 minutes and when the time is up, the user fails the level.
- Pause Game – When the user presses the pause button or presses ‘P’ on their keyboard, a pause menu shows up and the game is fully paused. The user can resume by pressing ‘P’ on their keyboard again.
- Show the Score – On the bottom left, there is some text which shows the score the user is currently on and it updates the score depending on what actions the player does.
- Player starts with 5 lives on each level – When the player loads up the level, they can see on the bottom right how many lives they have remaining. Whenever they start the level, they have 5 lives and they only fail when they lose all of their lives.
- Game ends when all lives are lost – When the player runs out of lives, they fail the level and an ending screen shows up which lets them know that they failed that level.
- Levels can be replayed – When the user ends a level, whether they won, ran out of time, or lost lives, they can constantly replay the level over and over again. In addition, in the menus, you can always access the levels that you’ve previously completed.
- Levels are displayed in their own menu – When you play the game, you go through the play menu and then click the ‘Levels’ option so you can access the levels. This helps to separate things out into their own areas so they can easily tell the difference between certain areas.
- Number of lives shown – At the bottom right of the screen, there is some text which shows how many lives the user has remaining and constantly updates when the user loses a life. This means that the user will always know how many lives they have during the game

- Player loses life if they fall off a platform – When the user falls off a platform, they die and respawn on the starting platform, losing a life when doing so and removing 30 from their score.

How the criteria was partially met in some areas and how can this be addressed

- Scoring System – This partially meets the requirements as the scoring system currently does track enemies killed, the deaths occurred, and the coins the player picks up. However, this system doesn't track for time, check for power-ups used, or add a multiplier when the difficulty increases. This is ultimately due to time limitations which has meant that some features, such as power-ups, haven't been added, which means that some features are missing. This can be addressed by adding the features necessary to meet the requirements in the future.
- Controls – This partially meets the requirements as the existing controls follow the requirements. However, like with the scoring system, some extra features that were needed to allow this to meet the requirements were not implemented, so the requirement is only partially met.
- Sound Effects – This is partially met as the sound effects for shooting were implemented. However, other sound effects, such as jumping, weren't added to the game and this can be fixed by adding these sound effects in a future update. Jumping has already been added, so this requirement shouldn't need much to be fully complete.
- Shop with different weapons and armour – This partially meets the requirements as the shop in the game contains 2 different weapons, one of which can be purchased. However, there is no armour in the game currently, so this requirement can be fully met by adding more options for weapons and adding functioning armour into the game that gives the player extra survivability when playing the game.

Why the criteria wasn't met in some areas and how can this be addressed

- At least 5 levels – I only ended up making one level. This was ultimately due to time constraints which meant that I didn't have enough time to complete this requirement. In the future, this can be met by creating all of the levels within the game.
- Ability to change difficulty in Settings – The current settings menu has no capabilities in regards to changing difficulty. This is one again due to time constraints, so I didn't have enough time to add this feature. This can be met in the future by adding an option in the settings to change the difficulty, which would

change the enemies by increasing their movement speed and by making them stronger by forcing the user to need multiple shots to kill them.

- Having at least 3 different weapons – Even though there was more than 1 weapon, there were only 2, so this requirement wasn't met. This can be met by adding a 3rd more powerful weapon into the shop for the user to collect.
- Each level has around 50-100 coins – This requirement wasn't met because the one level that was made was ultimately too small to have 50-100 coins within it. To meet this in the future, I could change the way the levels work by perhaps adding a scrolling background to the gameplay, or perhaps adding more platforms for the user so the level is a bit longer and the user could gain more coins.
- Reward players with coins and more lives at end of levels – Even though the user can earn coins during the levels, the purpose of this requirement was to grant the user extra coins and more lives when the level ends. This wasn't done ultimately due to time constraints, which, as shown earlier, caused me to not include some of these features.
- Final Boss Level – Because I only did the first level, it wasn't possible for me to add a final boss level, ultimately due to the same reasons as to why there was only 1 level in the end. By completing the future levels, I could meet this requirement by making the final level 5 the boss level.
- Levels 2-5 will be locked from the start – Once again, due to the fact that levels 2-5 simply don't exist, this requirement hasn't been met. To meet this, if I do make these missing levels in the future, I could add a check that checks if the user has completed the previous level – if they have, the game lets them access the level, otherwise they cannot access it and the game blocks them from doing so.
- Health Bar is shown – Due to time constraints, health wasn't added into the game, therefore adding a health bar wasn't possible due to the current situation. I would need to add the health feature for the player in a future update, and only then could I add the health bar to the UI and meet this requirement.
- Some enemies have ranged attacks whereas others have melee attacks – Whilst it could be considered that the current attack is a melee, ultimately if the player makes any sort of contact with an enemy, regardless of where it is, so it cannot necessarily be considered a melee. This can be added in the future by creating the different enemy types and changing how the collisions work so that bumping into an enemy won't result in a death depending on where the player makes contact with the enemy.
- Harder Difficulties should give better rewards – This requirement cannot be met because the feature to alter difficulty hasn't been added yet. Therefore, I would

need to add the ability to change settings in the difficulty, add end of level rewards, and then adjust the rewards accordingly to complete this requirement.

Maintenance

Currently, there isn't any actual way to conduct any sort of maintenance for this game. However, if this game was to ever get distributed in the future, it would need to be constantly maintained by looking out for and fixing bugs or releasing updates to add new features.

If this project is complete in the future, I could add a feature which allows the user to check if there is an update of the game available to download. In addition, I could also add another option called "Submit Feedback", which would allow the user to give feedback for the game, which I would receive and act upon. I could probably release this on app stores, such as the Microsoft Store, or on a digital marketplace, such as Steam. This would allow me to easily roll out updates as well as allow me to easily view feedback based on the reviewing systems for both of these applications.

Currently, I would say that my program can definitely be maintained. This is because there are comments within the program that could help other programmers understand my code, so they can understand different aspects of my code easily. In addition, the code is split into multiple files that perform different tasks, so finding certain code should be a bit easier compared to trying to find it in one big file. In addition, the use of classes and subprograms will definitely help to improve the modularity of the program so most features are in their own section.

However, looking back at my code, there are definitely a few problems that could negatively effect maintenance. The first problem is the naming of some of my files, in particular the "settings.py" and "CL.py" files. With the "CL.py" file, the main problem is that this is meant to be an abbreviation of the word "Classes". However, if another programmer interacted with this and saw this file, they may be completely confused regarding what this file even does. In addition, the main problem with the "settings.py" file is that it has a very similar name to the "settings_menu" subprogram in "main_menu.py". Initially, I gave the function the name "settings", which meant that I had to add an extra line of code that gave the settings file a new name. I kept this in the code, even after changing the name of the function due to the fact that they had the same name. Ultimately, it can still be confusing when someone who isn't familiar with my code views it, but it's much better than pygame itself getting confused.

Bugs that currently exist

There are some bugs within the game that currently exist which can affect the way the game works and may cause unexpected actions to take place. The table below shows specific bugs, what they do, any evidence showing how it works, and any potential solutions to these bugs so the game can run as intended.

Bug	Evidence	How could I fix this?
Player can go onto platforms when they shouldn't be able to – When the player moves between platforms, they can still move left and right. If they make any contact with a platform, they instantly go on top of it, which shouldn't be possible.	 Bug 1.mp4	I could fix this by adding a check that prevents the user from being able to get back onto the platform when they've fallen beside a platform. However, as the video shows, this isn't the easiest bug to replicate, so this might not even be a priority for me to fix this. I could also change the way collisions work, as they currently involve rectangular collisions and perhaps changing the way collisions work could limit these types of interactions.
Enemy collisions – When you make any contact with the enemy, even if it involves making contact with the head, as the video shows, you always die. This shouldn't be how the game works but ultimately is due to the way the enemies have been programmed, which involves the player dying and losing a life when the player makes direct contact with the enemy in any possible way.	 Bug 2.mp4	I could fix this by making all of the different types of enemies, and then removing the parts of the code that causes the player to die whenever they make any contact with the enemy and making it so that they only die whenever they make contact with the weapon that the enemy is holding (if it's a sword) or if they get hit by a bullet from a gun that the enemy is using.

How limitations could be managed

This following section refers back to the limitations from the Analysis part of this write-up.

Requirement	Justification
Spawn Protection	This will allow the user to be protected for a little while, so they can have a period of time where the enemies cannot kill them.

To complete this requirement, I could create a flag called “protection”, which would become true when the user respawns, and will become false after a few seconds. I could

create a subprogram that takes the exact time when the user spawns in, and will automatically make “protection” false when it realises that it’s been around 5 seconds.

Requirement	Justification
Two-Player Mode	This can allow 2 friends to take on the game simultaneously so they could complete the game much easier.

To complete this requirement, I could develop a separate game mode for this requirement. This would involve 2 players working together to complete the same level together, where the rewards for each individual player would be dependent on how well the individual player performs. The levels for this mode would be much bigger than the normal levels, with more enemies to account for the increased number of players that will be playing that level.

Requirement	Justification
Scrolling Background	This can allow levels to be much bigger, so the user can take more time on a level whilst being careful as to what is outside of the screen

To complete this requirement, I would do some research on how to add a scrolling background in pygame. I would then have to adjust my code to accommodate for a scrolling background that would move depending on where the user goes.

Requirement	Justification
Option to choose sides in Two-Player Mode	This can allow 2 friends to play different roles within the game by controlling certain characters within the game, one could control the main character whilst another could control an enemy to try to disrupt their friend’s progress.

To complete this requirement, I would first have to make a Two-Player mode as mentioned earlier. After creating it, I would have to add another mode within the 2 player mode where one player can control an enemy and another player can control the player. This would allow both players to attack and compete against each other.

Requirement	Justification
Large-scale levels	This will allow me to add more enemies and coins into certain levels and could potentially add extra difficulty to the game depending on how big it gets.

To complete this requirement, I would have to increase the scale of the levels. This can be done by implementing the scrolling background mentioned above. This would allow me to add more to the level outside of the screen that the player would interact with as they play the game. It could also improve the player experience as they would be able to explore

bigger levels instead of playing the smaller levels, which could quickly become stale due to how much smaller and easier they probably are.

Requirement	Justification
More than 5 levels	This can allow the story to go on further – I can experiment with it and add more storyline to the game, and the user could get immersed into a bigger story with more lore.

To complete this requirement, I would first need to make 5 playable levels first. After that, I could expand the number of playable levels to around 10-20 different levels, perhaps set in different worlds to help set a bit of a story which the user can enjoy. This would allow players to enjoy the game a little bit more.

There are some extra limitations that weren't mentioned earlier that I've found now that can impact the game and the maintenance of the game:

Limitation	Why is it a limitation?	How could I work around this?
Compatibility	Currently, I'm not sure if this game can actually run on other operating systems, such as MacOS. I haven't tested my code so it's likely that the current solution can't run on any other operating system besides Windows.	I could work around this by making separate versions for different operating systems so this game could be accessed from a variety of different devices.
Optimisation	Currently, for what it is, the game takes much longer than it should to load up. This could potentially be due to how I've programmed this as it involves multiple files working together constantly and it also involves loading images from folders, so this could've potentially caused the code to be unoptimized in some areas, causing the slow start-up.	I could work around this by looking back at my code and removing any unnecessary lines of code that have little to no effect on the program. This would mean that there are fewer lines to compile, so it could slightly increase the speed of the startup.
Ease of installation	Currently, there is an executable file which loads up all of the programs. It might not be the most convenient as the assets also need to be with it in order to load the game, so it's not convenient for an average user if they downloaded this app.	I could try to use an installer which could compile all of the necessary files together to make installation easier.

Final Code

Main_Menu.py

```
1  import pygame
2  import CL
3  import database
4  import sqlite3
5  import settings
6  import json
7
8  pygame.init()
9  #initilises pygame
10 SCREEN_WIDTH = 1850#sets the screen width in pixels
11 SCREEN_HEIGHT = 1000#sets the screen height in pixels
12 clock = pygame.time.Clock()
13
14 coin_rect = pygame.Rect(1645,40,85,45)
15
16 screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))#creates a 1850 by 1000 screen
17 pygame.display.set_caption("Treasure Chase")#gives the window the name "Treasure Chase"
18
19 player_weapon = "Start"
20
21 app_settings = settings.load_settings()
22
23 pygame.mixer.init() # Initialize sound system
24
25 # Load weapon sounds if sound is enabled
26 weapon_sounds = {}
27 if app_settings["sound"]:
28     for weapon, sound_file in app_settings["weapon_sounds"].items():
29         weapon_sounds[weapon] = pygame.mixer.Sound(sound_file) # Load sound file
30
31 # Define game states
32 MAIN_MENU = "main_menu"
33 PLAY_MENU = "play_menu"
34 LEVEL_SELECTION = "level_selection"
35 SHOP = "shop"
36 SETTINGS = "settings"
37
38 # Initialize the current state
39 current_state = MAIN_MENU
40
41 #loads all images that will be used in the program
42 play_img = pygame.image.load('play_button.png').convert_alpha()
43 shop_img = pygame.image.load('shop_button.png').convert_alpha()
44 settings_img = pygame.image.load('settings_button.png').convert_alpha()
45 leader_img = pygame.image.load('P.leaderboard_button.png').convert_alpha()
46 shop_coin_img = pygame.image.load('menu_coin.png').convert_alpha()
47 #Initilises the Button class
48 play_button = CL.Button(799, 225, play_img)
49 shop_button = CL.Button(799, 450, shop_img)
50 settings_button = CL.Button(799, 675, settings_img)
```

```

51 #sets a font that I may want to use - will be used with the text class
52 font = pygame.font.SysFont("Comic Sans MS",40)
53 font_2 = pygame.font.SysFont("Comic Sans MS", 30)
54 #Initilises the Text class
55 text_drawer = CL.Text(screen)
56
57 con = sqlite3.connect("treasureChase_data.db")
58 cur = con.cursor()
59 cur.execute(["CREATE TABLE IF NOT EXISTS high_scores(score INT)"])
60
61 def insert_score(score):
62     cur.execute("INSERT INTO high_scores (score) VALUES (?)", (score,))
63     con.commit()
64
65 def get_top_5_scores():
66     cur.execute("SELECT score FROM high_scores ORDER BY score DESC LIMIT 5")
67     scores = cur.fetchall()
68     return scores
69
70 def display_top_5_scores(screen, text_drawer):
71     leader = True
72     font_2 = pygame.font.SysFont("Comic Sans MS", 40)
73     while leader:
74         for event in pygame.event.get():
75             if event.type == pygame.QUIT:
76                 pygame.quit()
77                 quit()
78             elif event.type == pygame.MOUSEBUTTONDOWN:
79                 mouse_pos = pygame.mouse.get_pos()
80                 if return_button_rect.collidepoint(mouse_pos):
81                     return False
82         screen.fill((0, 162, 232))
83         top_scores = get_top_5_scores()
84         y_offset = 300
85         text_drawer.draw_text("Top 5 High Scores:", font, (255,242,0), 780, 150)
86         for rank, score in enumerate(top_scores, start=1):
87             text_drawer.draw_text(f"{rank}. {score[0]}", font, (255,242,0), 800, y_offset)
88             y_offset += 50
89
90         return_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2 + 100, 300, 50)
91
92         pygame.draw.rect(screen, (255, 0, 0), return_button_rect)
93         text_drawer.draw_centered_text("Return", font_2, (255, 242, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 250)
94
95         pygame.display.update()
96         clock.tick(60)
97

```

```

98 def display_coin_total(text_drawer):
99     total_coins = database.get_coin_total()
100     coin_text = f"{total_coins}"
101     text_drawer.draw_text(coin_text, font, (255, 201, 14), 1700, 30)
102
103 def pl():
104     pl = True
105     font = pygame.font.SysFont("Comic Sans MS", 40)
106     small_font = pygame.font.SysFont("Comic Sans MS", 30)
107     while pl:
108         for event in pygame.event.get():
109             if event.type == pygame.QUIT:
110                 pygame.quit()
111                 quit()
112             elif event.type == pygame.MOUSEBUTTONDOWN:
113                 mouse_pos = pygame.mouse.get_pos()
114                 if level_button_rect.collidepoint(mouse_pos):
115                     lvl()
116                 elif return_button_rect.collidepoint(mouse_pos):
117                     return False
118                 elif leader_button_rect.collidepoint(mouse_pos):
119                     display_top_5_scores(screen, text_drawer)
120
121         screen.fill((0, 162, 232))
122         text = font.render("Play Menu", True,(255,255,255))
123         screen.blit(text,(SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 - text.get_height() // 2 - 400 ))
124
125         level_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2, 300, 50)
126         return_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2 + 100, 300, 50)
127         leader_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2 + 200, 300, 50)
128
129         pygame.draw.rect(screen, (0, 255, 0), level_button_rect)
130         text_drawer.draw_centered_text("Levels", small_font, (255, 242, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 50)
131
132         pygame.draw.rect(screen, (255, 0, 0), return_button_rect)
133         text_drawer.draw_centered_text("Return", small_font, (255, 242, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 250)
134
135         pygame.draw.rect(screen, (255, 242, 0), leader_button_rect)
136         text_drawer.draw_centered_text("Personal Leaderboard", small_font, (237, 28, 36), SCREEN_WIDTH, SCREEN_HEIGHT + 450)
137
138         pygame.display.update()
139         clock.tick(60)
140

```

```

141 def lvl():
142     import play
143     lvl = True
144     font = pygame.font.SysFont("Comic Sans MS", 40)
145     small_font = pygame.font.SysFont("Comic Sans MS", 30)
146     while lvl:
147         for event in pygame.event.get():
148             if event.type == pygame.QUIT:
149                 pygame.quit()
150                 quit()
151             elif event.type == pygame.MOUSEBUTTONDOWN:
152                 mouse_pos = pygame.mouse.get_pos()
153                 if level1_button_rect.collidepoint(mouse_pos):
154                     play.start_game()
155                     return True
156                 elif return_button_rect.collidepoint(mouse_pos):
157                     return False
158             screen.fill((0, 162, 232))
159             text = font.render("Play Menu", True, (255, 255, 255))
160             screen.blit(text, (SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 - text.get_height() // 2 - 400))
161
162             level1_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2, 300, 50)
163             return_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2 + 100, 300, 50)
164
165             pygame.draw.rect(screen, (0, 255, 0), level1_button_rect)
166             text_drawer.draw_centered_text("Level 1", small_font, (255, 242, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 50)
167
168             pygame.draw.rect(screen, (255, 0, 0), return_button_rect)
169             text_drawer.draw_centered_text("Return", small_font, (255, 242, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 250)
170
171             pygame.display.update()
172             clock.tick(60)
173
174 def shop():
175     global player_weapon
176     shop = True
177     player_weapon = "Start"
178     start_showcase_img = pygame.image.load('start_showcase.png').convert_alpha()
179     start_show = CL.Button(600, 350, start_showcase_img)
180     relic_showcase_img = pygame.image.load('relic_showcase.png').convert_alpha()
181     relic_show = CL.Button(923, 350, relic_showcase_img)
182     while shop:

```

```

174 def shop():
183     for event in pygame.event.get():
184         if event.type == pygame.QUIT:
185             pygame.quit()
186             quit()
187         elif event.type == pygame.MOUSEBUTTONDOWN:
188             mouse_pos = pygame.mouse.get_pos()
189             if start_show.rect.collidepoint(mouse_pos):
190                 start()
191             elif start_equip_rect.collidepoint(mouse_pos):
192                 equip_weapon("Start")
193             elif relic_show.rect.collidepoint(mouse_pos):
194                 relic()
195             elif relic_equip_rect.collidepoint(mouse_pos):
196                 if database.has_weapon("Relic"):
197                     equip_weapon("Relic")
198                 elif purchase_relic():
199                     database.purchase_weapon("Relic")
200                     equip_weapon("Relic")
201             elif return_button_rect.collidepoint(mouse_pos):
202                 return False
203
204             screen.fill((136, 0, 21))
205             start_show.draw()
206             relic_show.draw()
207             pygame.draw.rect(screen, (136, 0, 21), coin_rect)
208             text = font.render("Shop", True, (255, 242, 0))
209             screen.blit(text, (SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 - text.get_height() // 2 - 400))
210             text = font.render("Buy anything you desire!", True, (255, 242, 0))
211             screen.blit(text, (SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 + text.get_height() // 2 - 300))
212
213             coins = [CL.Coin(shop_coin_img, 1650, 45)]
214             for coin in coins:
215                 coin.draw_coin()
216             display_coin_total(text_drawer)
217
218             return_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 880, SCREEN_HEIGHT // 2 + 330, 120, 45)
219             start_equip_rect = pygame.Rect(600, 700, 300, 50)
220             relic_equip_rect = pygame.Rect(923, 700, 300, 50)
221
222             pygame.draw.rect(screen, (0, 0, 0), return_button_rect)
223             text_drawer.draw_centered_text("Return", font_2, (255, 242, 0), 200, 1700)
224
225             pygame.draw.rect(screen, (0, 255, 0), start_equip_rect)
226             text_drawer.draw_centered_text("Equip Start", font_2, (255, 242, 0), 1500, 1450)
227
228             pygame.draw.rect(screen, (0, 255, 0), relic_equip_rect)
229             text_drawer.draw_centered_text("Equip Relic", font_2, (255, 242, 0), 2160, 1450)
230
231             pygame.display.update()
232             clock.tick(60)
233

```

```

233 def start():
234     start = True
235     while start:
236         for event in pygame.event.get():
237             if event.type == pygame.QUIT:
238                 pygame.quit()
239                 quit()
240             elif event.type == pygame.MOUSEBUTTONDOWN:
241                 mouse_pos = pygame.mouse.get_pos()
242                 if return_button_rect.collidepoint(mouse_pos):
243                     return False
244             screen.fill((0,0,0))
245             text = font.render("Start", True, (255,255,255))
246             screen.blit(text,(SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 - text.get_height() // 2 - 400 ))
247             text = font.render("Weapon Information: ",True,(255,255,255))
248             screen.blit(text,(SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 + text.get_height() // 2 - 300 ))
249             text = font.render("The basic starter weapon which you get",True,(255,255,255))
250             screen.blit(text,(SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 + text.get_height() // 2 - 200 ))
251             text = font.render("Special Abilities: N/A",True,(255,255,255))
252             screen.blit(text,(SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 + text.get_height() // 2 - 100 ))
253
254             return_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2 + 100, 300, 50)
255
256             pygame.draw.rect(screen, (255, 0, 0), return_button_rect)
257             text_drawer.draw_centered_text("Return", font_2, (255, 242, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 250)
258
259             pygame.display.update()
260             clock.tick(60)
261
262 def relic():
263     relic = True
264     while relic:
265         for event in pygame.event.get():
266             if event.type == pygame.QUIT:
267                 pygame.quit()
268                 quit()
269             elif event.type == pygame.MOUSEBUTTONDOWN:
270                 mouse_pos = pygame.mouse.get_pos()
271                 if return_button_rect.collidepoint(mouse_pos):
272                     return False
273             screen.fill((0,0,0))
274             text = font.render("Relic", True, (255,255,255))
275             screen.blit(text,(SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 - text.get_height() // 2 - 400 ))
276             text = font.render("Weapon Information: ",True,(255,255,255))
277             screen.blit(text,(SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 + text.get_height() // 2 - 300 ))
278             text = font.render("A vintage weapon with a special shine",True,(255,255,255))
279             screen.blit(text,(SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 + text.get_height() // 2 - 200 ))
280             text = font.render("Special Abilities: Can pierce through enemies and gain multikills",True,(255,255,255))
281             screen.blit(text,(SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 + text.get_height() // 2 - 100 ))
282

```

```

262 def relic():
283     return_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2 + 100, 300, 50)
284
285     pygame.draw.rect(screen, (255, 0, 0), return_button_rect)
286     text_drawer.draw_centered_text("Return", font_2, (255, 242, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 250)
287
288     pygame.display.update()
289     clock.tick(60)
290
291 def purchase_relic():
292     if database.has_weapon("Relic"):#Checks if Relic is already purchased
293         return True #Allows user to equip without using coins
294
295     current_coins = database.get_coin_total()
296     if current_coins >= 150:
297         database.insert_coins(-150)
298         database.purchase_weapon("Relic")
299         return True
300     else:
301         return False
302
303 def equip_weapon(weapon_name):
304     global player_weapon
305     player_weapon = weapon_name
306     database.set_weapon(weapon_name)
307
308 def settings_menu():
309     global app_settings
310     settings_open = True
311     font = pygame.font.SysFont("Comic Sans MS", 40)
312     toggle_sound_button = pygame.Rect(725, 400, 400, 60)
313
314     while settings_open:
315         screen.fill((0, 120, 0))
316
317         for event in pygame.event.get():
318             if event.type == pygame.QUIT:
319                 pygame.quit()
320                 quit()
321             elif event.type == pygame.MOUSEBUTTONDOWN:
322                 mouse_pos = pygame.mouse.get_pos()
323                 if return_button_rect.collidepoint(mouse_pos):
324                     return False
325                 elif toggle_sound_button.collidepoint(mouse_pos):
326                     app_settings["sound"] = not app_settings["sound"] # Toggle sound
327                     settings.save_settings(app_settings)
328
329                     # Reload settings and update weapon sounds
330                     app_settings = settings.load_settings()
331                     weapon_sounds.clear()

```

```

main_menu.py > ...
308 def settings_menu():
309     if app_settings["sound"]:
310         for weapon, sound_file in app_settings["weapon_sounds"].items():
311             weapon_sounds[weapon] = pygame.mixer.Sound(sound_file)
312
313     # Draw the button
314     pygame.draw.rect(screen, (0, 201, 200), toggle_sound_button)
315     sound_status = "ON" if app_settings["sound"] else "OFF"
316     text_drawer.draw_centered_text(f"Weapon Sounds: {sound_status}", font, (255, 255, 255), SCREEN_WIDTH, SCREEN_HEIGHT - 400)
317     text_drawer.draw_centered_text("Press to toggle sound", font, (0, 120, 0), SCREEN_WIDTH, SCREEN_HEIGHT - 150)
318
319     text = font.render("Settings", True, (255, 255, 255))
320     screen.blit(text, (SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 - text.get_height() // 2 - 400))
321
322     return_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 880, SCREEN_HEIGHT // 2 + 330, 120, 45)
323
324     pygame.draw.rect(screen, (0, 0, 0), return_button_rect)
325     text_drawer.draw_centered_text("Return", font_2, (255, 242, 0), 200, 1700)
326
327     pygame.display.update()
328     clock.tick(60)
329
330 def toggle_sound():
331     app_settings["sound"] = not app_settings["sound"] # Toggle True/False
332
333     # Save the new setting
334     with open("settings.json", "w") as file:
335         json.dump(app_settings, file, indent=4)
336
337 def main_menu():
338     main = True
339     while main:
340         screen.fill((0, 162, 232))
341
342         play_button.draw()
343         shop_button.draw()
344         settings_button.draw()
345
346         text_drawer.draw_text("Treasure Chase", font, (237, 28, 36), 780, 50)
347
348         for event in pygame.event.get():
349             if event.type == pygame.QUIT:
350                 main = False
351             elif event.type == pygame.MOUSEBUTTONDOWN:
352                 mouse_pos = pygame.mouse.get_pos()
353                 if play_button.rect.collidepoint(mouse_pos):
354                     pl()
355                 elif shop_button.rect.collidepoint(mouse_pos):
356                     shop()
357

```

```

362 def main_menu():
363     shop()
364     elif settings_button.rect.collidepoint(mouse_pos):
365         settings_menu()
366
367     pygame.display.update()
368     clock.tick(60)
369
370     pygame.quit()
371     quit()
372
373 def main_loop():
374     global current_state
375     run = True
376     while run:
377         if current_state == MAIN_MENU:
378             current_state = main_menu()
379         elif current_state == PLAY_MENU:
380             current_state = pl()
381         elif current_state == LEVEL_SELECTION:
382             current_state = lvl()
383             current_state = MAIN_MENU
384         elif current_state == SHOP:
385             current_state = shop()
386         elif current_state == SETTINGS:
387             current_state = settings_menu()
388             current_state = MAIN_MENU
389
390         for event in pygame.event.get():
391             if event.type == pygame.QUIT:
392                 run = False
393
394     pygame.quit()
395     quit()
396
397 if __name__ == "__main__":
398     main_loop()
399

```


Play.py

```
1 import pygame
2 import CL
3 import database
4 import settings
5 from pygame.locals import *
6 pygame.init()
7
8 pygame.mixer.init()
9
10 FONT_NAME = "Comic Sans MS"
11
12 def shoot_weapon(weapon, bullet_img, current_frame, app_settings):
13     if app_settings["sound"]:
14         sound_file = app_settings["weapon_sounds"].get(weapon.name)
15         pygame.mixer.Sound(sound_file).play()
16     return weapon.shoot(current_frame, bullet_img)
17
18 def start_game():
19     SCREEN_WIDTH = 1850
20     SCREEN_HEIGHT = 1000
21     screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
22     pygame.display.set_caption("Treasure Chase")
23
24     score = 0
25
26     clock = pygame.time.Clock()
27
28     # Timer setup
29     start_ticks = pygame.time.get_ticks() # Get the initial time
30     total_time = 5 * 60 * 1000 # 5 minutes in milliseconds
31
32     timer_rect = pygame.Rect(5, 5, 65, 35)
33
34     font = pygame.font.SysFont(FONT_NAME, 35)
35     font = pygame.font.SysFont("Comic Sans MS", 35)
36     font2 = pygame.font.SysFont("Calibri", 30)
37
38     text_drawer = CL.Text(screen)
39
40 def pause_screen():
41     pause = True
42     font = pygame.font.SysFont("Comic Sans MS", 50)
43     while pause:
44         for event in pygame.event.get():
45             if event.type == pygame.QUIT:
46                 pygame.quit()
47                 quit()
48             elif event.type == pygame.KEYDOWN and event.key == pygame.K_p: # Press 'p' to resume
49                 pause = False
50
51     screen.fill((0, 0, 0))
52     text = font.render("Paused", True, (255, 255, 255))
```

```

17 def start_game():
39     def pause_screen():
52         screen.blit(text, (SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 - text.get_height() // 2 - 400))
53         text = font.render("Press 'P' to resume", True, (255, 255, 255))
54         screen.blit(text, (SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 + text.get_height() // 2 - 100))
55         pygame.display.update()
56         clock.tick(5)
57
58     pygame.event.clear()
59
60 def end_screen(score, text_drawer, screen, clock):
61     end = True
62     while end:
63         for event in pygame.event.get():
64             if event.type == pygame.QUIT:
65                 pygame.quit()
66                 quit()
67             elif event.type == pygame.MOUSEBUTTONDOWN:
68                 mouse_pos = pygame.mouse.get_pos()
69                 if replay_button_rect.collidepoint(mouse_pos):
70                     return 'replay'
71                 elif return_button_rect.collidepoint(mouse_pos):
72                     return 'menu'
73
74         screen.fill((0, 0, 0))
75         text_drawer.draw_centered_text(f"Your Score: {score}", font, (255, 255, 255), SCREEN_WIDTH, SCREEN_HEIGHT - 200)
76         text = font.render("You Won!", True, (255, 255, 255))
77         screen.blit(text, (SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 - text.get_height() // 2 - 400))
78
79         small_font = pygame.font.SysFont("Comic Sans MS", 30)
80
81         replay_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2, 300, 50)
82         return_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2 + 100, 300, 50)
83
84         pygame.draw.rect(screen, (0, 255, 0), replay_button_rect)
85         text_drawer.draw_centered_text("Replay", small_font, (0, 0, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 50)
86
87         pygame.draw.rect(screen, (255, 0, 0), return_button_rect)
88         text_drawer.draw_centered_text("Return to Main Menu", small_font, (0, 0, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 250)
89
90         pygame.display.update()
91         clock.tick(60)
92
93 def time_screen(score, text_drawer, screen, clock):
94     time = True
95     font = pygame.font.SysFont("Comic Sans MS", 50)
96     small_font = pygame.font.SysFont("Comic Sans MS", 30)
97
98     replay_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2, 300, 50)
99     return_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2 + 100, 300, 50)

```

```

17 def start_game():
93     def time_screen(score, text_drawer, screen, clock):
101         while time:
102             for event in pygame.event.get():
103                 if event.type == pygame.QUIT:
104                     pygame.quit()
105                     quit()
106                 elif event.type == pygame.MOUSEBUTTONDOWN:
107                     mouse_pos = pygame.mouse.get_pos()
108                     if replay_button_rect.collidepoint(mouse_pos):
109                         return 'replay'
110                     elif return_button_rect.collidepoint(mouse_pos):
111                         return 'menu'
112
113             screen.fill((0, 0, 0))
114             text_drawer.draw_centered_text(f"Your Score: {score}", font, (237, 28, 36), SCREEN_WIDTH, SCREEN_HEIGHT - 200)
115             text = font.render("You ran out of time!", True, (237, 28, 36))
116             screen.blit(text, (SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 - text.get_height() // 2 - 400))
117
118             pygame.draw.rect(screen, (255, 0, 0), replay_button_rect)
119             text_drawer.draw_centered_text("Replay", small_font, (0, 0, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 50)
120
121             pygame.draw.rect(screen, (255, 0, 0), return_button_rect)
122             text_drawer.draw_centered_text("Return to Main Menu", small_font, (0, 0, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 250)
123
124             pygame.display.update()
125             clock.tick(60)
126
127     def live_screen(score, text_drawer, screen, clock):
128         live = True
129         font = pygame.font.SysFont("Comic Sans MS", 50)
130         small_font = pygame.font.SysFont("Comic Sans MS", 30)
131
132         replay_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2, 300, 50)
133         return_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2 + 100, 300, 50)
134
135         while live:
136             for event in pygame.event.get():
137                 if event.type == pygame.QUIT:
138                     pygame.quit()
139                     quit()
140                 elif event.type == pygame.MOUSEBUTTONDOWN:
141                     mouse_pos = pygame.mouse.get_pos()
142                     if replay_button_rect.collidepoint(mouse_pos):
143                         return 'replay'
144                     elif return_button_rect.collidepoint(mouse_pos):
145                         return 'menu'
146
147             screen.fill((0, 0, 0))
148             text_drawer.draw_centered_text(f"Your Score: {score}", font, (237, 28, 36), SCREEN_WIDTH, SCREEN_HEIGHT - 200)
149             text = font.render("You died!", True, (237, 28, 36))

```

```

17 def start_game():
127     def live_screen(score,text_drawer,screen,clock):
150         screen.blit(text,(SCREEN_WIDTH // 2 - text.get_width() // 2, SCREEN_HEIGHT // 2 - text.get_height() // 2 - 400 ))
151
152         pygame.draw.rect(screen, (255, 0, 0), replay_button_rect)
153         text_drawer.draw_centered_text("Replay", small_font, (0, 0, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 50)
154
155         pygame.draw.rect(screen, (255, 0, 0), return_button_rect)
156         text_drawer.draw_centered_text("Return to main menu", small_font, (0, 0, 0), SCREEN_WIDTH, SCREEN_HEIGHT + 250)
157
158         pygame.display.update()
159         clock.tick(60)
160
161     pause_img = pygame.image.load ('pause.png').convert_alpha()
162     coin_img = pygame.image.load('Coin.png').convert_alpha()
163     player_img = pygame.image.load('Guy.png').convert_alpha()
164     plat_1_img = pygame.image.load('plat_1.png').convert_alpha()
165     plat_2_img = pygame.image.load('plat_2.png').convert_alpha()
166     plat_3_img = pygame.image.load('plat_3.png').convert_alpha()
167     bullet_img = pygame.image.load('Bullet.png').convert_alpha()
168     start_img = pygame.image.load('Gun.png').convert_alpha()
169     enemy_img = pygame.image.load('Enemy.png').convert_alpha()
170     finish_line_img = pygame.image.load('finish_line.png').convert_alpha()
171     relic_img = pygame.image.load('Gun2.png').convert_alpha()
172
173     pause_button = CL.Button(0,150,pause_img)
174     player = CL.Player(player_img,100,100)
175     platforms = [
176         CL.Platform (plat_1_img,100,200),
177         CL.Platform (plat_2_img,220,270),
178         CL.Platform (plat_3_img,340,340),
179         CL.Platform (plat_3_img,1040,340),
180         CL.Platform (plat_3_img,740,340),
181         CL.Platform (plat_2_img,600,340)
182     ]
183     coins = [
184         CL.Coin(coin_img,200,100),
185         CL.Coin(coin_img,300,150),
186         CL.Coin(coin_img,450,200)
187     ]
188     bullets = []
189
190     start = CL.StartWeapon(start_img, player,bullet_img)
191     relic = CL.RelicWeapon(relic_img, player,bullet_img)
192
193     import main_menu
194     main_menu.player_weapon = database.get_weapon()
195
196     if main_menu.player_weapon == "Relic" and database.has_weapon("Relic"):
197         current_weapon = relic
198     else:

```

```

17 def start_game():
199     current_weapon = start
200     main_menu.player_weapon = "Start" # Ensure the default is Start if relic isn't owned
201
202     enemies = [CL.Enemy(enemy_img, 1040,240,3),
203               CL.Enemy(enemy_img, 340,240,3)]
204
205     finish_line = CL.FinishLine(finish_line_img, 1150,250)
206
207     is_shooting = False
208     current_frame = 0
209
210     run = True
211     while run:
212         app_settings = settings.load_settings()
213         if app_settings["sound"]:
214             start_weapon_sound = pygame.mixer.Sound("sounds/startShot.mp3")
215             relic_weapon_sound = pygame.mixer.Sound("sounds/relicShot.mp3")
216
217         current_frame += 1
218         clock.tick(60)
219         screen.fill((0,162,232))
220
221         seconds = (pygame.time.get_ticks() - start_ticks)//1000
222         remaining_time = total_time - (seconds * 1000)
223
224         minutes = remaining_time // 60000
225         seconds = (remaining_time % 60000)//1000
226
227         pygame.draw.rect(screen, (0,0,0), timer_rect)
228
229         time_text = f"{minutes}:{seconds:02d}"
230         text_drawer.draw_text(time_text, font2, (237,28,36), 10, 10)
231
232         if remaining_time <= 0:
233             database.insert_score(score)
234             result = time_screen(score, text_drawer, screen, clock) # Call the end screen function and check if replay is chosen
235             if result == 'replay':
236                 # Reset the game state for replay
237                 player = CL.Player(player_img, 100, 100)
238                 enemies = [CL.Enemy(enemy_img, 1040,240,3), CL.Enemy(enemy_img, 340,240,3)]
239                 coins = [CL.Coin(coin_img,200,100), CL.Coin(coin_img,300,150), CL.Coin(coin_img,450,200)]
240                 start = CL.StartWeapon(start_img, player,bullet_img)
241                 relic = CL.RelicWeapon(relic_img, player,bullet_img)
242
243                 # Correctly set the weapon only if the player actually owns Relic
244                 if database.has_weapon("Relic") and main_menu.player_weapon == "Relic":
245                     current_weapon = relic
246                 else:
247                     current_weapon = start

```

```

17 def start_game():
248     main_menu.player_weapon = "Start" # Reset to Start if Relic is not owned
249     score = 0
250     start_ticks = pygame.time.get_ticks() # Reset the timer
251     elif result == 'menu':
252         return
253
254     pause_button.draw()
255     if pause_button.detect():
256         pause_screen()
257
258     player.draw()
259
260     if player.update() or player.check_collision(platforms, enemies):
261         score += 30
262
263     if player.lives <= 0:
264         database.insert_score(score)
265         result = live_screen(score, text_drawer, screen, clock) # Call the end screen function and check if replay is chosen
266         if result == 'replay':
267             # Reset the game state for replay
268             player = CL.Player(player_img, 100, 100)
269             enemies = [CL.Enemy(enemy_img, 1040, 240, 3), CL.Enemy(enemy_img, 340, 240, 3)]
270             coins = [CL.Coin(coin_img, 200, 100), CL.Coin(coin_img, 300, 150), CL.Coin(coin_img, 450, 200)]
271             start = CL.StartWeapon(start_img, player, bullet_img)
272             relic = CL.RelicWeapon(relic_img, player, bullet_img)
273             # Correctly set the weapon only if the player actually owns Relic
274             if database.has_weapon("Relic") and main_menu.player_weapon == "Relic":
275                 current_weapon = relic
276             else:
277                 current_weapon = start
278                 main_menu.player_weapon = "Start" # Reset to Start if Relic is not owned
279             score = 0
280             start_ticks = pygame.time.get_ticks() # Reset the timer
281         elif result == 'menu':
282             return
283
284     current_weapon.draw(screen)
285
286     for platform in platforms:
287         platform.draw_platform()
288
289     for coin in coins:
290         coin.draw_coin()
291         if coin.check(player):
292             coins.remove(coin)
293             score += 50
294
295     for bullet in bullets:
296         bullet.update()

```

```

17 def start_game():
297     enemy_hit = bullet.check_collision(enemies)
298     if enemy_hit:
299         enemies.remove(enemy_hit)
300         bullets.remove(bullet)
301         score = score + 100
302     elif bullet.rect.x > SCREEN_WIDTH:
303         bullets.remove(bullet)
304     else:
305         bullet.draw(screen)
306
307     enemies = [enemy for enemy in enemies if enemy.update(player, platforms)]
308     for enemy in enemies:
309         enemy.draw(screen)
310
311     finish_line.draw(screen)
312
313     if finish_line.check(player): # Check if player reaches the finish line
314         database.insert_score(score)
315         result = end_screen(score, text_drawer, screen, clock) # Call the end screen function and check if replay is chosen
316         if result == 'replay':
317             # Reset the game state for replay
318             player = CL.Player(player_img, 100, 100)
319             enemies = [CL.Enemy(enemy_img, 1040, 240, 3), CL.Enemy(enemy_img, 340, 240, 3)]
320             coins = [CL.Coin(coin_img, 200, 100), CL.Coin(coin_img, 300, 150), CL.Coin(coin_img, 450, 200)]
321             start = CL.StartWeapon(start_img, player, bullet_img)
322             relic = CL.RelicWeapon(relic_img, player, bullet_img)
323             # Correctly set the weapon only if the player actually owns Relic
324             if database.has_weapon("Relic") and main_menu.player_weapon == "Relic":
325                 current_weapon = relic
326             else:
327                 current_weapon = start
328                 main_menu.player_weapon = "Start" # Reset to Start if Relic is not owned
329             score = 0
330             start_ticks = pygame.time.get_ticks() # Reset the timer
331         elif result == 'menu':
332             return
333
334     score_text = f"Score: {score}"
335     text_drawer.draw_text(score_text, font, (237, 28, 36), 150, 850)
336     lives_text = f"Lives: {player.lives}"
337     text_drawer.draw_text(lives_text, font, (237, 28, 36), 1550, 850)
338
339     for event in pygame.event.get():
340         if event.type == pygame.QUIT:
341             run = False
342         elif event.type == pygame.MOUSEBUTTONDOWN:
343             if event.button == 1 and not is_shooting:
344                 bullet = shoot_weapon(current_weapon, bullet_img, current_frame, app_settings)
345                 if bullet:

```

```

17  def start_game():
346      bullets.append(bullet)
347      is_shooting = True
348      elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
349          is_shooting = False
350      keys = pygame.key.get_pressed()
351      if keys[pygame.K_a]:
352          player.rect.x -= 5
353      if keys[pygame.K_d]:
354          player.rect.x += 5
355      if keys[pygame.K_p]:
356          pause_screen()
357      elif keys[pygame.K_SPACE] and player.on_ground: #only jumps if player is on solid ground
358          player.vel_y = -10
359
360      player.draw()
361      pygame.display.update()
362
363      pygame.quit()
364      quit()
365

```

CL.py

```
1 import pygame
2 import database
3 from pygame.locals import *
4 pygame.init()
5 #initialises pygame
6 SCREEN_WIDTH = 1850#sets the screen width in pixels
7 SCREEN_HEIGHT = 1000#sets the screen height in pixels
8 GRAVITY = 0.5
9 BULLET_SPEED = 10
10 screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))#creates a 1000 by 800 screen
11 pygame.display.set_caption("Treasure Chase")#gives the window the name "Treasure Chase"
12
13
14 class Button():#Creates the button class
15     def __init__(self, x, y, image):
16         self.image = image
17         self.rect = self.image.get_rect()#sets the image as a rectangle
18         self.rect.topleft = (x, y)#gets the coordinate of the top left pixel of the image
19         self.clicked = False
20
21     def draw(self): #allows the buttons to be drawn onto the screen
22         screen.blit(self.image, (self.rect.x, self.rect.y))
23
24     def detect(self):
25         action = False
26         pos = pygame.mouse.get_pos()
27         if self.rect.collidepoint(pos):
28             if pygame.mouse.get_pressed()[0] == 1 and self.clicked == False:
29                 self.clicked = True
30                 action = True
31             if pygame.mouse.get_pressed()[0] == 0:
32                 self.clicked = False
33         return action
34
35 class Text():#Creates the Text class
36     def __init__(self, screen):
37         self.screen = screen
38
39     def draw_text(self, text, font, text_col, x, y):#Allows the text to be drawn on the screen
40         img = font.render(text, True, text_col)
41         self.screen.blit(img, (x, y))
42
43     def draw_centered_text(self, text, font, text_col, screen_width, screen_height):
44         img = font.render(text, True, text_col)
45         x = screen_width // 2 - img.get_width() // 2
46         y = screen_height // 2 - img.get_height() // 2
47         self.screen.blit(img, (x, y))
48
49 class Player():
50     def __init__(self, image, x, y):
51         self.image = image
```

```

49 class Player():
50     def __init__(self, image, x, y):
51
52         self.rect = self.image.get_rect()
53         self.start_x = x
54         self.start_y = y
55         self.rect.topleft = (x, y)
56         self.vel_y = 0
57         self.lives = 5
58         self.on_ground = False #checks if player is on the ground
59
60     def update(self):
61         self.vel_y += GRAVITY
62         self.rect.y += self.vel_y
63         if self.rect.bottom >= SCREEN_HEIGHT:
64             self.rect.bottom = SCREEN_HEIGHT
65             self.vel_y = 0
66             if self.die():
67                 return True
68         return False
69
70     def die(self):
71         self.lives -= 1
72         self.respawn()
73         return True
74
75     def draw(self):
76         screen.blit(self.image, (self.rect.x, self.rect.y))
77
78     def check_collision(self, platforms, enemies):
79         self.on_ground = False #resets every time this function takes place
80         for platform in platforms:
81             if self.rect.colliderect(platform.rect) and self.vel_y > 0: # falling down
82                 self.rect.bottom = platform.rect.top
83                 self.vel_y = 0
84                 self.on_ground = True #player is on solid ground
85         for enemy in enemies:
86             if self.rect.colliderect(enemy.rect):
87                 self.lives -= 1
88                 self.respawn()
89                 return True
90         return False
91
92     def respawn(self):
93         self.rect.topleft = (self.start_x, self.start_y)
94         self.vel_y = 0
95
96 class Platform():
97     def __init__(self, image, x, y):
98         self.image = image
99         self.rect = self.image.get_rect()

```

```

96 class Platform():
97     def __init__(self, image, x, y):
100         self.rect.topleft = (x, y)
101
102     def draw_platform(self):
103         screen.blit(self.image, (self.rect.x, self.rect.y))
104
105 class Coin():
106     def __init__(self, image, x, y):
107         self.image = image
108         self.rect = self.image.get_rect()
109         self.rect.topleft = (x, y)
110     def draw_coin(self):
111         screen.blit(self.image, (self.rect.x, self.rect.y))
112     def check(self, player):
113         if self.rect.colliderect(player.rect):
114             database.insert_coins(1)
115             return True
116         return False
117
118 class Bullet:
119     def __init__(self, image, x, y):
120         self.image = image
121         self.rect = self.image.get_rect()
122         self.rect.center = (x, y)
123         self.speed = BULLET_SPEED
124
125     def update(self):
126         self.rect.x += self.speed # Moves to the right
127
128     def draw(self, screen):
129         screen.blit(self.image, (self.rect.x, self.rect.y)) # Draws the bullet
130
131     def check_collision(self, enemies):
132         for enemy in enemies:
133             if self.rect.colliderect(enemy.rect):
134                 return enemy
135         return None
136
137
138 class Weapon():
139     def __init__(self, image, player, bullet_image):
140         self.image = image
141         self.player = player
142         self.bullet_image = bullet_image
143         self.last_shot = 0
144         self.cooldown = 20
145         self.rect = self.image.get_rect()
146         self.rect.topleft = (170, 125)
147         self.initial_position_set = False

```



```

138 class Weapon():
139     def draw(self, screen):
140         if not self.initial_position_set:
141             self.rect.topleft = (170, 125)
142             self.initial_position_set = True
143         else:
144             self.rect.centerx = self.player.rect.centerx + 60
145             self.rect.y = self.player.rect.top + 20
146             screen.blit(self.image, self.rect.topleft)
147
148     def shoot(self, current_frame, bullet_image):
149         if current_frame - self.last_shot >= self.cooldown:
150             bullet = Bullet(self.bullet_image, self.player.rect.centerx + 60, self.player.rect.centery - 5)
151             self.last_shot = current_frame
152             return bullet
153         return None
154
155 class StartWeapon(Weapon):
156     def __init__(self, image, player, bullet_image):
157         super().__init__(image, player, bullet_image)
158         self.name = "Start"
159
160 class RelicWeapon(Weapon):
161     def __init__(self, image, player, bullet_image):
162         super().__init__(image, player, bullet_image)
163         self.cooldown = 15
164         self.name = "Relic"
165
166     def shoot(self, current_frame, bullet_image):
167         if current_frame - self.last_shot >= self.cooldown:
168             bullet = Bullet(self.bullet_image, self.player.rect.centerx + 60, self.player.rect.centery - 5)
169             bullet.piercing = True # Ensure the Bullet class supports this
170             self.last_shot = current_frame
171             return bullet
172         return None
173
174 class Enemy():
175     def __init__(self, image, x, y, speed):
176         self.image = image
177         self.rect = self.image.get_rect()
178         self.rect.topleft = (x, y)
179         self.speed = speed
180         self.vel_y = 0
181         self.on_platform = False
182
183     def update(self, player, platforms):
184         if not self.on_platform:
185             self.vel_y += GRAVITY
186             self.rect.y += self.vel_y
187
188 class Enemy():
189     def update(self, player, platforms):
190         #calculate direction towards player
191         direction_x = player.rect.centerx - self.rect.centerx
192         if direction_x != 0:
193             direction_x = direction_x // abs(direction_x)
194         #move towards player
195         self.rect.x += direction_x * self.speed
196         self.on_platform = False
197         for platform in platforms:
198             if self.rect.colliderect(platform.rect) and self.vel_y > 0:
199                 self.rect.bottom = platform.rect.top
200                 self.vel_y = 0
201                 self.on_platform = True
202         # Check if the player is at a higher position or there's no platform in front
203         if not any(platform.rect.collidepoint(self.rect.centerx + direction_x * self.speed, self.rect.bottom + 1) for platform in platforms):
204             if self.on_platform and self.vel_y == 0: # Jump only if on platform and not already jumping/falling
205                 self.vel_y = -10 # Jump velocity
206         if self.rect.right >= SCREEN_WIDTH or self.rect.left <= 0:
207             self.speed = -self.speed
208         if self.rect.top > SCREEN_HEIGHT:
209             return False
210         return True
211
212     def draw(self, screen):
213         screen.blit(self.image, self.rect.topleft)
214
215 class FinishLine():
216     def __init__(self, image, x, y):
217         self.image = image
218         self.rect = self.image.get_rect()
219         self.rect.topleft = (x, y)
220
221     def draw(self, screen):
222         screen.blit(self.image, self.rect.topleft)
223
224     def check(self, player):
225         return self.rect.colliderect(player.rect)
226
227
228
229
230
231
232
233
234

```

Settings.py

```
1 import json
2
3 # Default settings
4 default_settings = {
5     "sound": True,
6     "weapon_sounds": {
7         "Start": "sounds/startShot.mp3",
8         "Relic": "sounds/relicShot.mp3"
9     }
10 }
11
12 # Load settings
13 def load_settings():
14     try:
15         with open("settings.json", "r") as file:
16             return json.load(file)
17     except (FileNotFoundError, json.JSONDecodeError):
18         # Return default settings if the file is missing or corrupted
19         save_settings(default_settings)
20         return default_settings
21
22 # Save settings
23 def save_settings(settings):
24     with open("settings.json", "w") as file:
25         json.dump(settings, file, indent=4)
26
27 settings = load_settings() # Load settings at the start
```

Database.py

```
1 import sqlite3
2 con = sqlite3.connect("treasureChase_data.db")
3 cur = con.cursor()
4 cur.execute("CREATE TABLE IF NOT EXISTS high_scores(score INT)")
5 cur.execute("CREATE TABLE IF NOT EXISTS coin_total(coins INT)")
6 cur.execute("CREATE TABLE IF NOT EXISTS player_data(weapon TEXT)")
7 cur.execute("CREATE TABLE IF NOT EXISTS owned_weapons (weapon TEXT UNIQUE)")
8
9 #-----
10
11 def insert_score(score):
12     cur.execute("INSERT INTO high_scores (score) VALUES (?)", (score,))
13     con.commit()
14
15 #-----
16
17 def insert_coins(coins):
18     cur.execute("INSERT INTO coin_total (coins) VALUES (?)", (coins,))
19     con.commit()
20
21 def get_coin_total():
22     cur.execute("SELECT SUM(coins) FROM coin_total")
23     total_coins = cur.fetchone()[0]
24     if total_coins is None:
25         total_coins = 0
26     return total_coins
27
28 #-----
29
30 def set_weapon(weapon_name):
31     cur.execute("DELETE FROM player_data") # Clear previous selection
32     cur.execute("INSERT INTO player_data (weapon) VALUES (?)", (weapon_name,))
33     con.commit()
34
35 def get_weapon():
36     cur.execute("SELECT weapon FROM player_data")
37     result = cur.fetchone()
38     return result[0] if result else "Start" # Default to "Start" if no value is stored
39
40 def purchase_weapon(weapon_name):
41     #stores purchased weapon into database
42     cur.execute("INSERT OR IGNORE INTO owned_weapons (weapon) VALUES (?)", (weapon_name,))
43
44 def has_weapon(weapon_name):
45     #checks if weapon is already purchased
46     cur.execute("SELECT weapon FROM owned_weapons WHERE weapon = ?", (weapon_name,))
47     return cur.fetchone() is not None # Returns True if the weapon is owned, False otherwise
48
```